

CHULA ENGINEERING
Foundation toward Innovation
COMPUTER

Data Mining Group
Machine Intelligence and Knowledge Discovery Lab
Chulalongkorn University

# Introduction to Deep Learning (CNN)


Prof. Peerapon Vateekul, Ph.D.
Department of Computer Engineering,
Faculty of Engineering, Chulalongkorn University
Peerapon.v@chula.ac.th
www.cp.eng.chula.ac.th/~peerapon/

# Outline

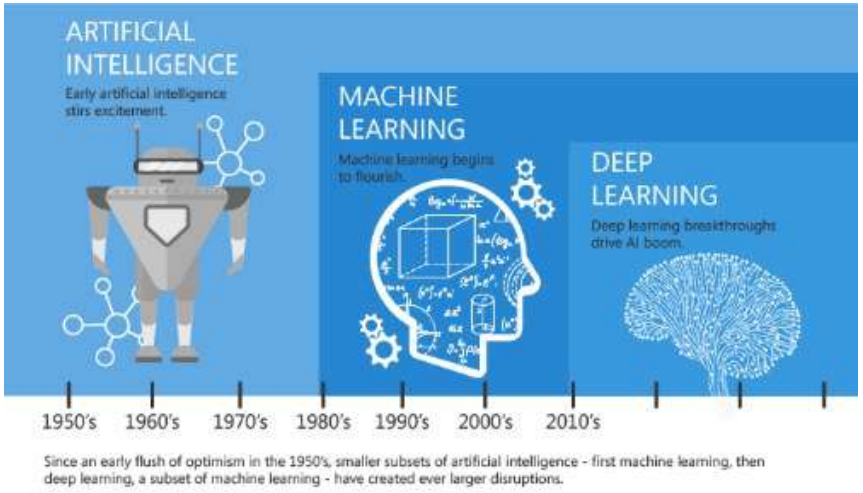
- Introduction to Deep Learning
- Convolutional Neural Networks (CNN) [Imaging Task]
- Pretrained Models
- Beyond CNN
- Imaging Tasks
  - Image Classification
  - Object Detection
  - Semantic Segmentation

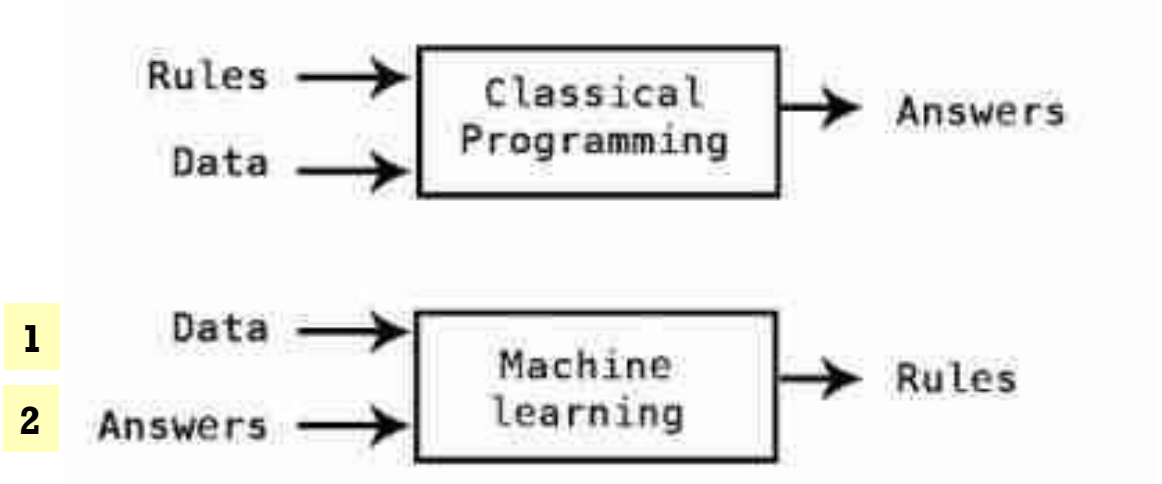


# Introduction to Deep Learning

# AI = Automation

- 1) Rule-based AI (Symbolic AI)
- 2) Machine Learning

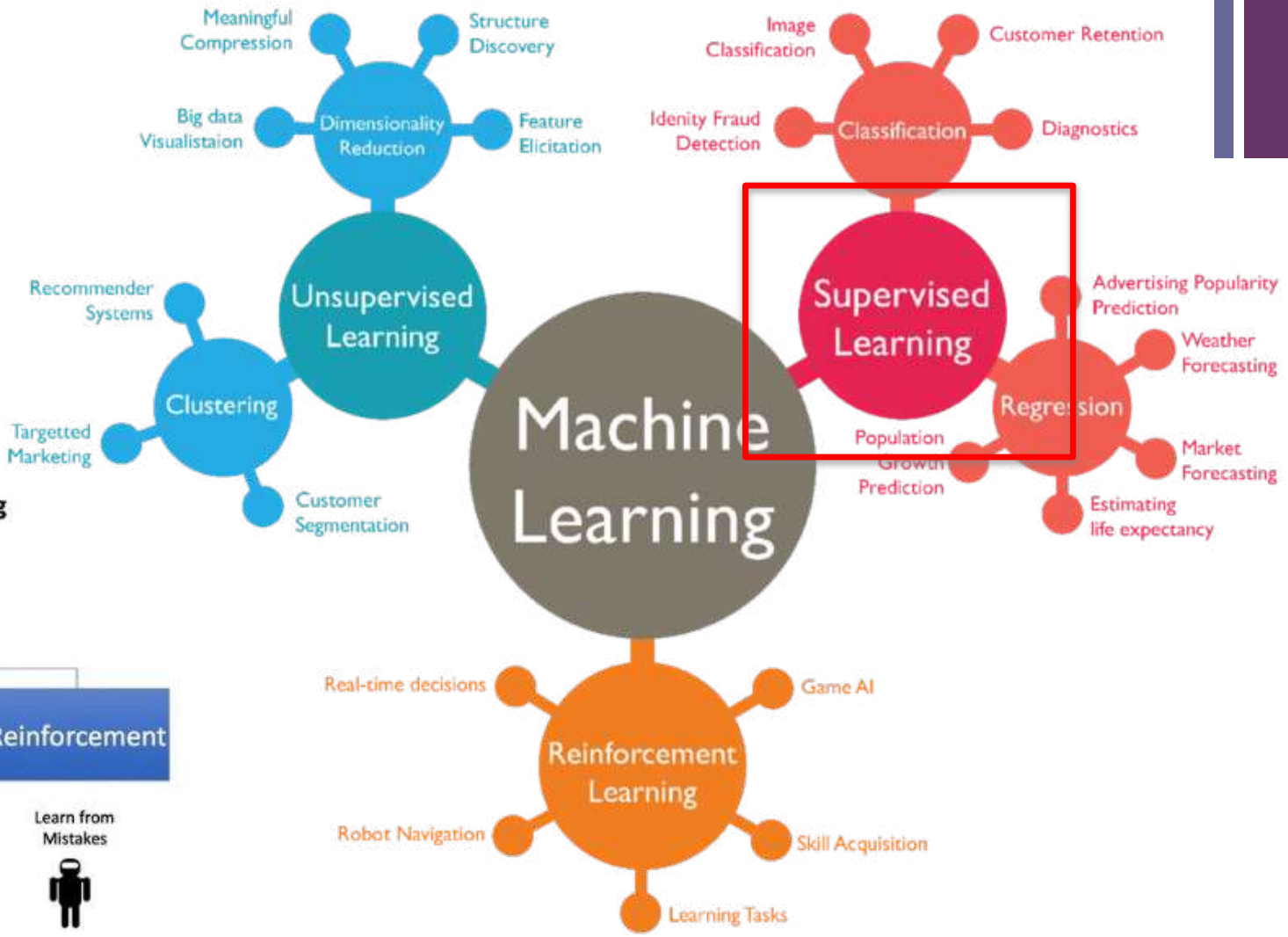


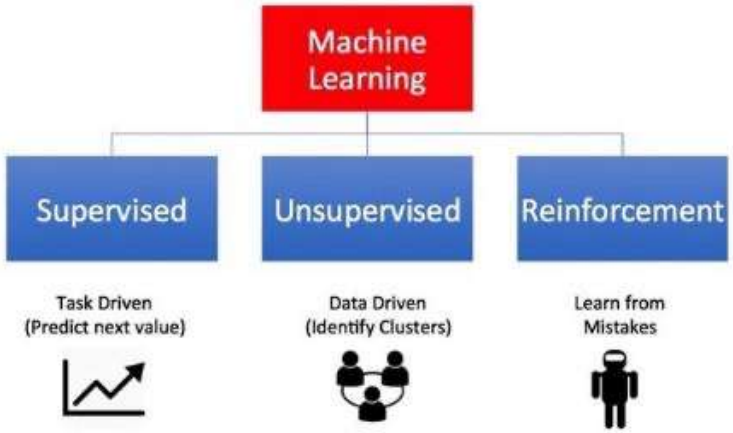

https://mc.ai/machine-learning-basics-artificial-intelligence-machine-learning-and-deep-learning/

# Machine Learning






**Types of Machine Learning**



# Task1: Supervised learning

**Handcrafted features**

Training Data




| | inputs | | | target |
|---|---|---|---|---|
| **Age** | **Gender** | **BodyTemp** | **Cough** | **Corona** |
| 12 | Female | 37 | Yes | Yes |
| 35 | Female | 39 | No | Yes |
| 32 | Male | 38 | Yes | No |

Testing Data



| **Age** | **Gender** | **BodyTemp** | **Cough** | **Corona** |
|---|---|---|---|---|
| 25 | Male | 40 | No | ? |

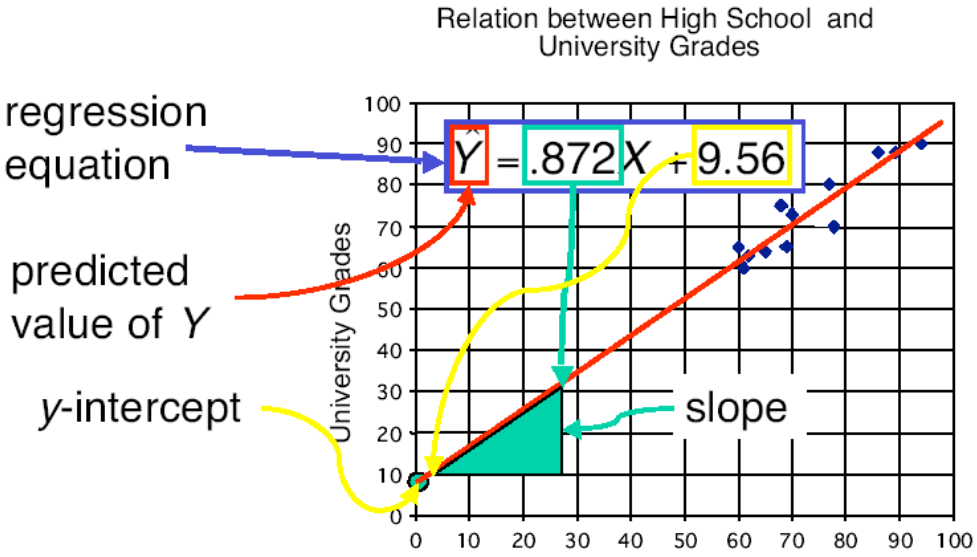
Application: Corona Prediction

# Prediction algorithms

- Decision Tree
- (Logistic) Regression
- kNN
- Support Vector Machine
- Neural Networks (NN)
- Deep Learning



# Regression – Linear Relationship




weight, coefficient

$$\underset{\text{target}}{\hat{y}} = \underset{\text{intercept}}{\hat{w}_0} + \hat{w}_1 x_1 + \hat{w}_2 x_2$$

input

- The least square method aims to minimize the following term

$$\sum_{\substack{training \\ data}} (y_i - \hat{y}_i)^2$$

# Logistic Regression (cont.)
## Linear Relationship

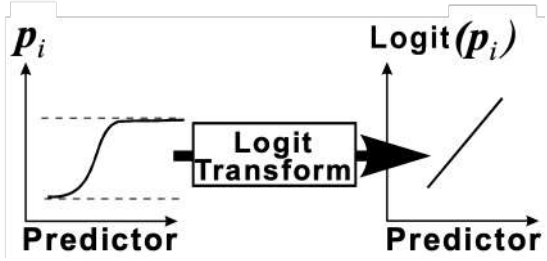
Training Data




| inputs | | | | target |
|---|---|---|---|---|
| **Age** | **Gender** | **BodyTemp** | **Cough** | **Corona** |
| 12 | Female (0) | 37 | Yes (1) | Yes |
| 35 | Female (0) | 39 | No (0) | Yes |
| 32 | Male (1) | 38 | Yes (1) | No |

Logit_score = w0 + w1*Age + w2*Gender + w3*Temp + w4*Cough
Logit_score = 0.01 – 0.3*Age +0.2*Gender + 0.2*Temp + 0.9*Cough

**Example**
Logit_score = 0.01 – 0.3*12 +0.2*0 + 0.2*37 + 0.9*1 = 4.71
prob = 0.9911 → "Yes"

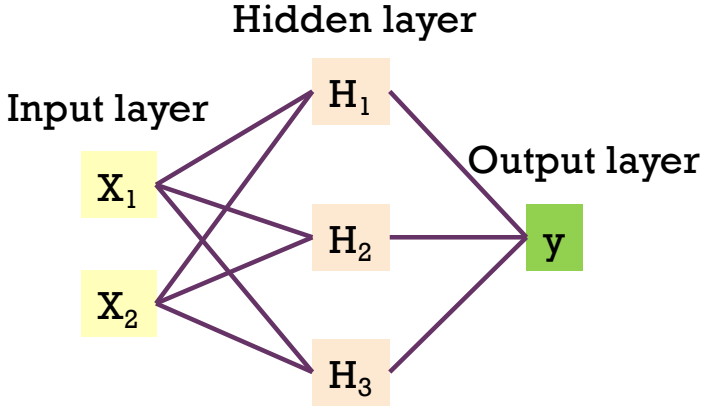



$$\hat{p} = \frac{1}{1 + e^{-\text{logit}(\hat{p})}}$$

Application: Corona Prediction

# Neural Networks (universal approximator)

## Non-linear relationship

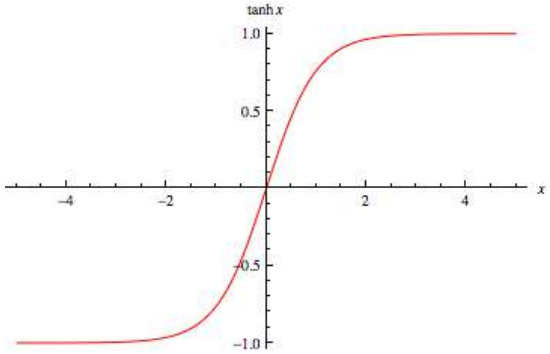



$$log\left(\frac{\hat{p}}{1-\hat{p}}\right) = \hat{w}_0 + \hat{w}_1 H_1 + \hat{w}_2 H_2 + \hat{w}_3 H_3$$

$$H_1 = tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

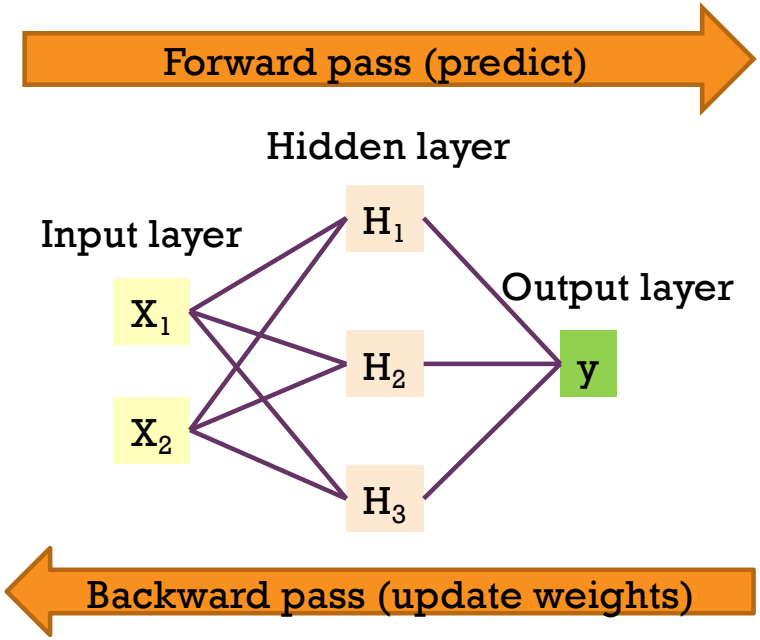



Stop when?
- Converge (no change in loss)
- Max epochs

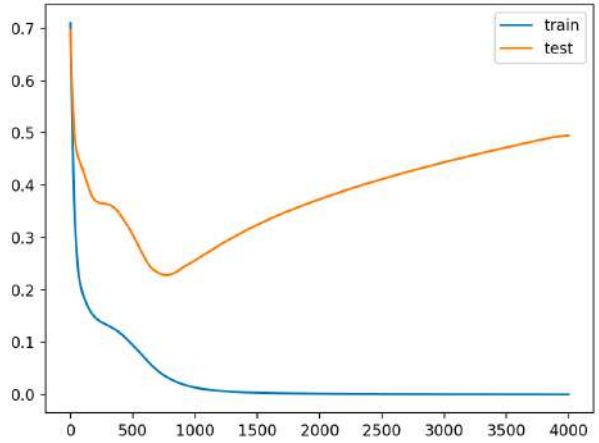
Important Params:
- #hidden units, #hidden layers
- Learning rate, Momentum, decay
- Seed number, etc.

# Neural Networks (cont.): Training

## Non-linear relationship




| Age | Income | Gender | Province | Corona |
|---|---|---|---|---|
| 25 | 25,000 | Female | Bangkok | Yes |
| 35 | 50,000 | Female | Nontaburi | Yes |
| 32 | 35,000 | Male | Bangkok | No |



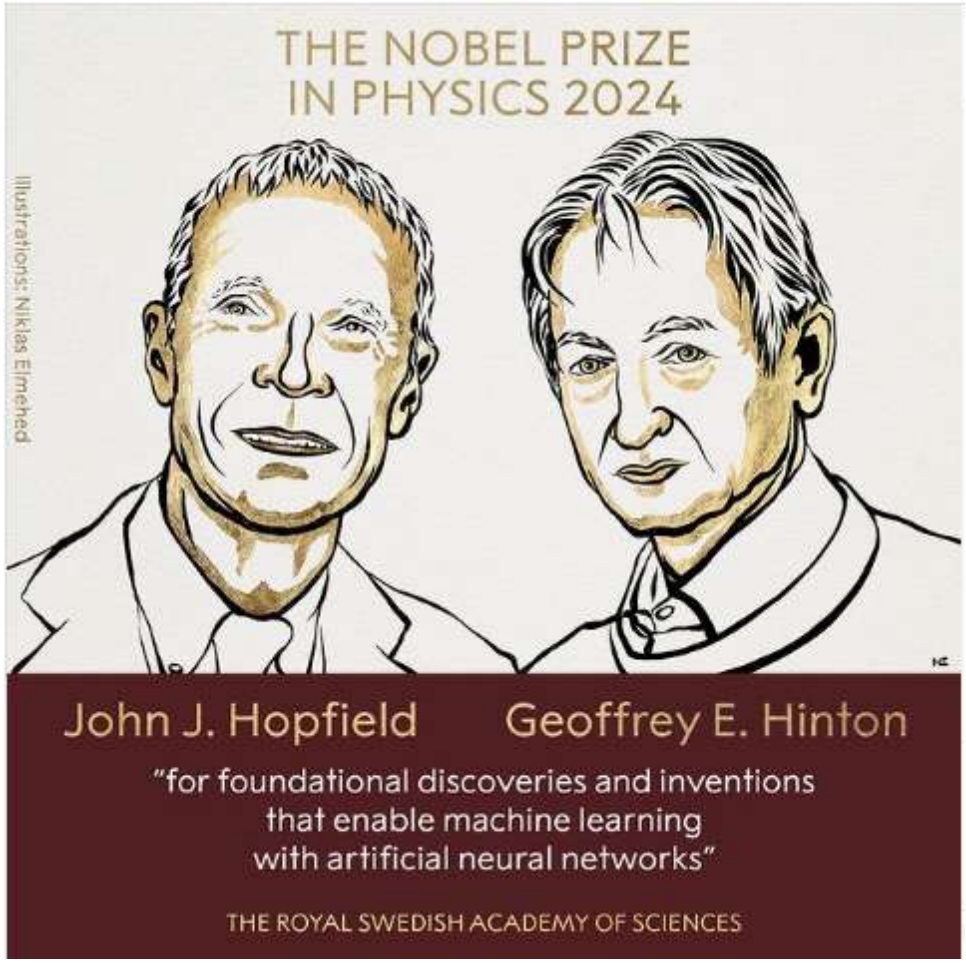

$$log\left(\frac{\hat{p}}{1-\hat{p}}\right) = \hat{w}_0 + \hat{w}_1 H_1 + \hat{w}_2 H_2 + \hat{w}_3 H_3$$

$$H_1 = tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

# Nobel Prize to Backpropagration's Inventors

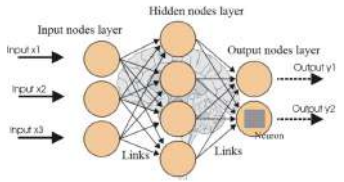



8 October 2024

The Royal Swedish Academy of Sciences has decided to award the Nobel Prize in Physics 2024 to

**John J. Hopfield**
Princeton University, NJ, USA

**Geoffrey E. Hinton**
University of Toronto, Canada




*"for foundational discoveries and inventions that enable machine learning with artificial neural networks"*

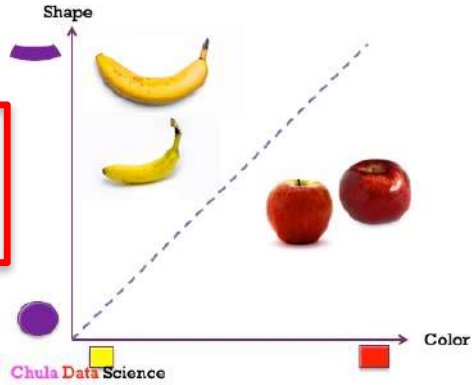
## They trained artificial neural networks using physics

**John Hopfield** invented a network that uses a method for saving and recreating patterns. We can imagine the nodes as pixels. The *Hopfield network* utilises physics that describes a material's characteristics due to its atomic spin – a property that makes each atom a tiny magnet.

**Geoffrey Hinton** used the Hopfield network as the foundation for a new network that uses a different method: the *Boltzmann machine*. This can learn to recognise characteristic elements in a given type of data.

# Handcrafted features

| Age | Income | Gender | Province | Corona |
|---|---|---|---|---|
| 25 | 25,000 | Female | Bangkok | Yes |
| 35 | 50,000 | Female | Nontaburi | Yes |
| 32 | 35,000 | Male | Bangkok | No |




## Can we still tell the features (columns)?

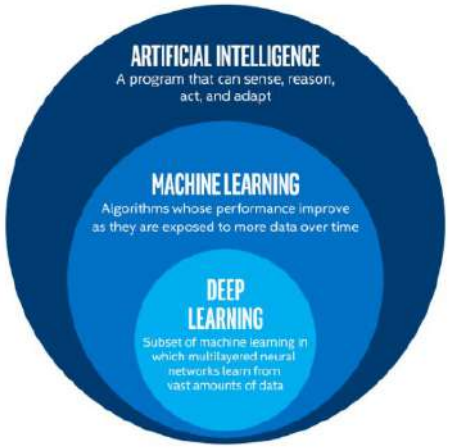


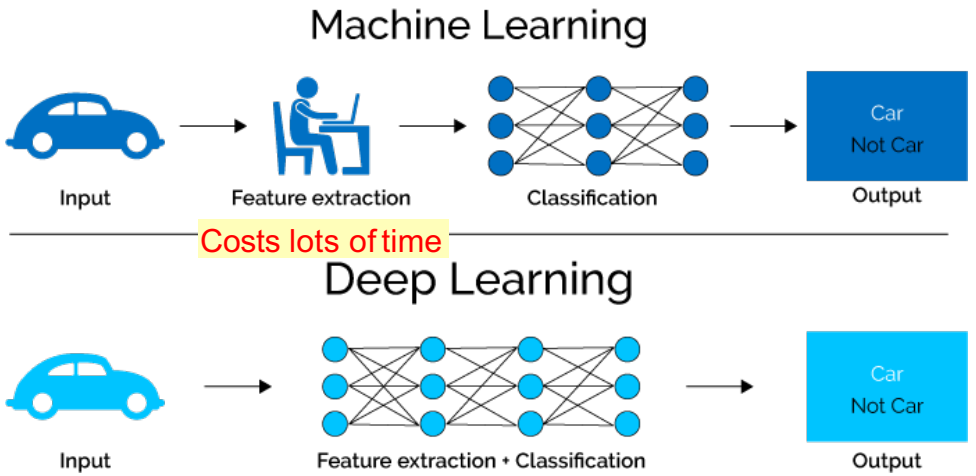


# What is Deep Learning (DL)?

Part of the machine learning field of learning representations of data. Exceptional effective at learning patterns.

Utilizes learning algorithms that derive meaning out of data by using a hierarchy of multiple layers that mimic the neural networks of our brain.



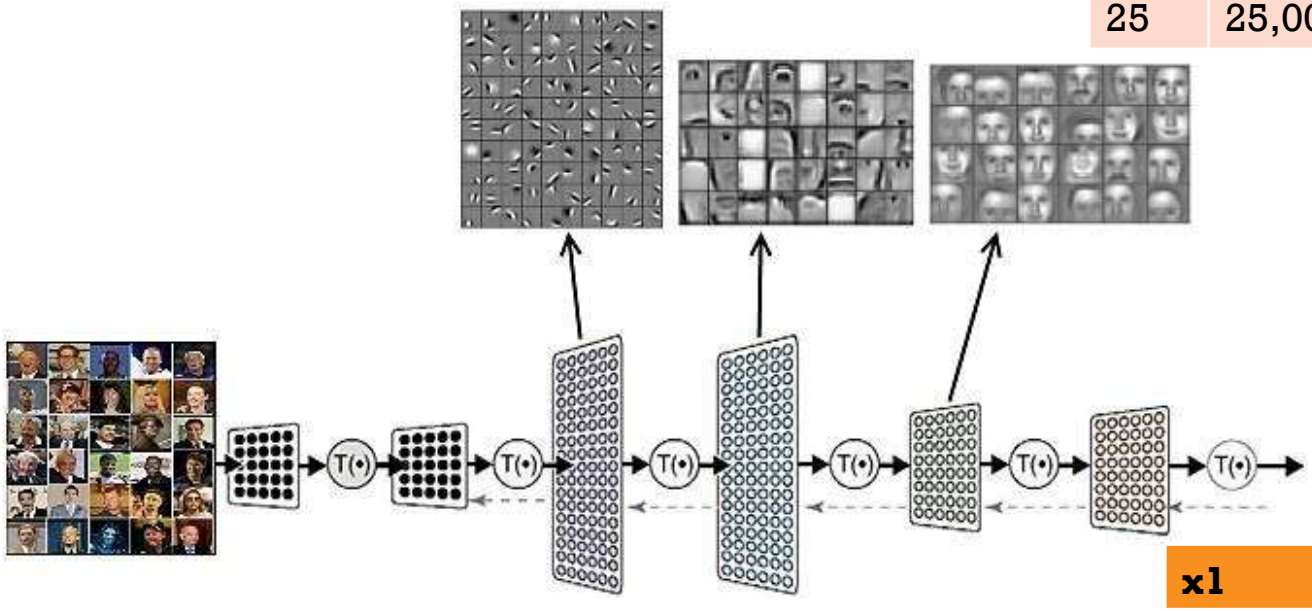
# Deep Learning – Basics (cont.)

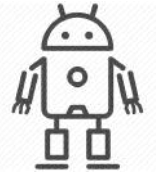
What did it learn?

A deep neural network consists of a hierarchy of layers, whereby each layer transforms the input data into more abstract representations (e.g., edge -> nose -> face). The output layer combines those features to make predictions.



| Age | Income | Gender | Province | Corona |
|---|---|---|---|---|
| 25 | 25,000 | Female | Bangkok | Yes |



| x1 | x2 | x3 | x4 | Corona |
|---|---|---|---|---|
| 0.7 | 0.2 | -0.5 | -0.1 | Yes |

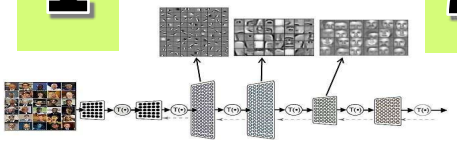
# Deep Learning Application

Speech Recognition

Computer Vision

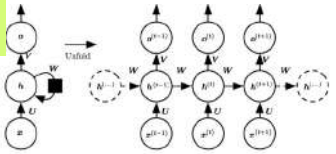
Natural Language Processing

1



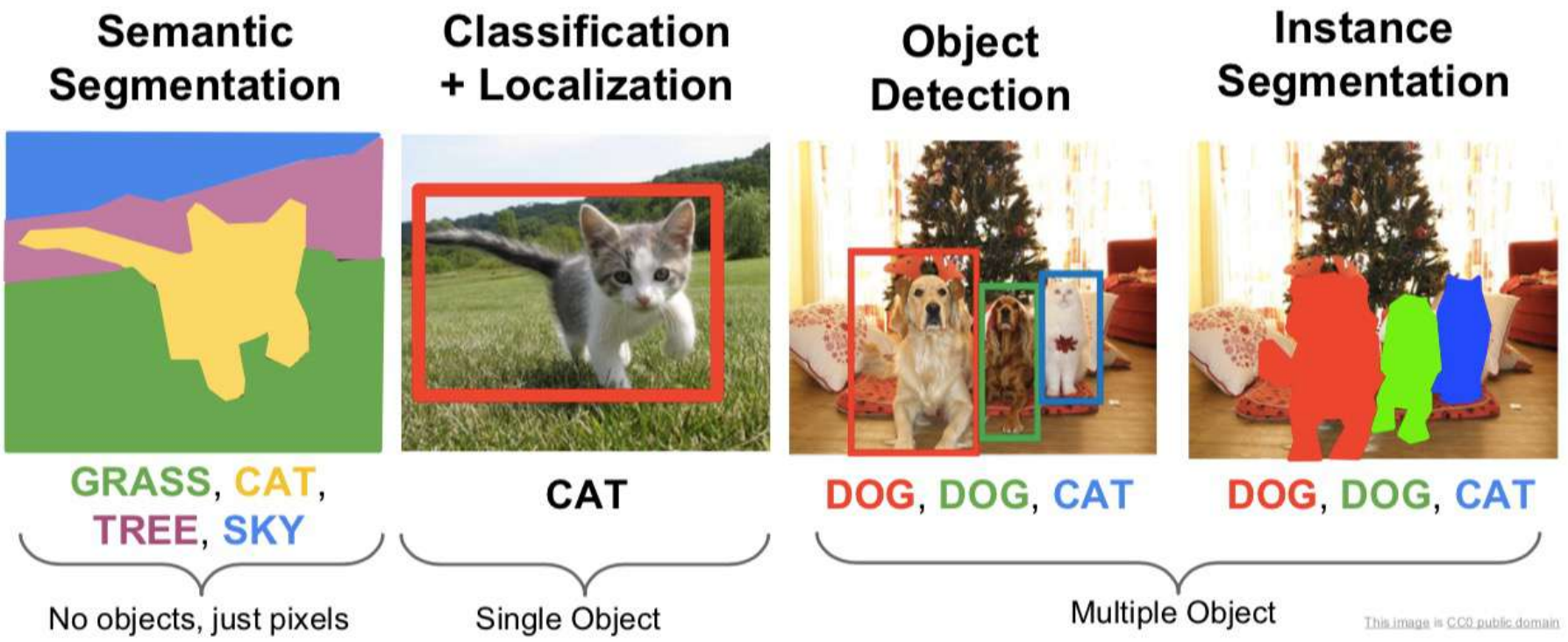
CNN

2



RNN

# Type of image tasks



# Face Recognition

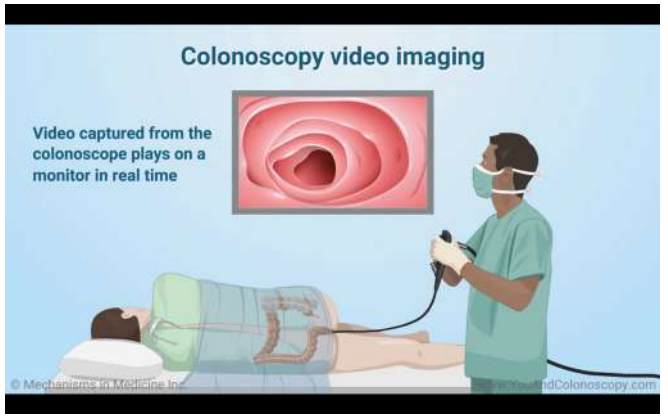


# Smart Medical Diagnosis

CHULA ENGINEERING
Foundation toward Innovation

King Chulalongkorn Memorial Hospital
The Thai Red Cross Society






+

# Convolutional Neural Networks (CNN)

IMAGENET www.image-net.org

The **ImageNet** project is a large visual database designed for use in visual object recognition software research. Over **14M** URLs of images have been hand-annotated by ImageNet to indicate what objects are pictured on **22K** categories.

IMAGENET Large Scale Visual Recognition Challenge

ILSVRC

The Image Classification Challenge:

1,000 object classes

1,431,167 images

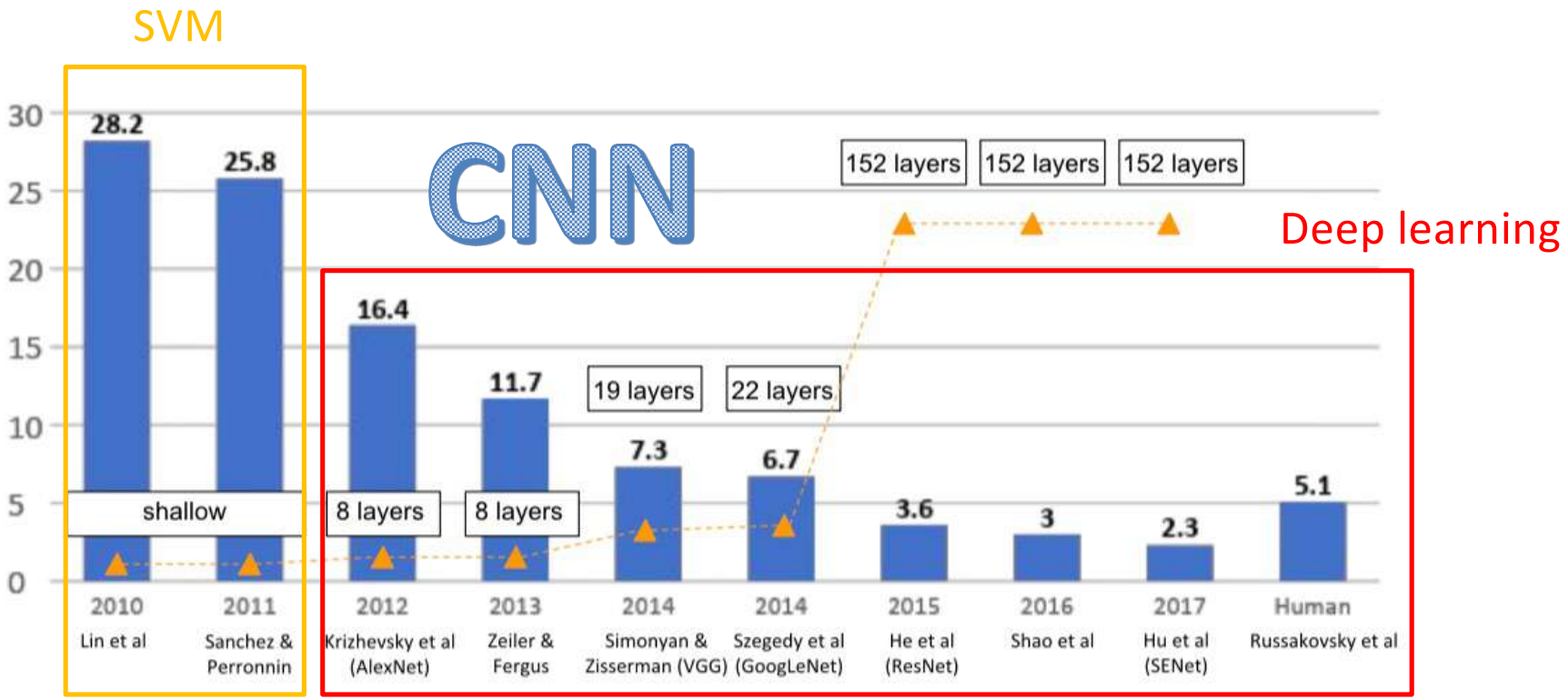
ImageNet 2017 is the last challenge.





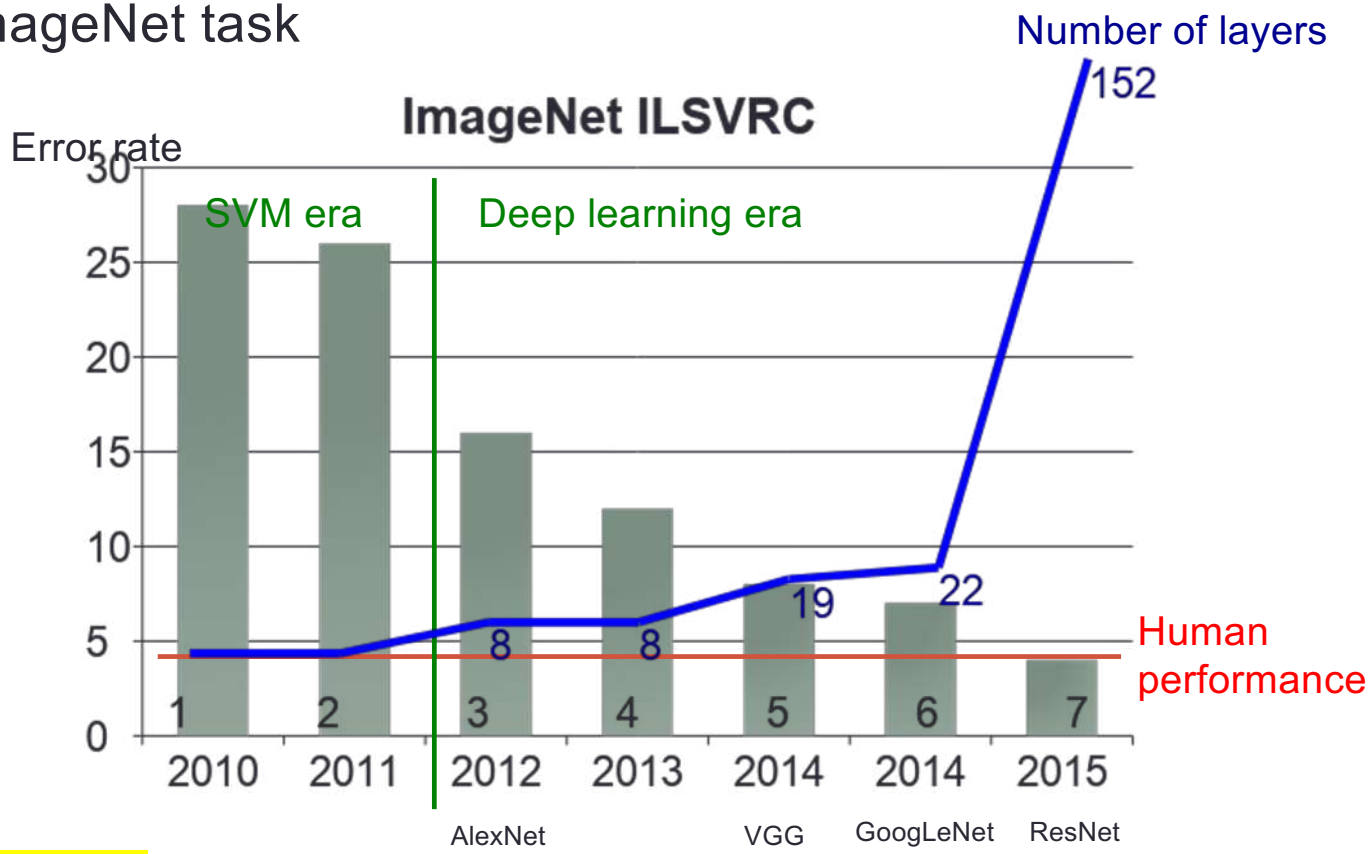

21

# ImageNet Large Scale Visual Recognition Challenge (ILSVRC winners): From SVM to Deep Learning



# Wider and deeper networks (Beyond Human)

- ImageNet task

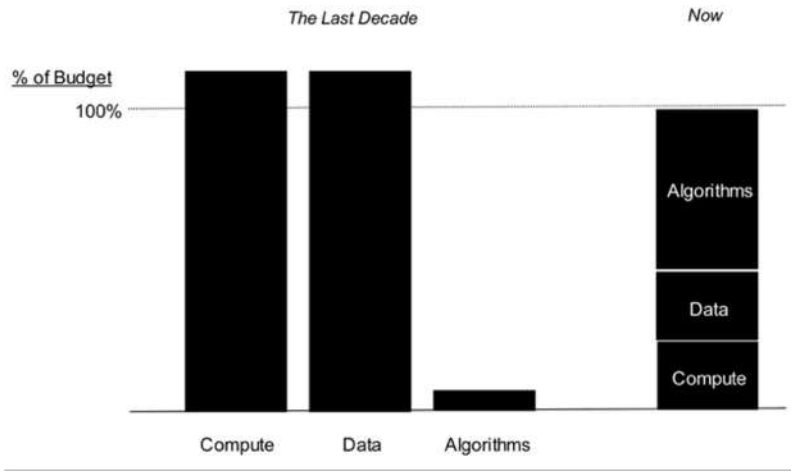



Based on the slide of Aj.Ekapol

Net Large Scale Visual Recognition Challenge, 2014 https://arxiv.org/abs/1409.0575

# Why now

- Neural Networks has been around since 1990s
- Big data – DNN can take advantage of large amounts of data better than other models
- GPU – Enable training bigger models possible
- Deep – Easier to avoid bad local minima when the model is large




Based on the slide of Aj.Ekapol /www.kdnuggets.com/2017/06/practical-guide-machine-learning-understand-differentiate-apply.html

# Application 1: Basic Image Processing

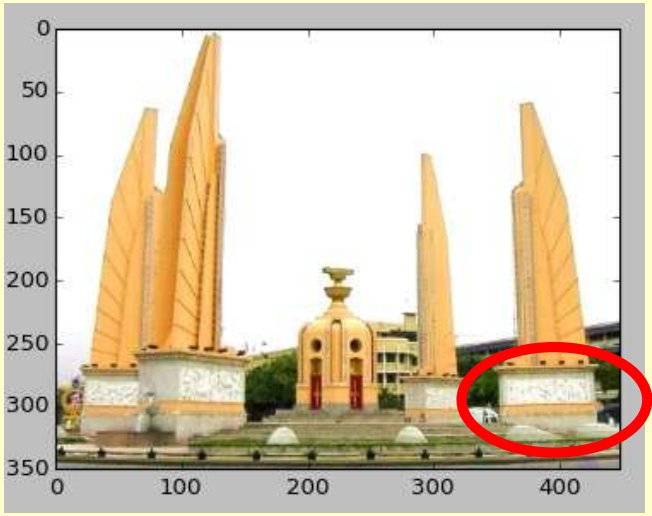
https://softwaredevelopmentperestroika.wordpress.com/2014/02/11/image-processing-with-python-numpy-scipy-image-convolution/

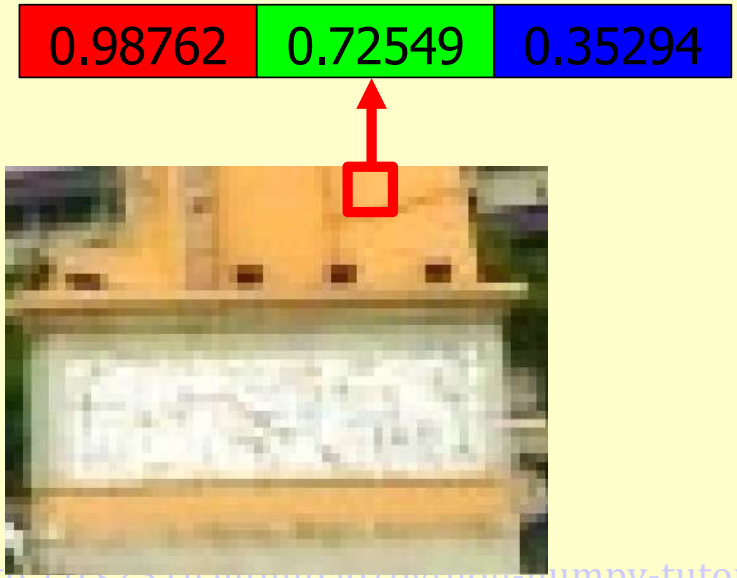
# Display an image in matplotlib

```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

image = mpimg.imread('monument.png')
plt.imshow(image)

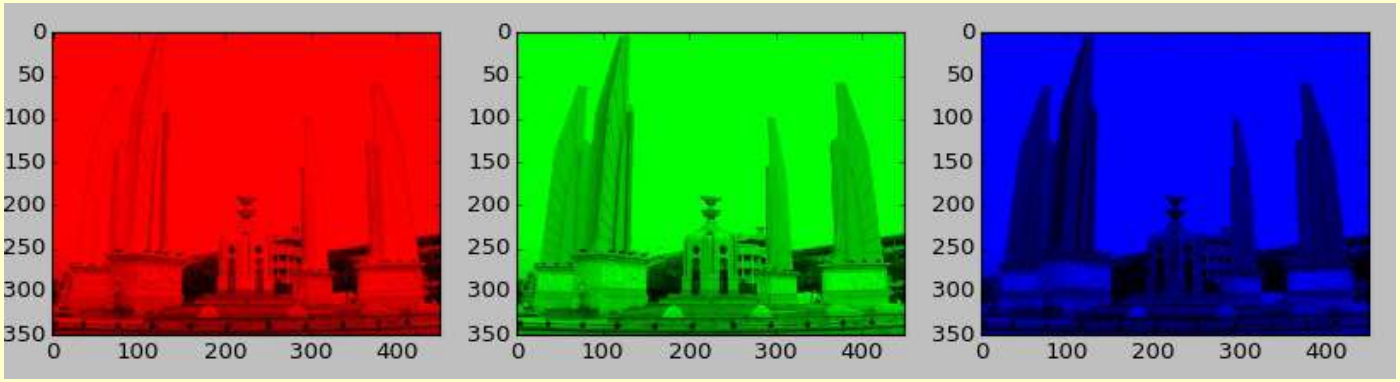
plt.show()
```



# It is an array of R,G,B (3 channels)

```
>>> image.shape
(350, 449, 3)
```



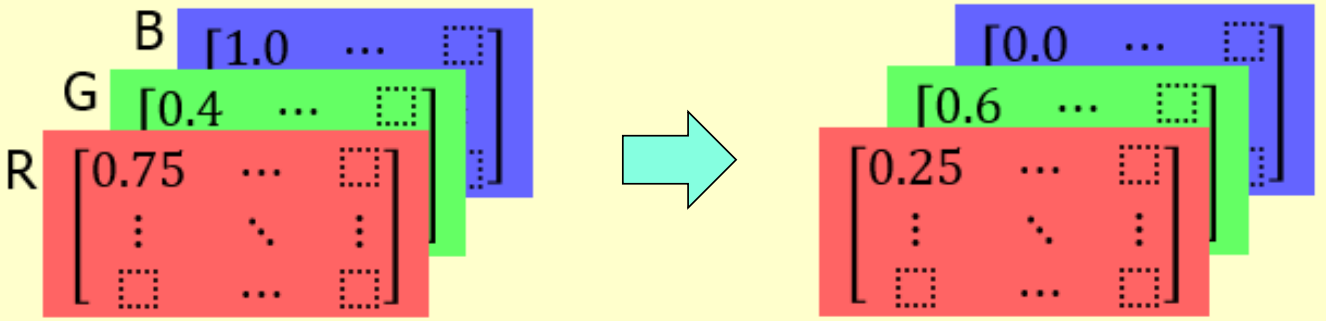

Red
image[:,:,0]

Green
image[:,:,1]

Blue
image[:,:,2]

# Basic Image Processing

- image = mpimg.imread('monument.png')
- image.shape → (350, 449, 3)
- image = 1 – image (broadcast & element-wise op.)




How's it changed?

# Basic Image Processing (1): Negative Image

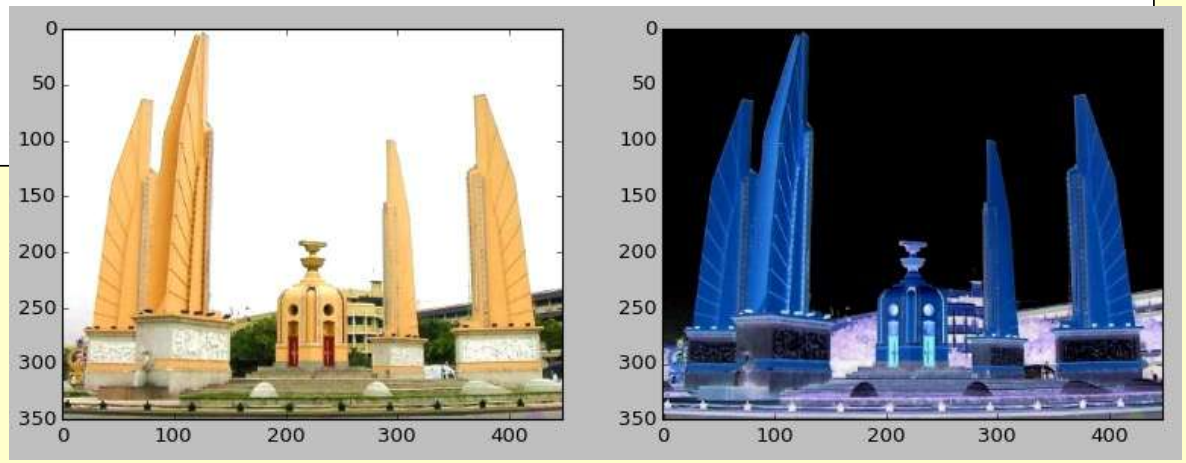
```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

image = mpimg.imread('monument.png')
plt.subplot(1, 2, 1)
plt.imshow(image)

negative = 1 - image
plt.subplot(1, 2, 2)
plt.imshow(negative)

plt.show()
```

1 – image = Invert Tone
(White → Black, Yellow → Blue, ...)
*broadcast & element-wise subtract*

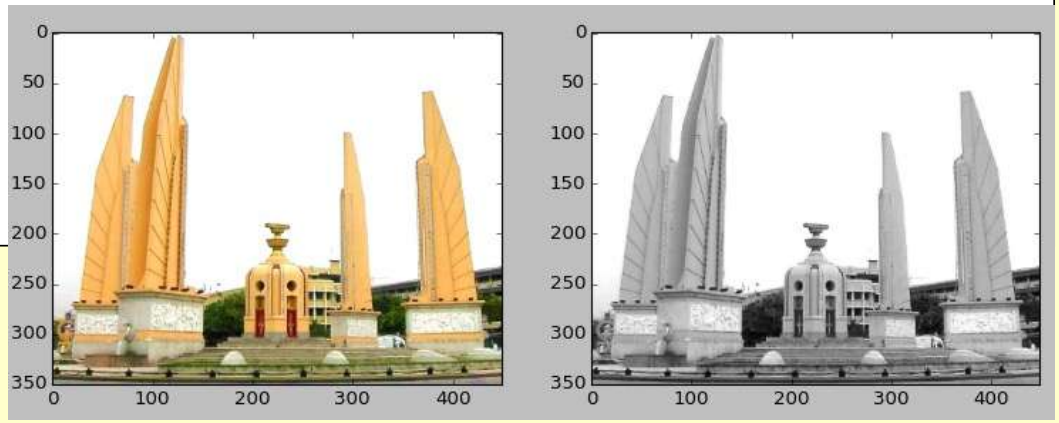


# Basic Image Processing (2): Grayscale Image

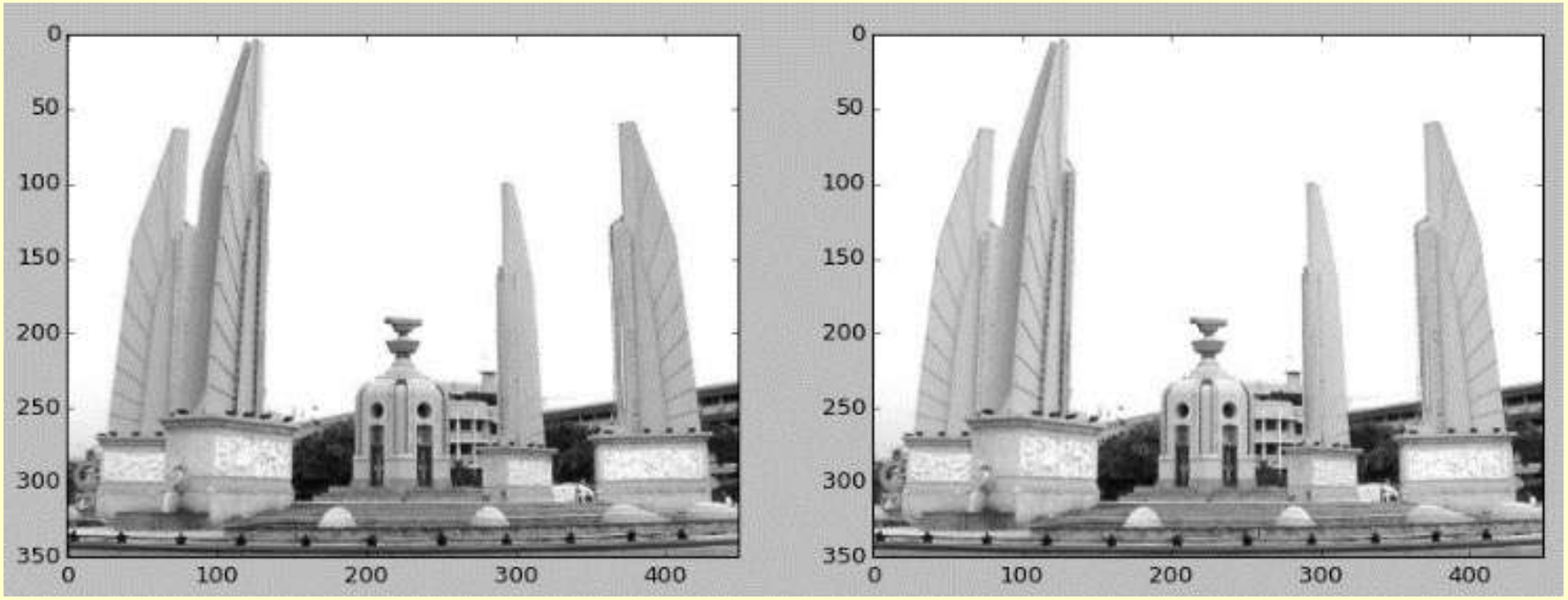
```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

image = mpimg.imread('monument.png')
plt.subplot(1, 2, 1)
plt.imshow(image)
gray = np.ndarray(image.shape)
gray[:,:,0] = \
gray[:,:,1] = \
gray[:,:,2] = (image[:,:,0]+image[:,:,1]+image[:,:,2]) / 3
plt.subplot(1, 2, 2)
plt.imshow(gray)

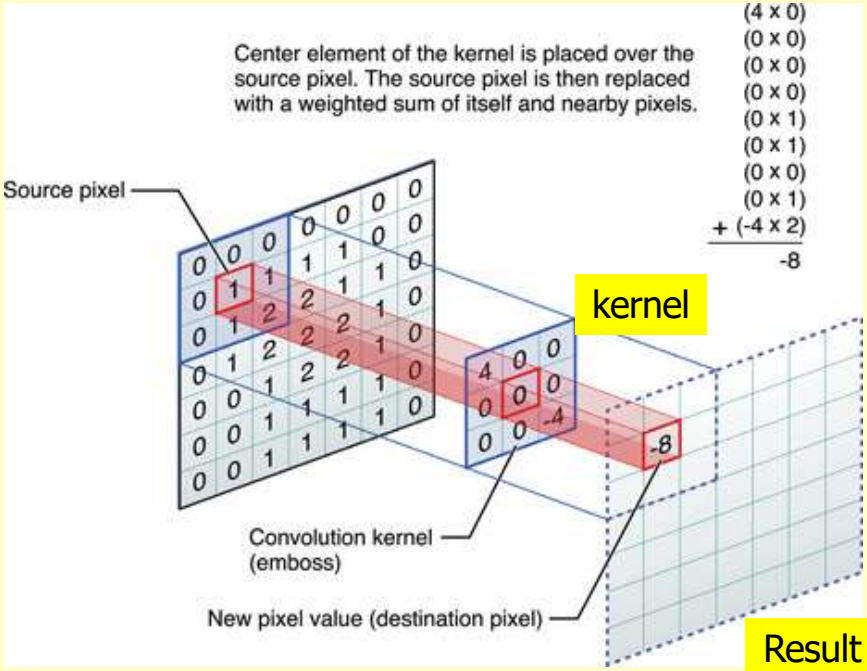
plt.show()
```



# Another Grayscale Image: 0.299*R + 0.587*G + 0.114*B

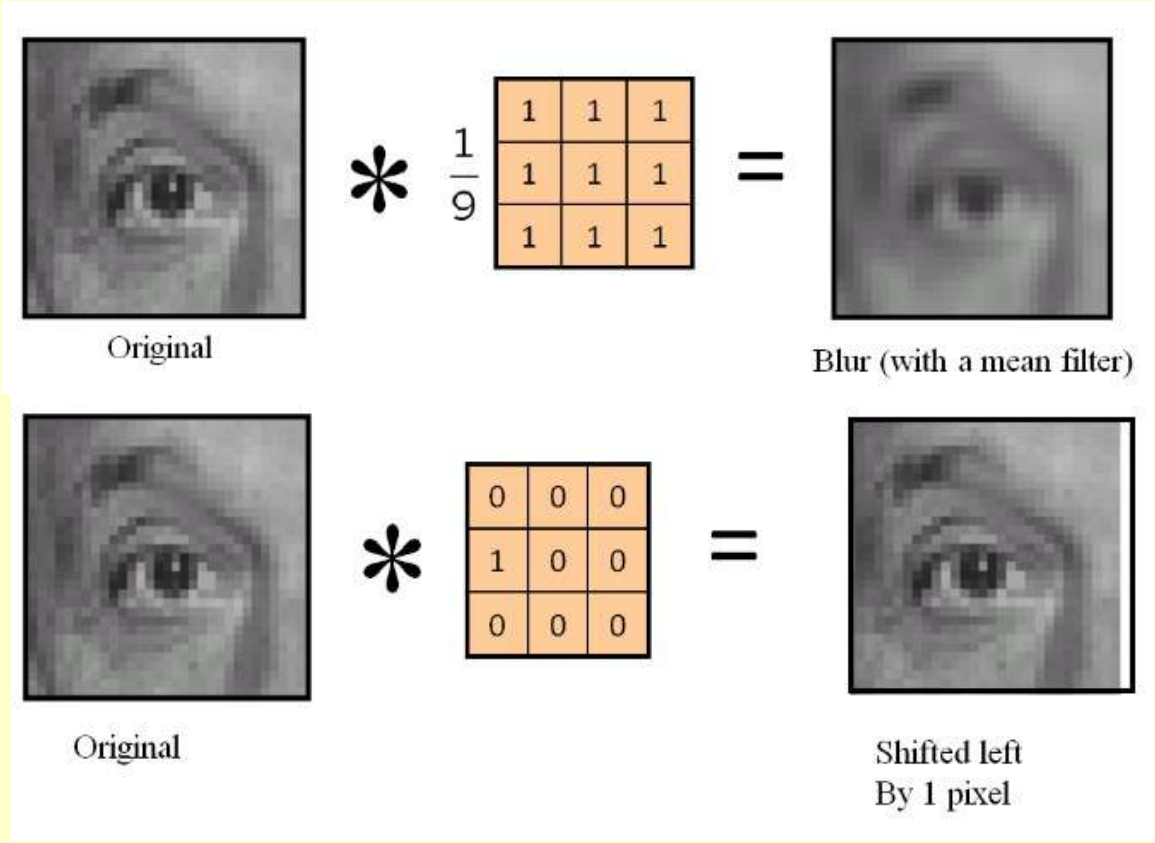


# Convolution



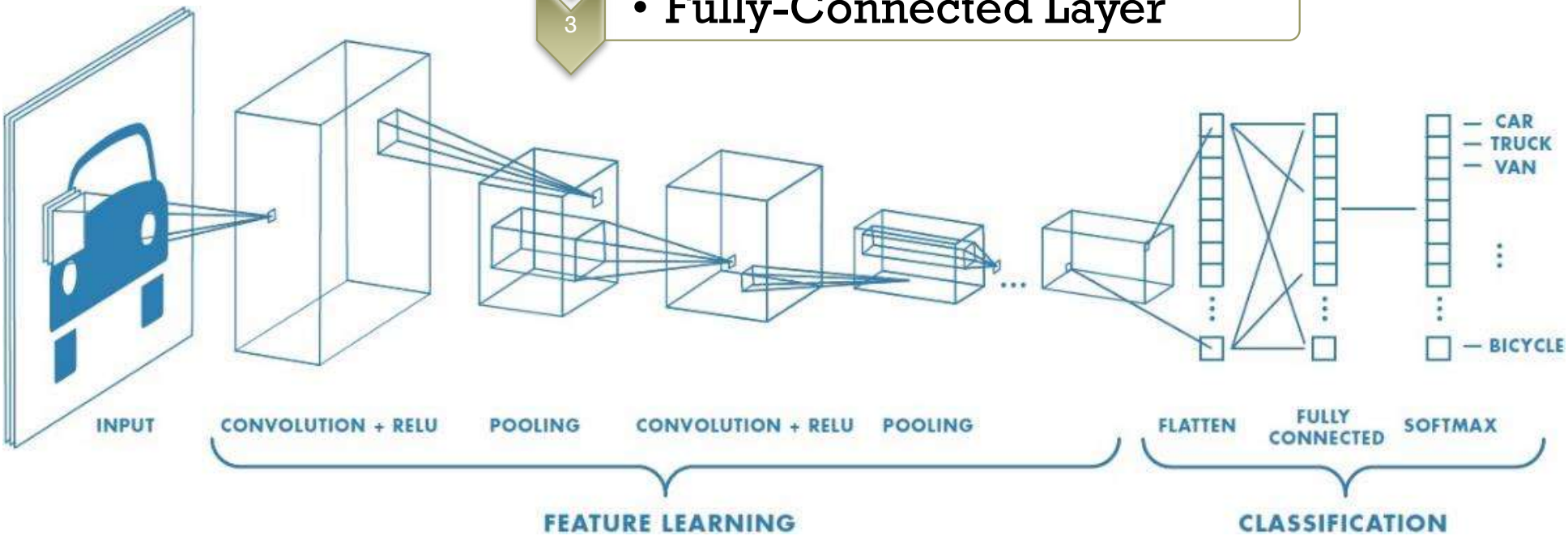

Changing the kernel (filter) results in different image processing outcomes

# Image Convolution

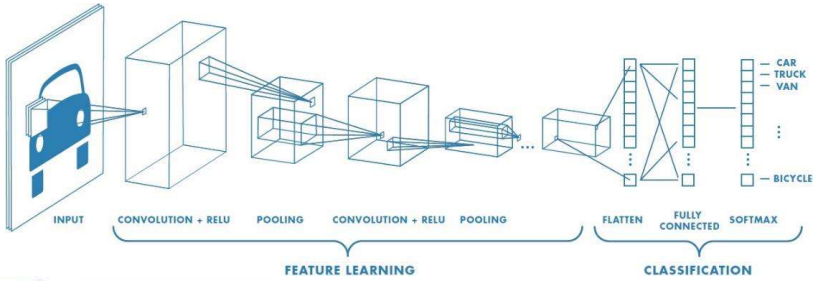


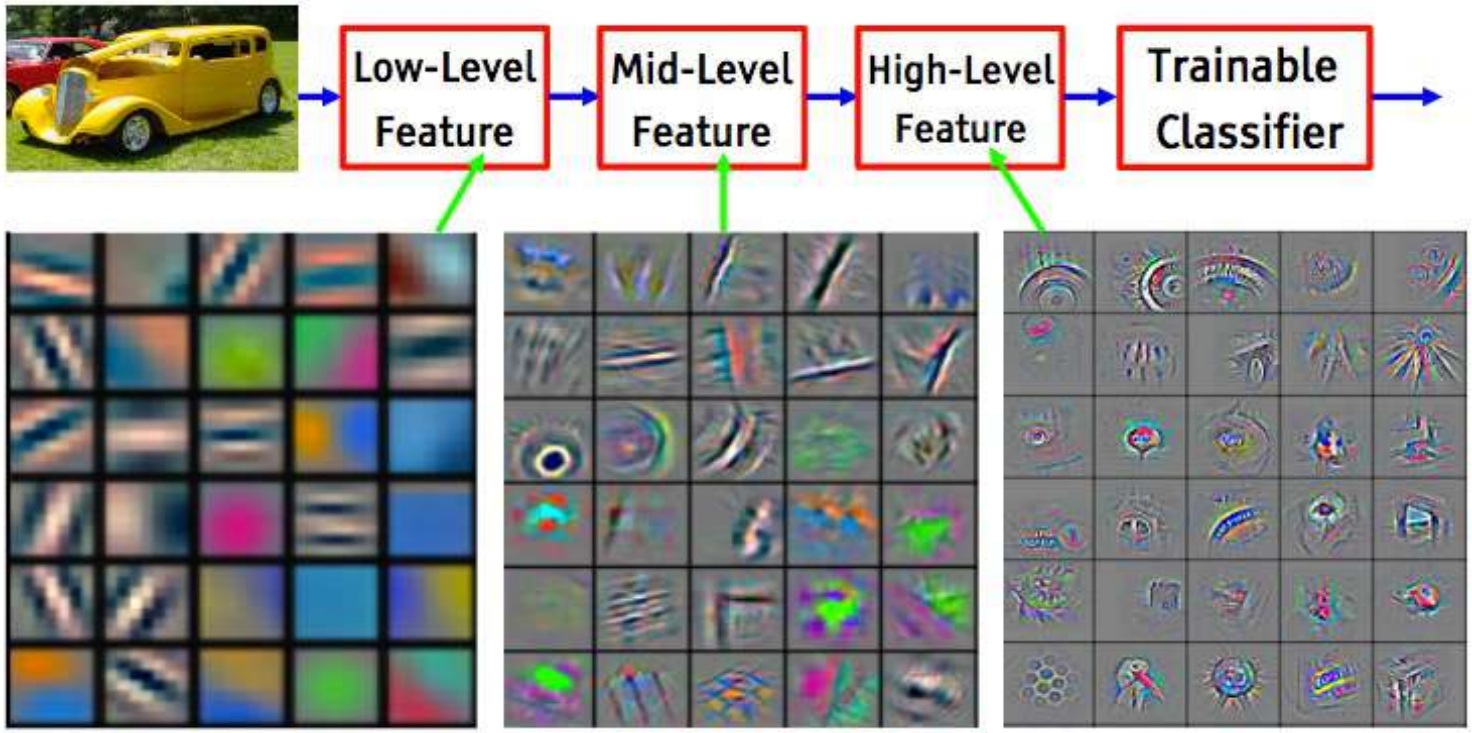
# Convolutional Neural Networks (CNN)

1. Convolutional Layer
2. Pooling Layer
3. Fully-Connected Layer



# CNN (cont.)

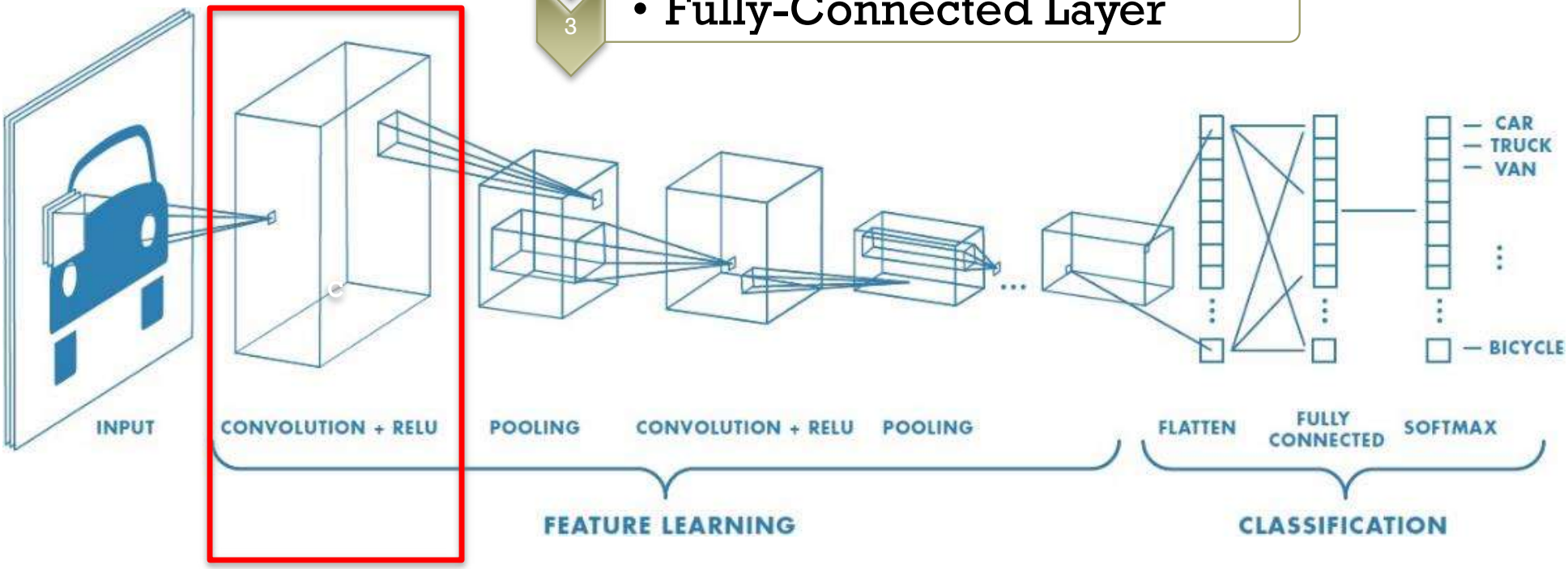

CAR
TRUCK
VAN
BICYCLE
INPUT
CONVOLUTION + RELU
POOLING
CONVOLUTION + RELU
POOLING
FLATTEN
FULLY CONNECTED
SOFTMAX
FEATURE LEARNING
CLASSIFICATION


Low-Level Feature
Mid-Level Feature
High-Level Feature
Trainable Classifier

# Convolutional Neural Networks (CNN)

## 1) Convolutional Layer

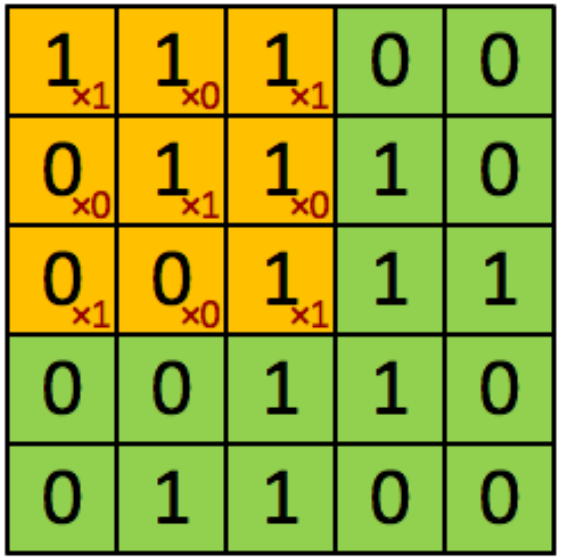
1. Convolutional Layer
2. Pooling Layer
3. Fully-Connected Layer



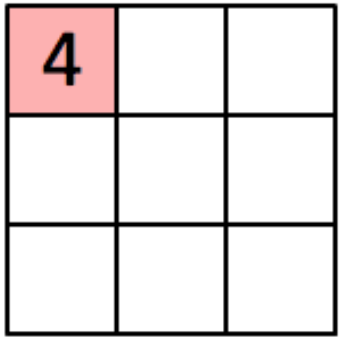
# Layer 1: Convolutional Layer

(3*3 filter)

| | | |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |

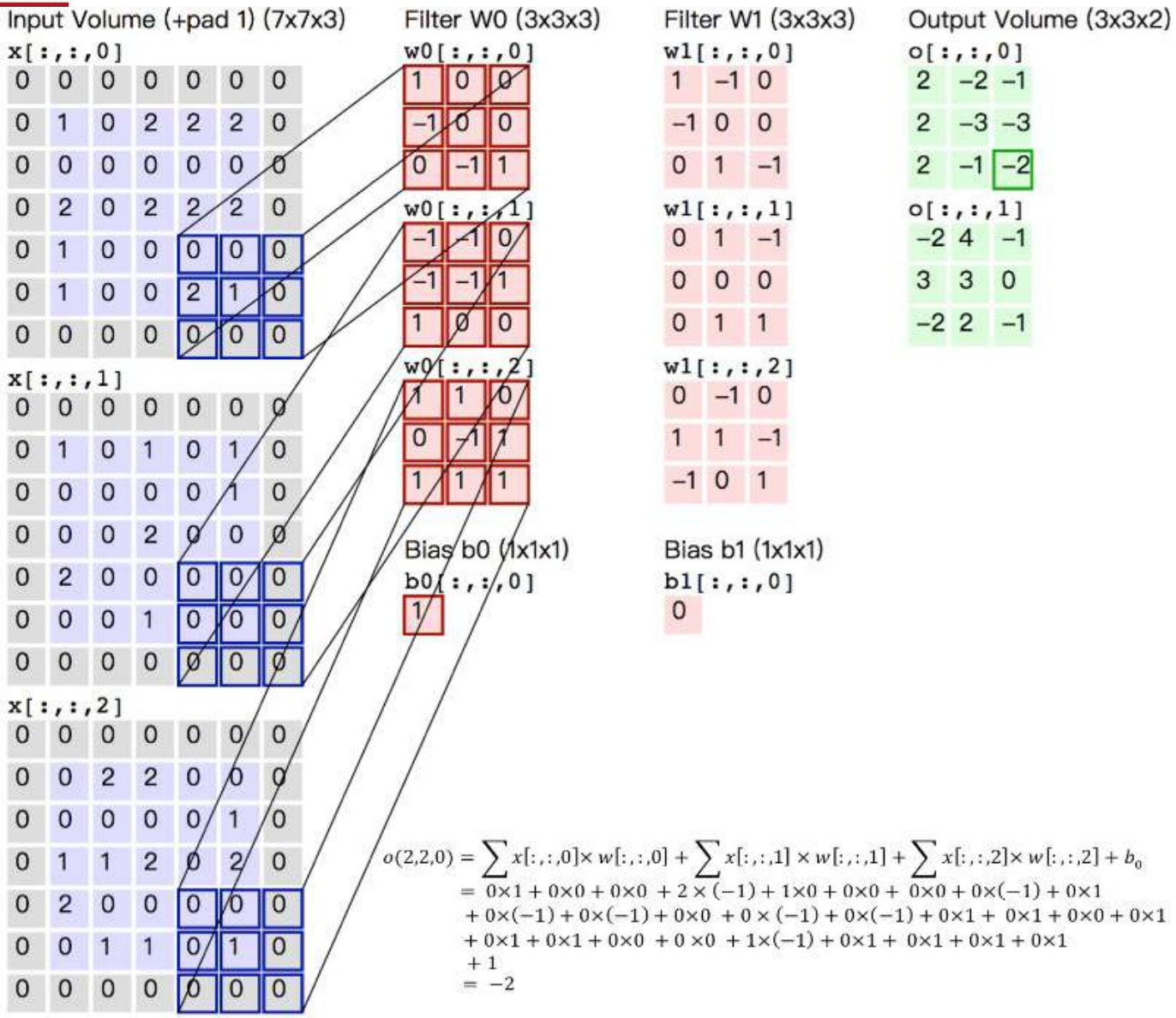



Image




Convolved Feature

1 • Convolutional Layer
2 • Pooling Layer
3 • Fully-Connected Layer




$$o(2,2,0) = \sum x[:,:,0] \times w[:,:,0] + \sum x[:,:,1] \times w[:,:,1] + \sum x[:,:,2] \times w[:,:,2] + b_0$$
$$= 0\times1 + 0\times0 + 0\times0 + 2\times(-1) + 1\times0 + 0\times0 + 0\times0 + 0\times(-1) + 0\times1$$
$$+ 0\times(-1) + 0\times(-1) + 0\times0 + 0\times(-1) + 0\times(-1) + 0\times1 + 0\times1 + 0\times0 + 0\times1$$
$$+ 0\times1 + 0\times1 + 0\times0 + 0\times0 + 1\times(-1) + 0\times1 + 0\times1 + 0\times1 + 0\times1$$
$$+ 1$$
$$= -2$$

# CNNs: 1) Convolutional Layer

## Feature Extraction

http://cs231n.github.io/convolutional-networks/

$$w^T x + b$$

(3*3 filter)

| | | |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |

| | | | | |
|---|---|---|---|---|
| $1_{\times 1}$ | $1_{\times 0}$ | $1_{\times 1}$ | 0 | 0 |
| $0_{\times 0}$ | $1_{\times 1}$ | $1_{\times 0}$ | 1 | 0 |
| $0_{\times 1}$ | $0_{\times 0}$ | $1_{\times 1}$ | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |

Image

| | | |
|---|---|---|
| 4 | | |
| | | |
| | | |

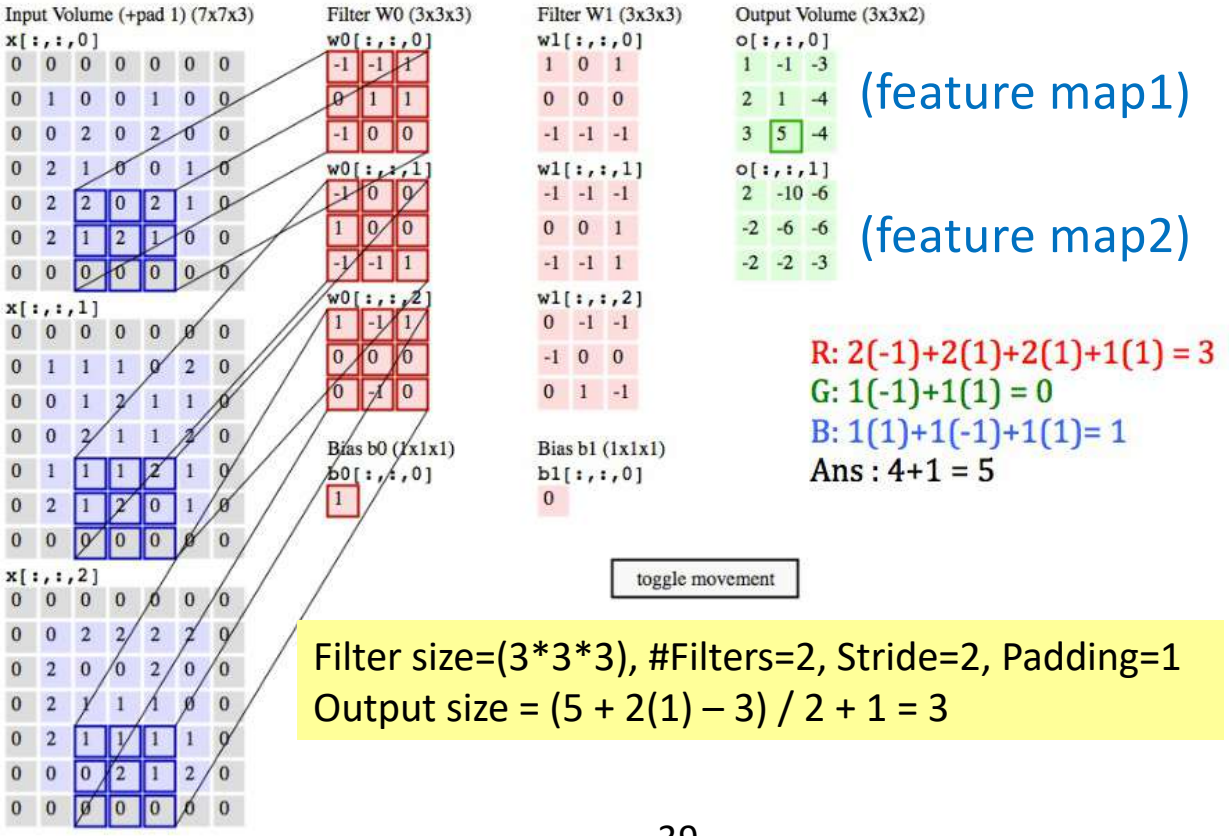
Convolved Feature (feature map)

Filter size=(3*3), #Filters=1, Stride=1, Padding=0
Output size = (5 +2(0) – 3) / 1 + 1 = 3

Summary: **4 parameters** for convolutional layer

- (1) Filter size, (2) #Filters, (3) Stride, (4) Padding

Output size = (N + 2*P – F) / S + 1




(feature map1)

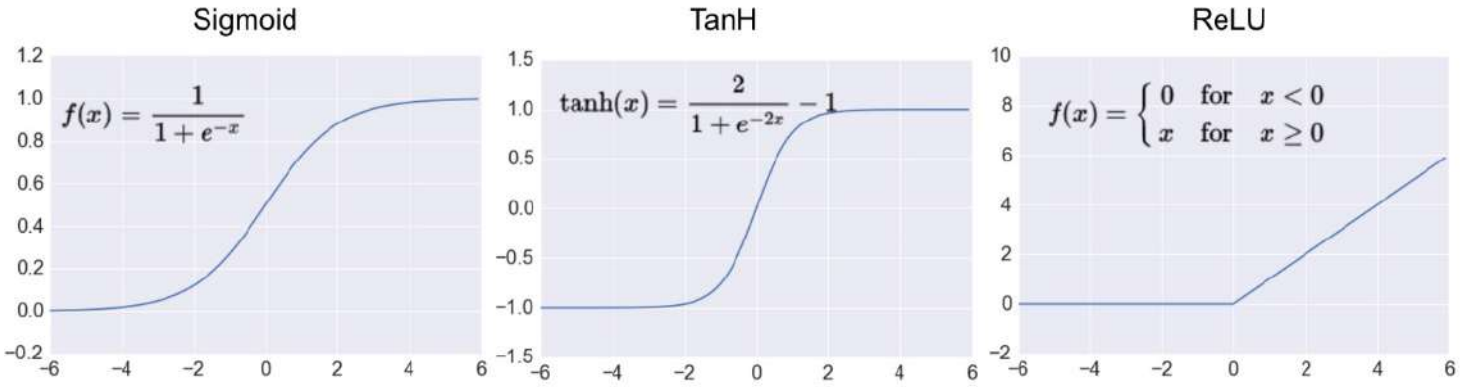
(feature map2)

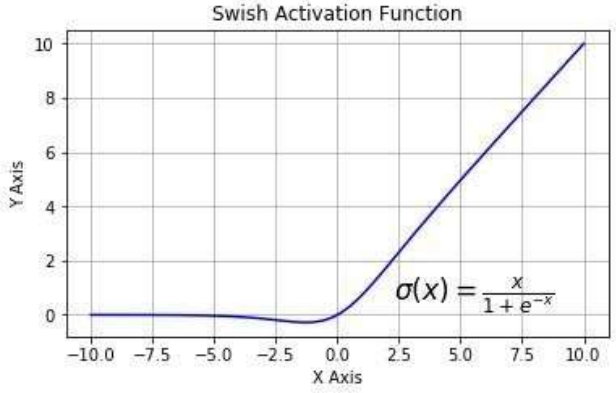
R: 2(-1)+2(1)+2(1)+1(1) = 3
G: 1(-1)+1(1) = 0
B: 1(1)+1(-1)+1(1)= 1
Ans : 4+1 = 5

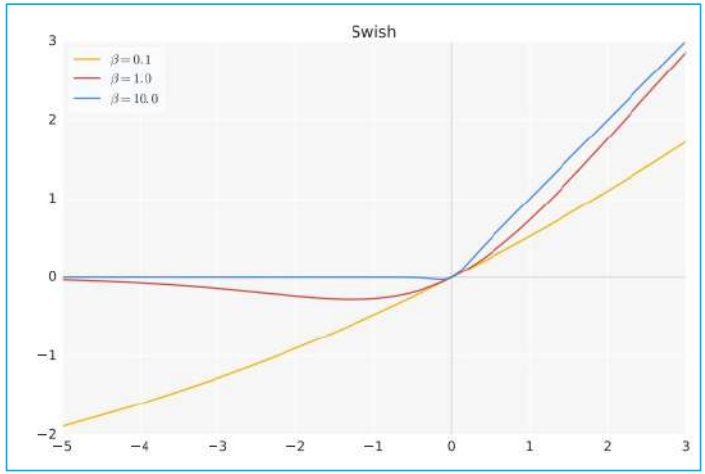
Filter size=(3*3*3), #Filters=2, Stride=2, Padding=1
Output size = (5 + 2(1) – 3) / 2 + 1 = 3

# CNNs: 1) Convolutional Layer (cont.)

## Non-Linear Activation Function



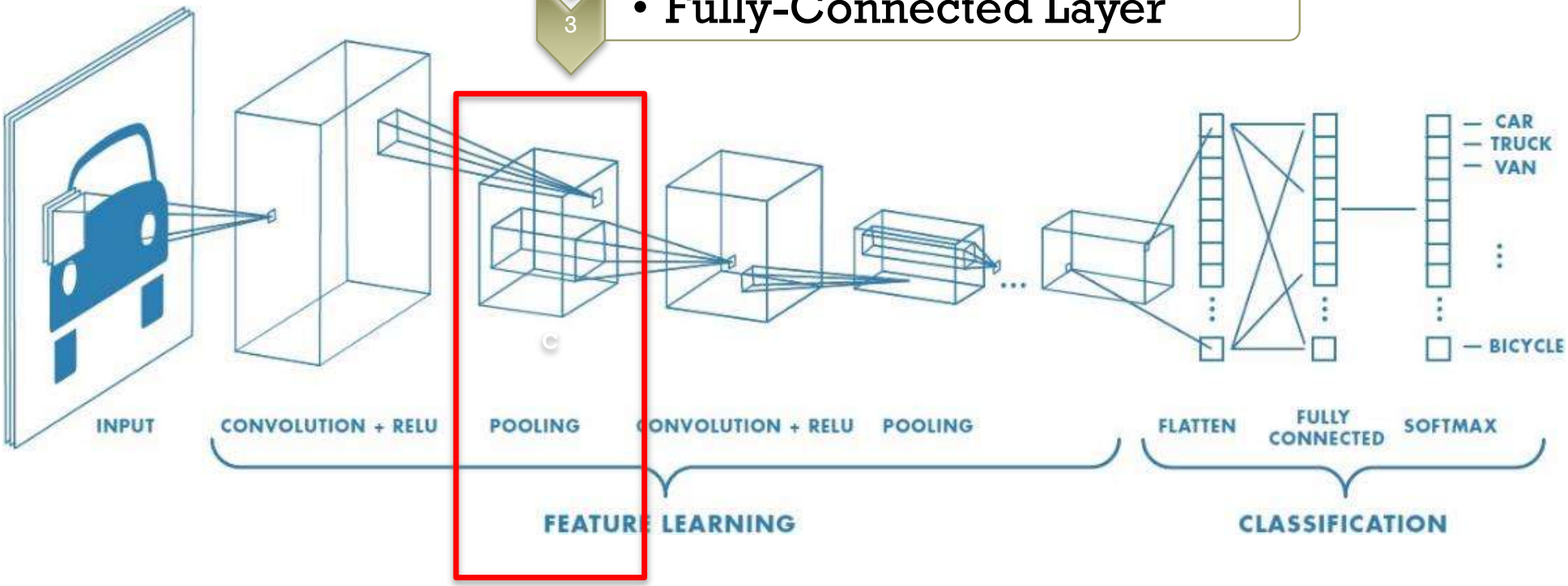



# Convolutional Neural Networks (CNN)

## 2) Pooling Layer

1. Convolutional Layer
2. Pooling Layer
3. Fully-Connected Layer

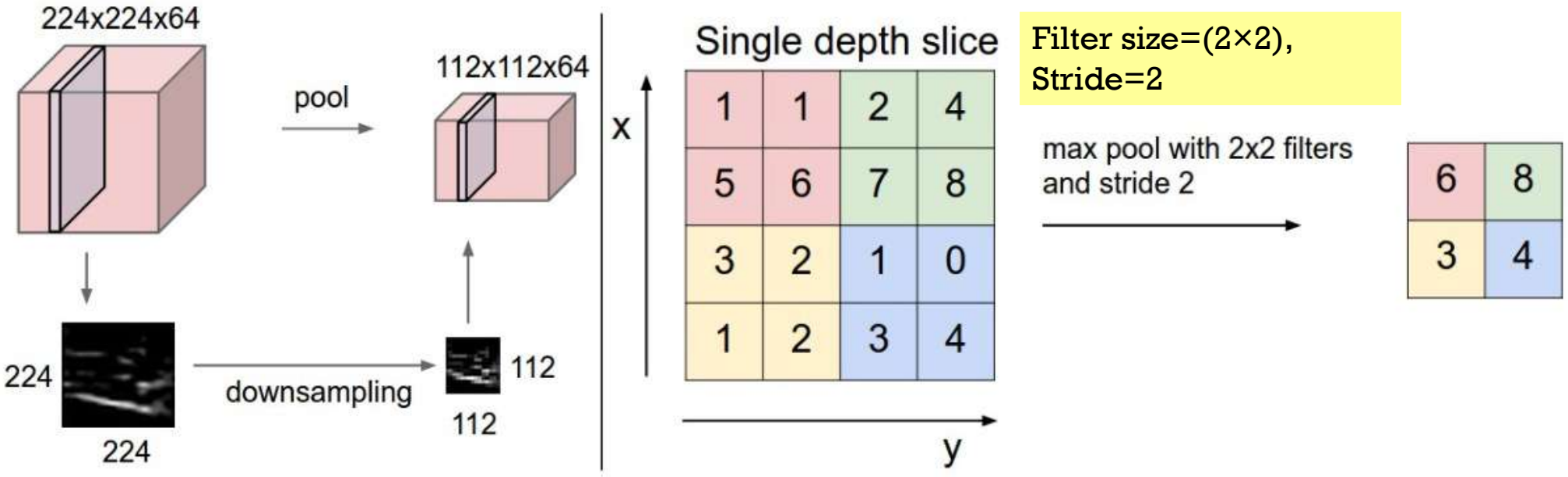


# CNNs: 2) Pooling Layer

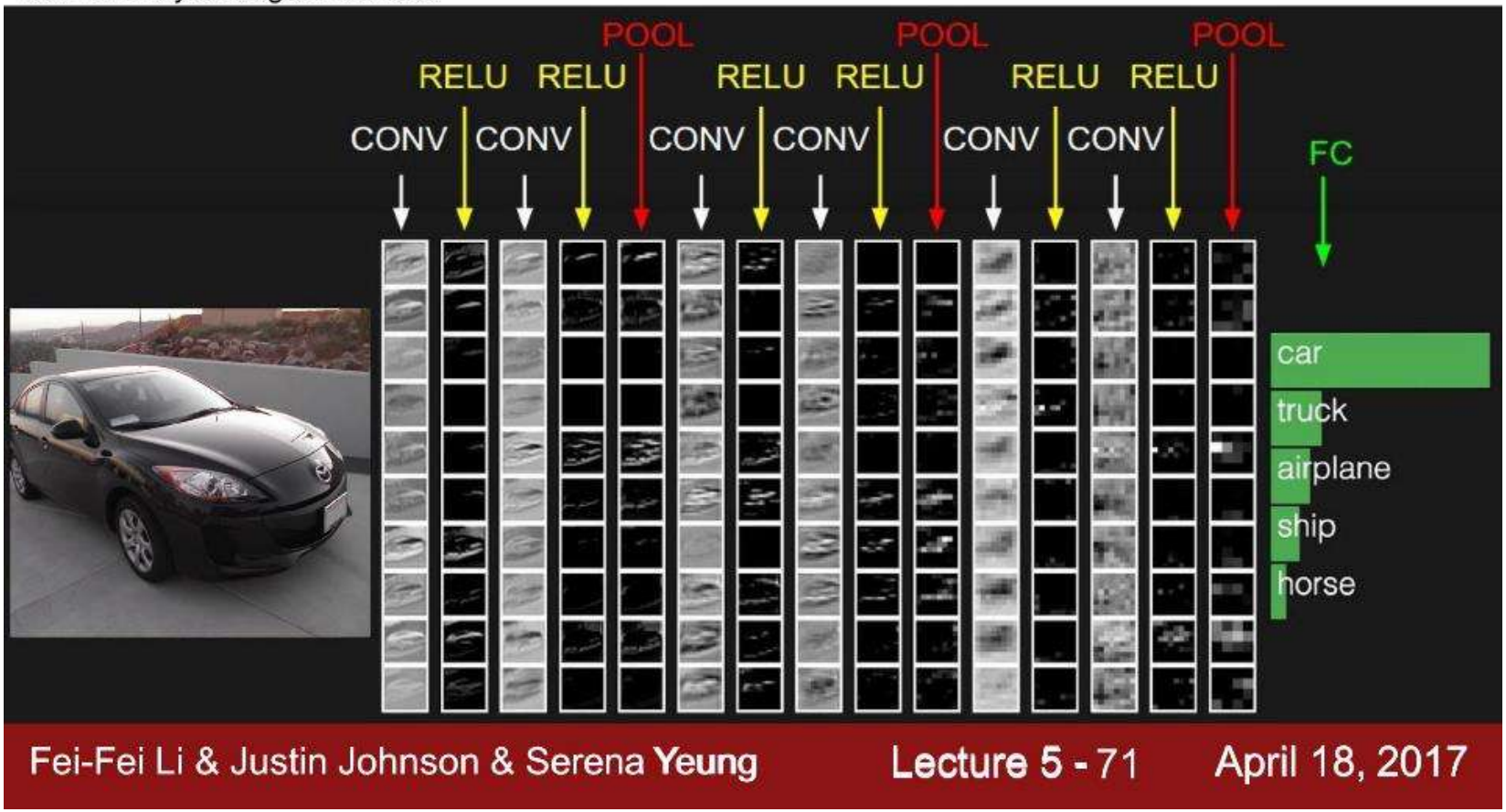
http://cs231n.github.io/convolutional-networks/

Summary: **2 parameters** for pooling layer
- Filter size, Stride

- Feature selection (reduction); downsampling



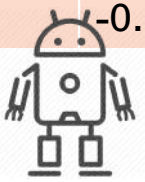
two more layers to go: POOL/FC



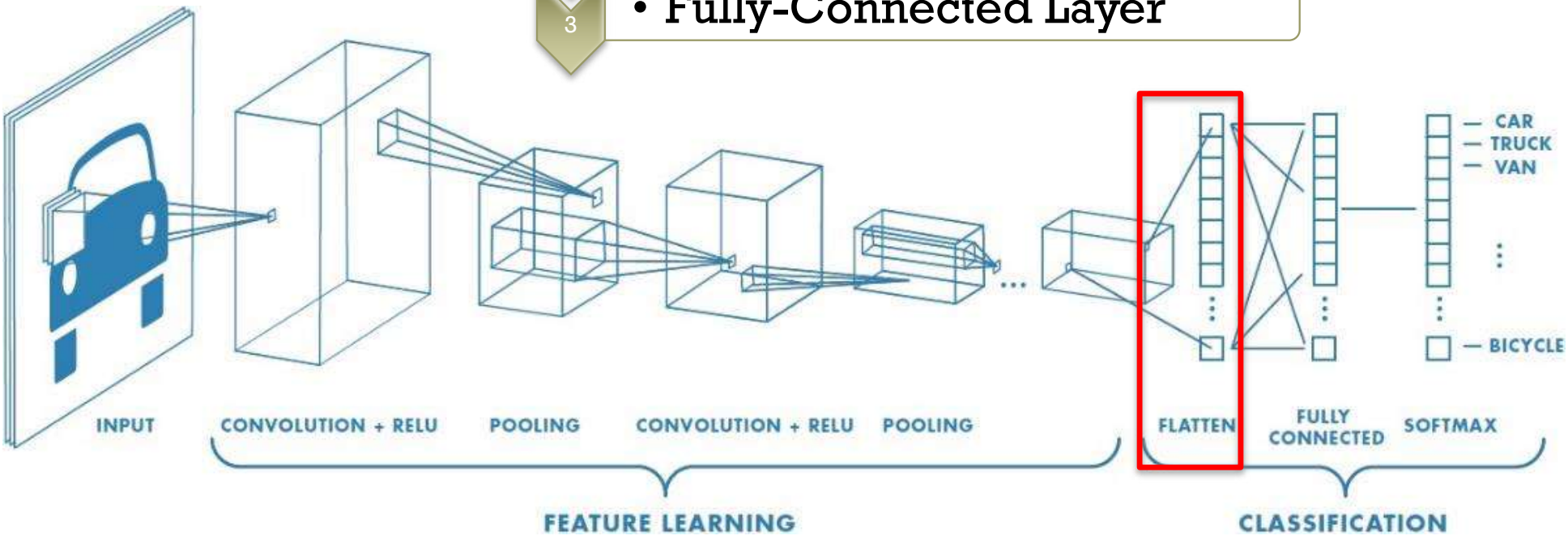
# Convolutional Neural Networks

## Embedding Vector

| x1 | x2 | x3 | x4 | Corona |
|---|---|---|---|---|
| 0.7 | 0.2 | -0.5 | -0.1 | Yes |



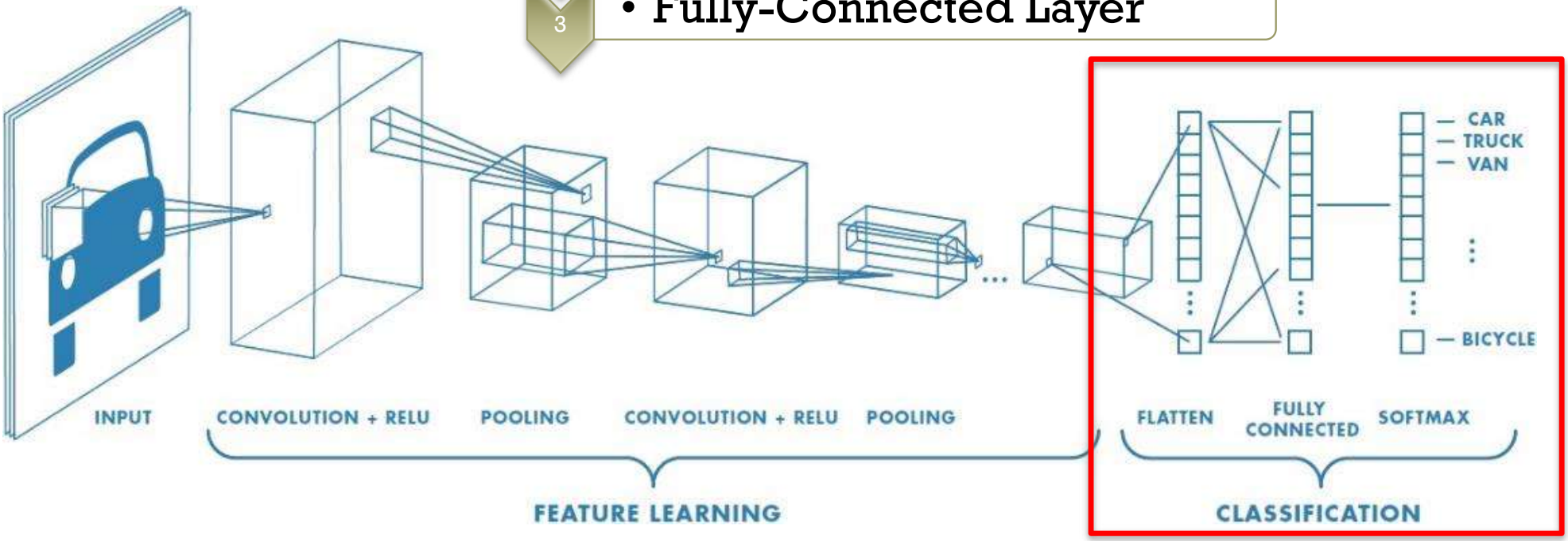
1. • Convolutional Layer
2. • Pooling Layer
3. • Fully-Connected Layer



# Convolutional Neural Networks (CNN)

## 3) Fully-Connected Layer (FC) = Neural Networks

1. Convolutional Layer
2. Pooling Layer
3. Fully-Connected Layer



# 3) Fully-Connected Layer (FC) = Neural Networks

## 3.1) Activation Function for Classification

### 1) Sigmoid — Binary Classification

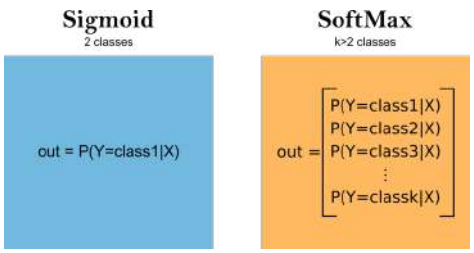
- Used when predicting a single binary outcome (e.g., Cat vs. Not Cat).
- Converts a logit into a probability between 0 and 1.
- Outputs the probability of the positive class.
- Typically used for binary classification.
- Also used for multi-label classification, with one sigmoid per label.

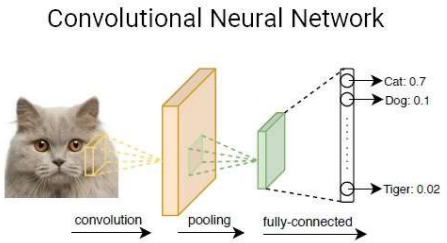
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

### 2) Softmax — Multi-Class

- Used when classifying an image into one of multiple mutually exclusive classes.
- Converts multiple logits into class probabilities.
- All probabilities sum to 1.
- Typically used for multi-class classification.

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

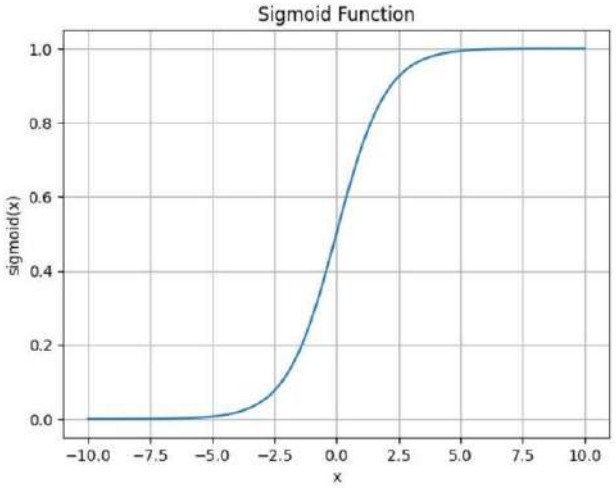


# 3.1) Activation Function for Classification (cont.)
# 1) Sigmoid – Binary Classification

- Example: Cat vs. Not Cat

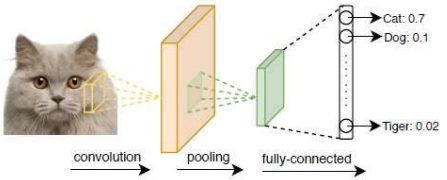
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$




| Model output | Prob | Decision |
|---|---|---|
| **Logit (z)** | **Sigmoid Output** | **Prediction (threshold = 0.5)** |
| -2.0 | 0.119 | Not Cat |
| 0.0 | 0.500 | Cat (at threshold) |
| 2.0 | 0.881 | Cat |

# 3.1) Activation Function for Classification (cont.)
# 2) Softmax – Multi-Class Classification

Convolutional Neural Network




- Example: Cat vs. Dog vs. Bird

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

Model output | Prob

| Class | Logit | Softmax Probability | |
|---|---|---|---|
| Cat | 2.0 | 0.665 | Predict as Cat |
| Dog | 1.0 | 0.245 | |
| Bird | 0.0 | 0.090 | |
| **Total** | | **1.000** | |

# 3) Fully-Connected Layer (FC) = Neural Networks
## 3.2) Loss for Classification

### 1) Binary Cross-Entropy (BCE)

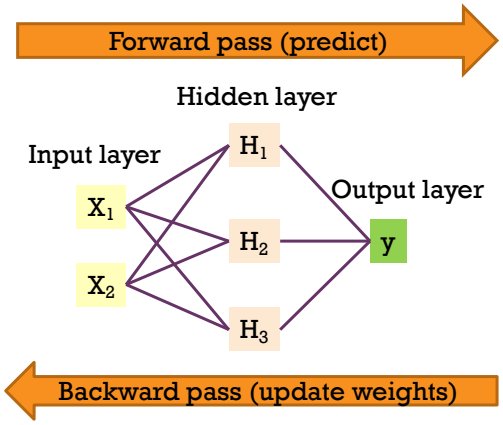
- Binary Classification

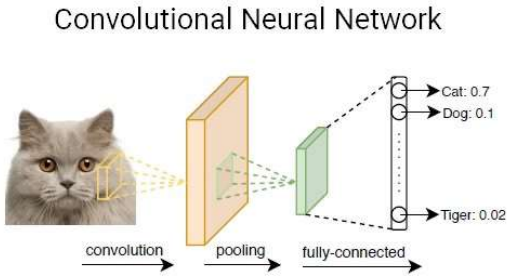
$$L = -[y \log(p) + (1-y) \log(1-p)]$$

### 2) Categorical Cross-Entropy (CCE)

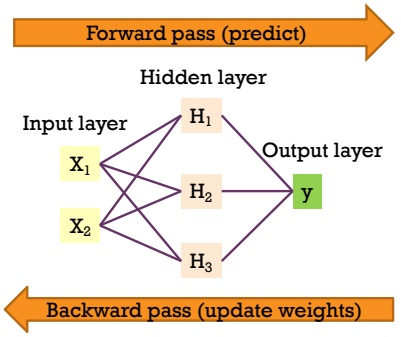
- Multi-Class Classification

$$L = -\sum_{i=1}^{C} y_i \log(p_i)$$



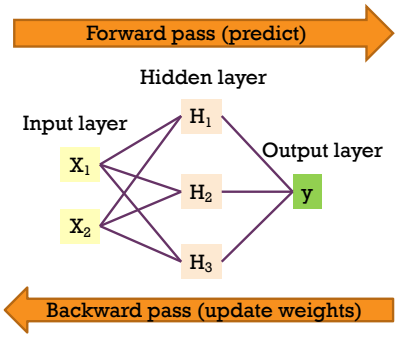
# 3.2) Loss for Classification
# 1) Binary Cross-Entropy (BCE)




$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

- Where:
  - y = Actual label (0 = Not Cat, 1 = Cat)
  - p = Predicted probability of Cat (from Sigmoid)
  - L = Binary Cross-Entropy Loss
- Example: Actual Label = Cat (y = 1)
  - Assume p = 0.9
  - L = -(1)*log(0.9) + (0)*log(0.1) = -log0.9 = 0.105

# 3.2) Loss for Classification
# 2) Categorical Cross-Entropy (CCE)




$$L = -\sum_{i=1}^{C} y_i \log(p_i)$$

- Where:
  - C = Number of classes
  - $y_i$ = Actual label (one-hot encoded: 0 or 1)
  - $p_i$ = Predicted probability of class i (from Softmax)
  - L = Categorical Cross-Entropy Loss

- Example: Actual Label = Cat (y = 1)

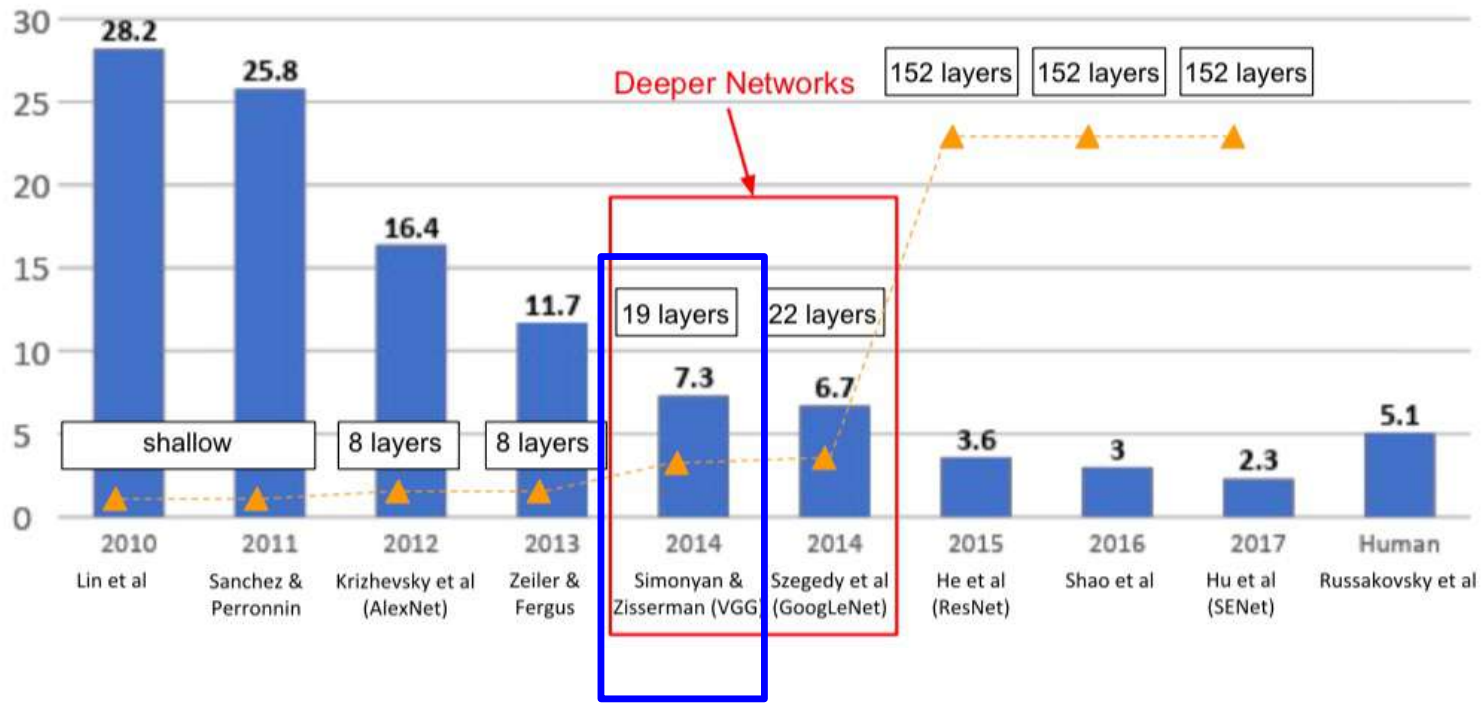
Softmax

| Class | Actual Label ($y_i$) | Predicted Probability ($p_i$) |
|---|---|---|
| Cat | 1 | 0.7 |
| Dog | 0 | 0.2 |
| Bird | 0 | 0.1 |
| **Total** | **1** | **1.0** |

$$L = -[1\log(0.7) + 0\log(0.2) + 0\log(0.1)]$$

$$L = -\log(0.7) = 0.357$$

# ImageNet Large Scale Visual Recognition Challenge (ILSVRC 2nd runner-up in 2014): VGG19



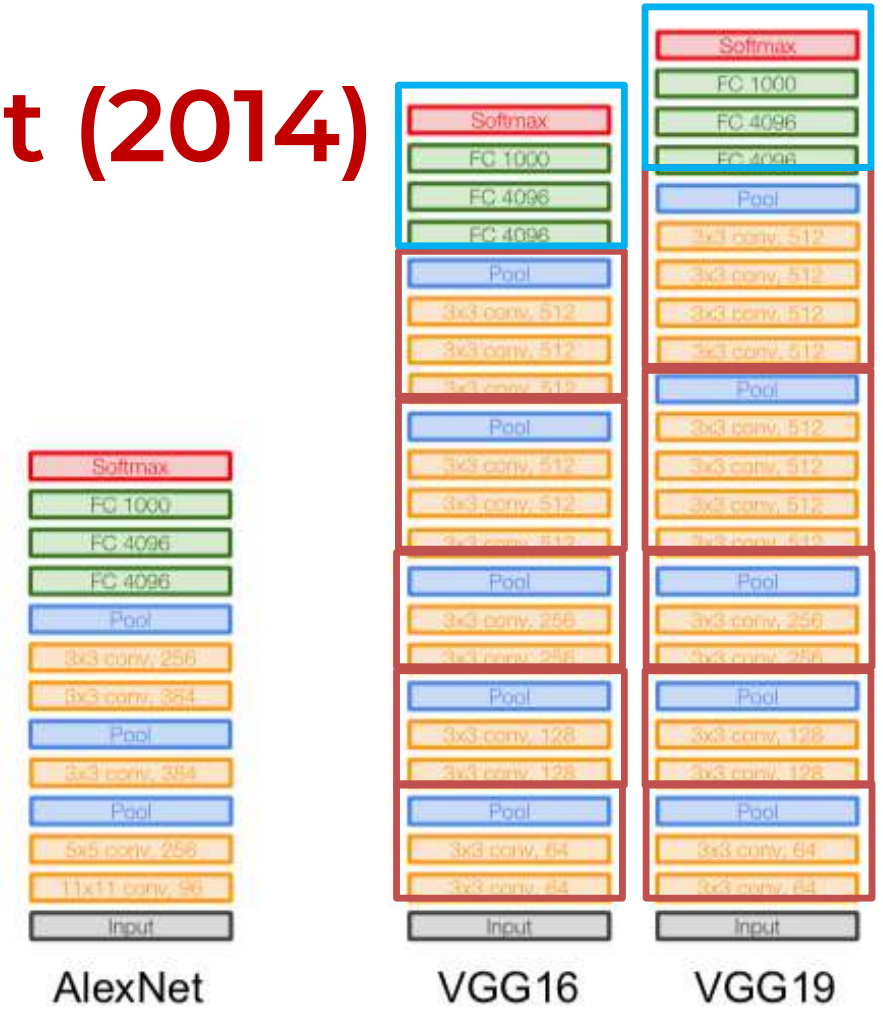
# CNN Example: VGGNet (2014)

*[Simonyan and Zisserman, 2014]*

Small filters, Deeper networks

8 layers (AlexNet) → 16-19 layers (VGG16Net)

Only 3x3 CONV Stride 1, pad 1
And 2x2 MAX POOL stride 2

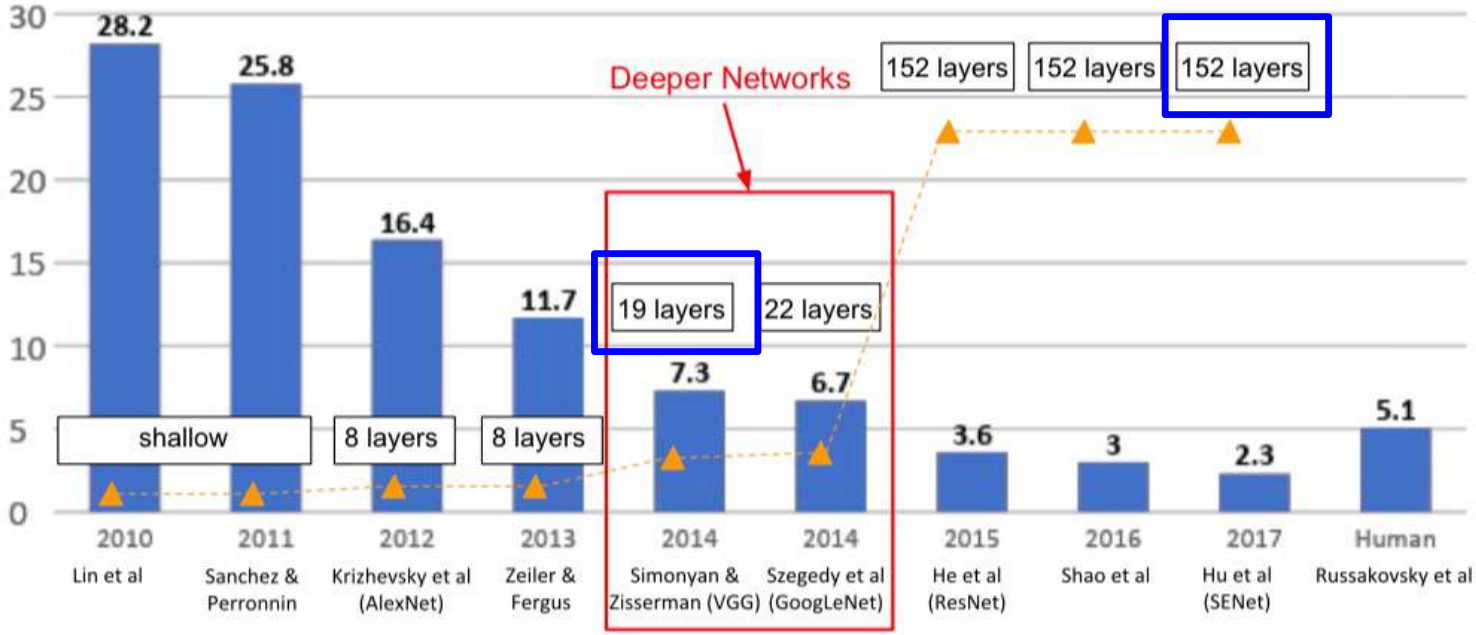



| Network | A | B | C | D | E |
|---|---|---|---|---|---|
| Number of parameters (Millions) | 133 | 133 | 134 | 138 | 144 |



53

# ImageNet Large Scale Visual Recognition Challenge (ILSVRC 2nd runner-up in 2014): Deeper than VGG19






54

## Model 1

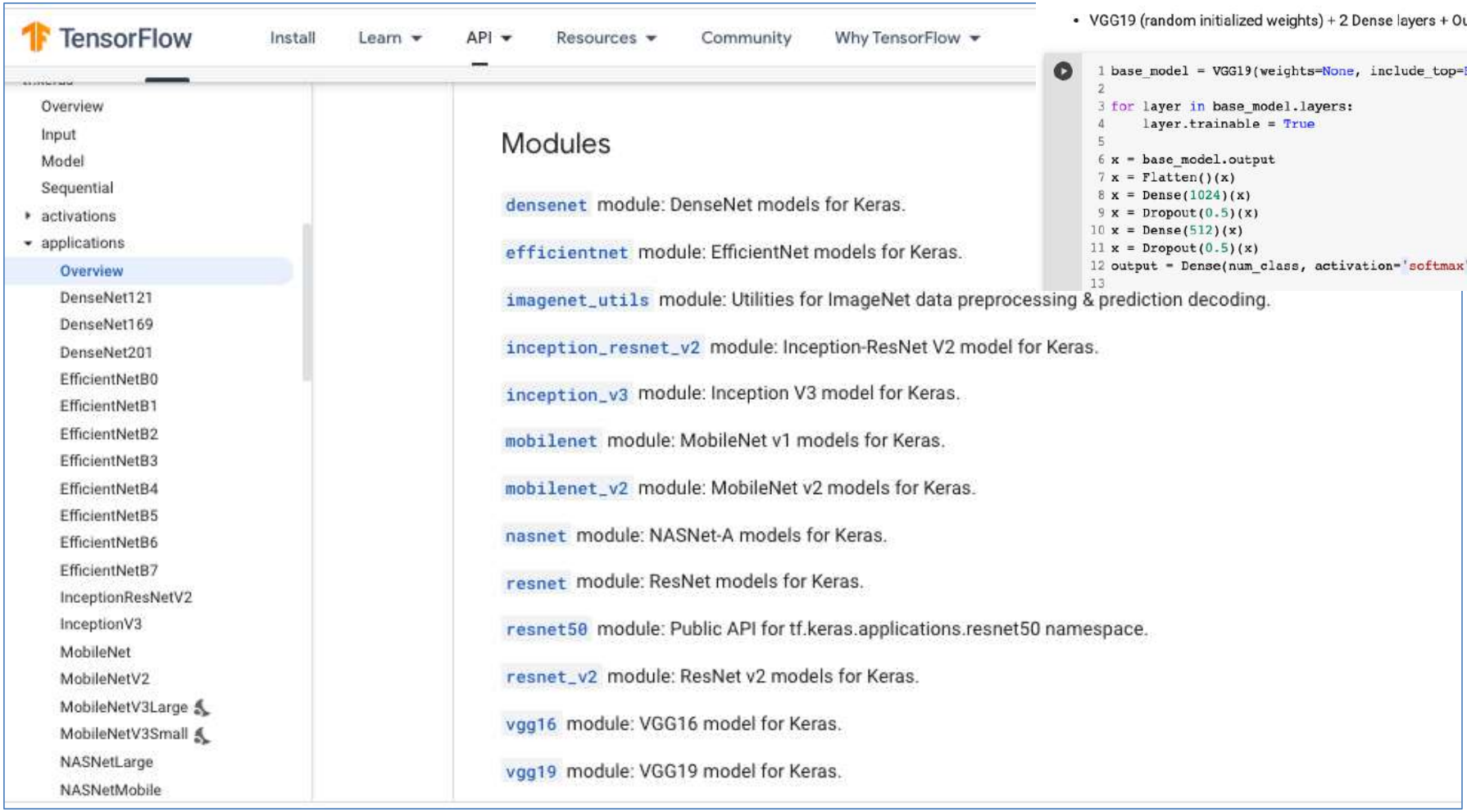
- VGG19 (random initialized weights) + 2 Dense layers + Output layer

```
base_model = VGG19(weights=None, include_top=False, input_shape=(224, 224, 3))

for layer in base_model.layers:
    layer.trainable = True

x = base_model.output
x = Flatten()(x)
x = Dense(1024)(x)
x = Dropout(0.5)(x)
x = Dense(512)(x)
x = Dropout(0.5)(x)
output = Dense(num_class, activation='softmax')(x)

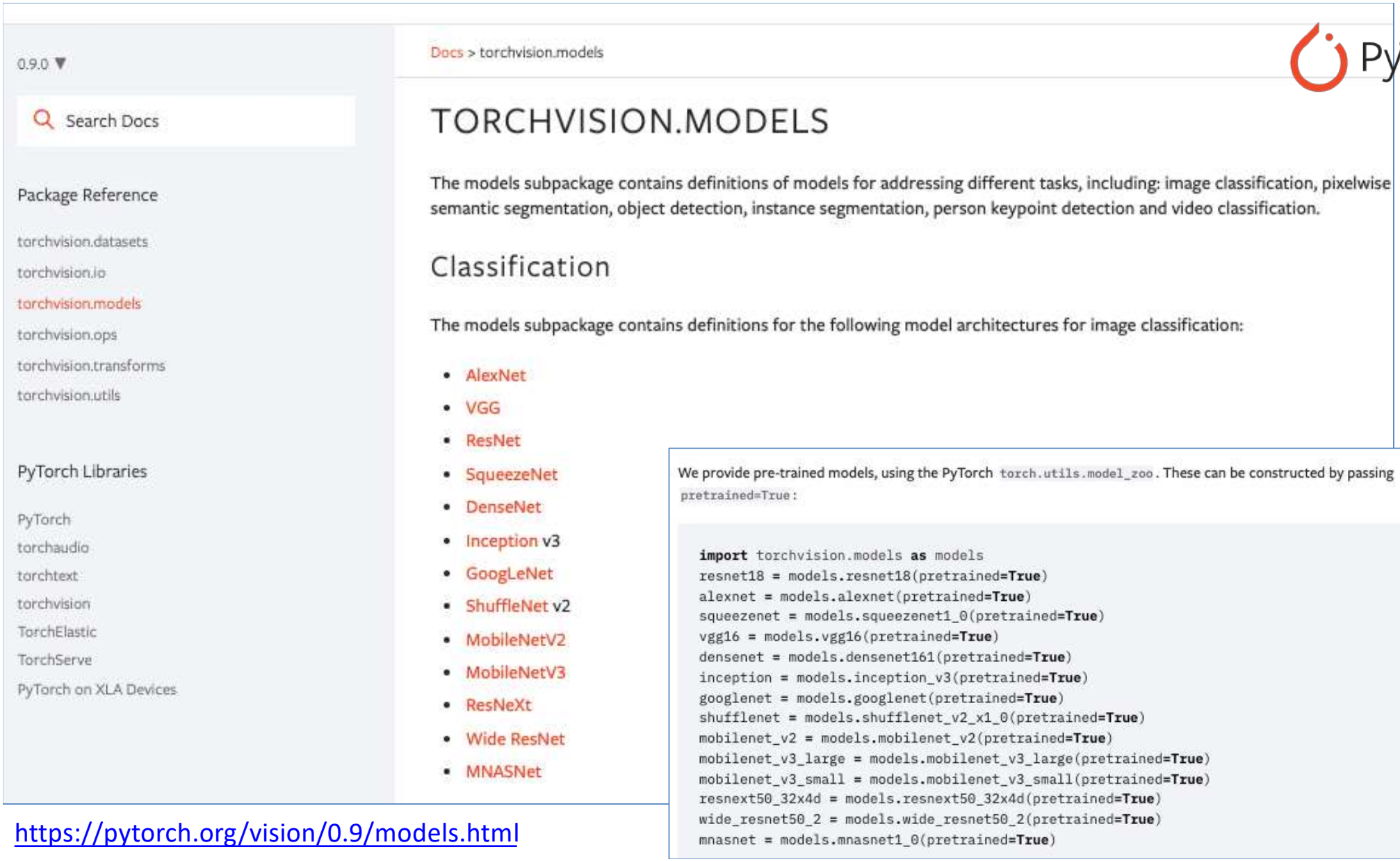
```




https://www.tensorflow.org/api_docs/python/tf/keras/applications



55




https://pytorch.org/vision/0.9/models.html

+

# Pretrained Models

Have you ever seen this creature before?
Can you guess whether it is a land animal or a water animal?



Have you ever seen this creature before?
Can you guess whether it is a land animal or a water animal?
You can transfer your knowledge from the past

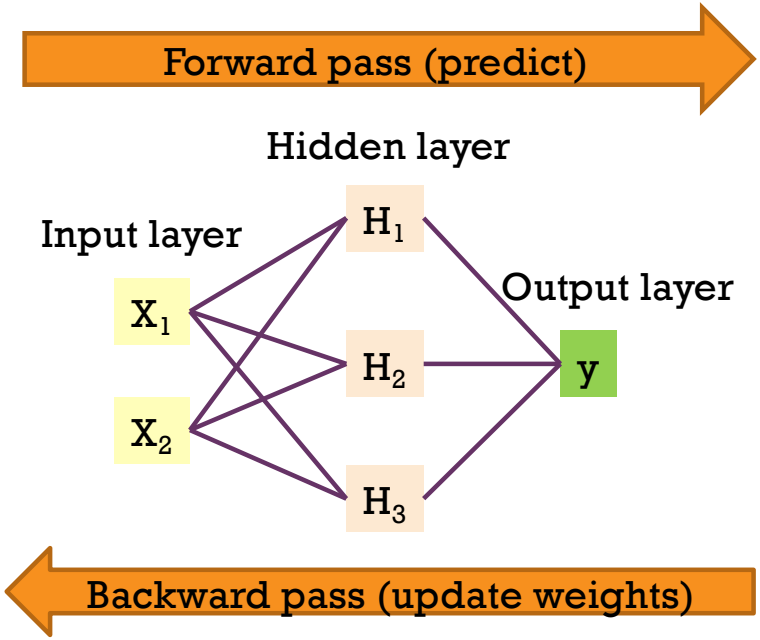




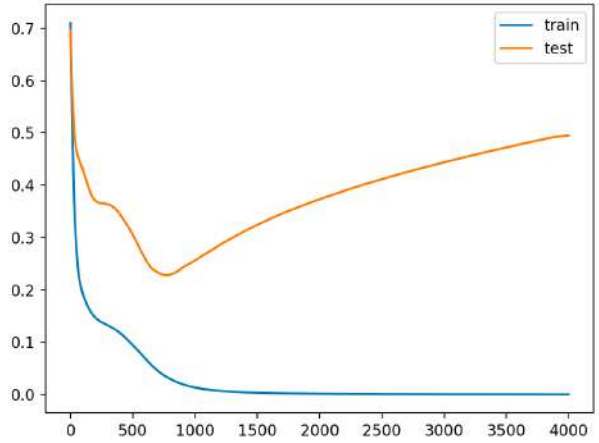


# How can we initialize weights?

## Randomize (from scratch)




| Age | Income | Gender | Province | Corona |
|---|---|---|---|---|
| 25 | 25,000 | Female | Bangkok | Yes |
| 35 | 50,000 | Female | Nontaburi | Yes |
| 32 | 35,000 | Male | Bangkok | No |




$$log\left(\frac{\hat{p}}{1-\hat{p}}\right) = \hat{w}_0 + \hat{w}_1 H_1 + \hat{w}_2 H_2 + \hat{w}_3 H_3$$

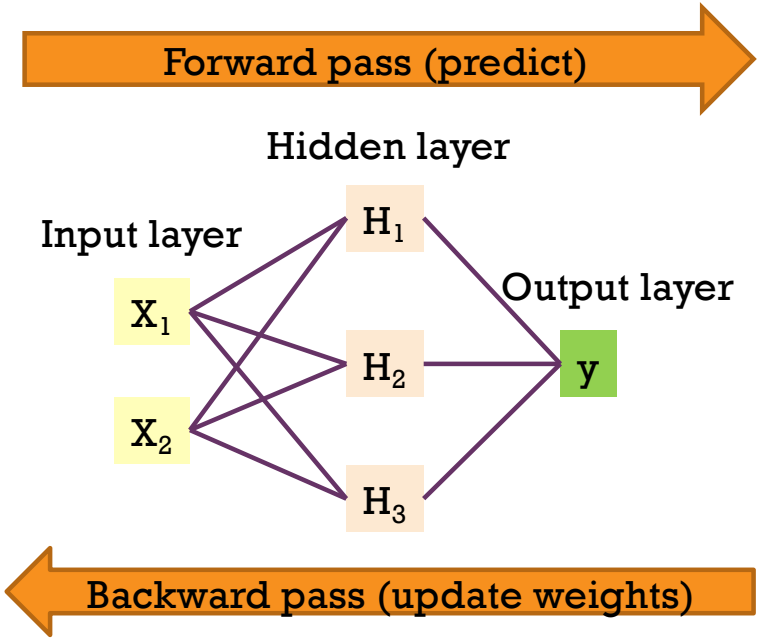
$$H_1 = tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

$$H_2 = tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

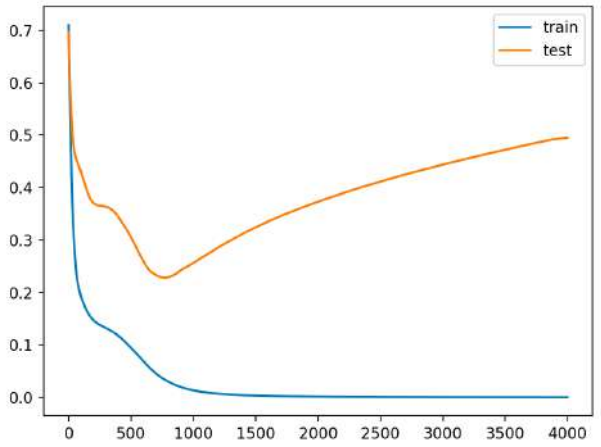
$$H_3 = tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

# What is the output of model training?

## Model weights (knowledge)




| Age | Income | Gender | Province | Corona |
|---|---|---|---|---|
| 25 | 25,000 | Female | Bangkok | Yes |
| 35 | 50,000 | Female | Nontaburi | Yes |
| 32 | 35,000 | Male | Bangkok | No |




$$log\left(\frac{\hat{p}}{1-\hat{p}}\right) = \hat{w}_0 + \hat{w}_1 H_1 + \hat{w}_2 H_2 + \hat{w}_3 H_3$$

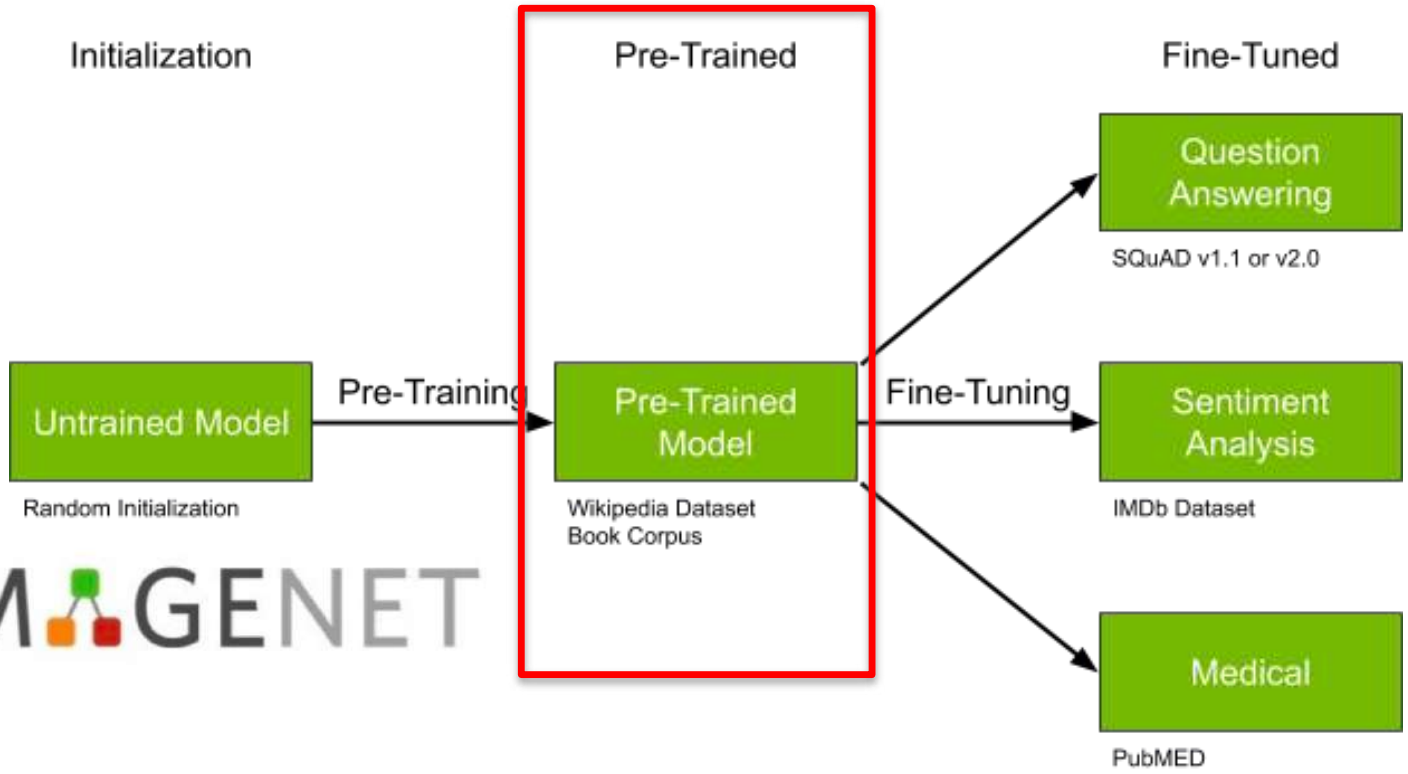
$$H_1 = tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2)$$

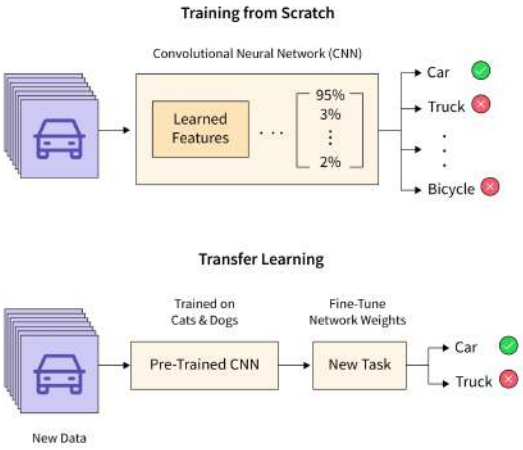
$$H_2 = tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2)$$

$$H_3 = tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)$$

# Pretrained models (aka. Transfer learning)

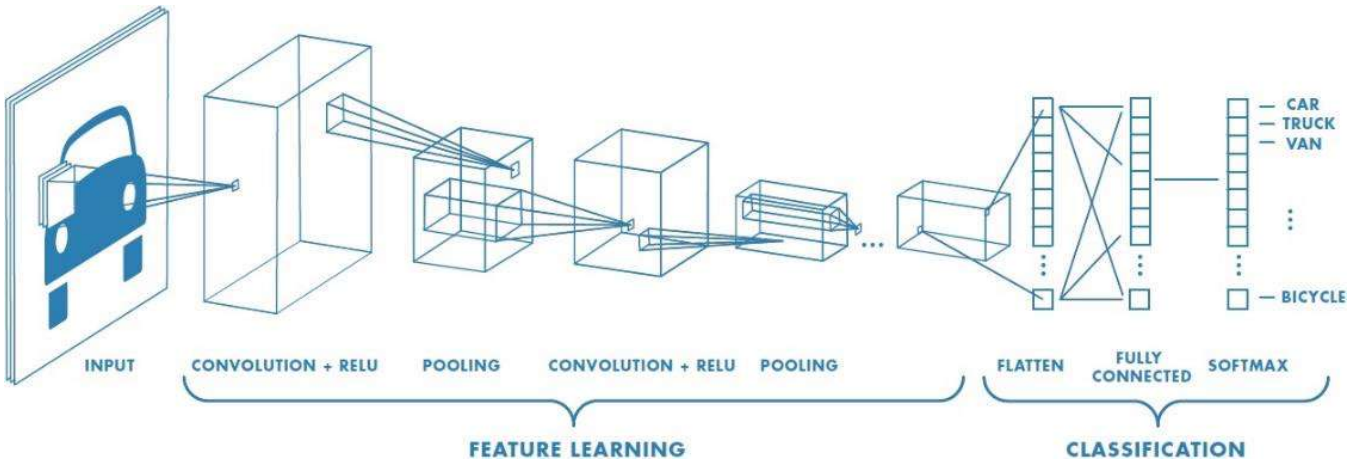
- Key idea: Pretraining provides a useful starting point; the new task may need less data and compute.



# Fine-Tuning

- Fine-Tuning = Pretrained Model + Additional Training on a New Task
- What if the number of output classes differs between the pretrained model and the new target task? How should we handle this?
- 1) Fine-tune only classification layer (frozen feature extraction)
- 2) Fina-tune all (feature extraction & classification layers)








## 1. Load a Pretrained Model

Python

```python
import torch
import torch.nn as nn
from torchvision import models

# Load pretrained ResNet50 (ImageNet: 1,000 classes)
model = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
```

## 2. Option A: Frozen Model (Feature Extraction)

Freeze all pretrained layers and train only the new classification layer.

Python

```python
# Freeze all pretrained layers
for param in model.parameters():
    param.requires_grad = False

# Replace the final layer (1,000 → 2 classes)
model.fc = nn.Linear(model.fc.in_features, 2)
```

## 3. Option B: Unfrozen Model (Fine-Tuning)

Allow all layers to be updated during training.

Python

```python
# Load pretrained ResNet50
model = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)

# Replace the final layer (1,000 → 2 classes)
model.fc = nn.Linear(model.fc.in_features, 2)

# Unfreeze all layers (default)
for param in model.parameters():
    param.requires_grad = True
```



v1.1.0 ▼

pytorch-transformers

Star 131,473

Search docs


NOTES

Installation

Quickstart

Pretrained models

Examples

Notebooks

Loading Google AI or OpenAI pre-trained weights or PyTorch dump

Serialization best-practices

Converting Tensorflow Checkpoints

Migrating from pytorch-pretrained-bert

BERTology



ⓘ You are viewing legacy docs. Go to latest documentation instead.




# Pytorch-Transformers

PyTorch-Transformers is a library of state-of-the-art pre-trained models for Natural Language Processing (NLP).

The library currently contains PyTorch implementations, pre-trained model weights, usage scripts and conversion utilities for the following models:

1. BERT (from Google) released with the paper BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding by Jacob Devlin, Ming-Wei Lee and Kristina Toutanova.
2. GPT (from OpenAI) released with the paper Improving Language Understanding by Generative Pre-Training by Alec Radford, Karthik Narasimhan, Tim Salim Sutskever.
3. GPT-2 (from OpenAI) released with the paper Language Models are Unsupervised Multitask Learners by Alec Radford*, Jeffrey Wu*, Rewon Child, David Lua and Ilya Sutskever**.
4. Transformer-XL (from Google/CMU) released with the paper Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context by Zihang Dai*, Z Yang, Jaime Carbonell, Quoc V. Le, Ruslan Salakhutdinov.
5. XLNet (from Google/CMU) released with the paper XLNet: Generalized Autoregressive Pretraining for Language Understanding by Zhilin Yang*, Zihang Dai Jaime Carbonell, Ruslan Salakhutdinov, Quoc V. Le.
6. XLM (from Facebook) released together with the paper Cross-lingual Language Model Pretraining by Guillaume Lample and Alexis Conneau.

## Notes

- Installation

https://www.martincid.com/technology-sv/nvidia-reportedly-buys-hugging-face-12-9-billion/





AI

# Nvidia bets $12.9B that owning Hugging Face won't kill open-source AI




Nvidia. Photo: 4300streetcar / CC BY 4.0, via Wikimedia Commons



By **Adrian Kessler**
31 August 2026, 3:37 am · 3 min read

# Beyond CNN

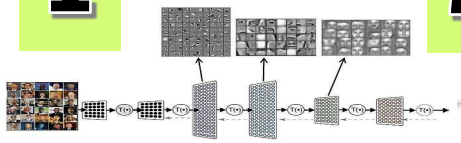
# Deep Learning Application

Speech Recognition

Computer Vision

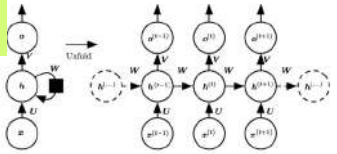
Natural Language Processing

1



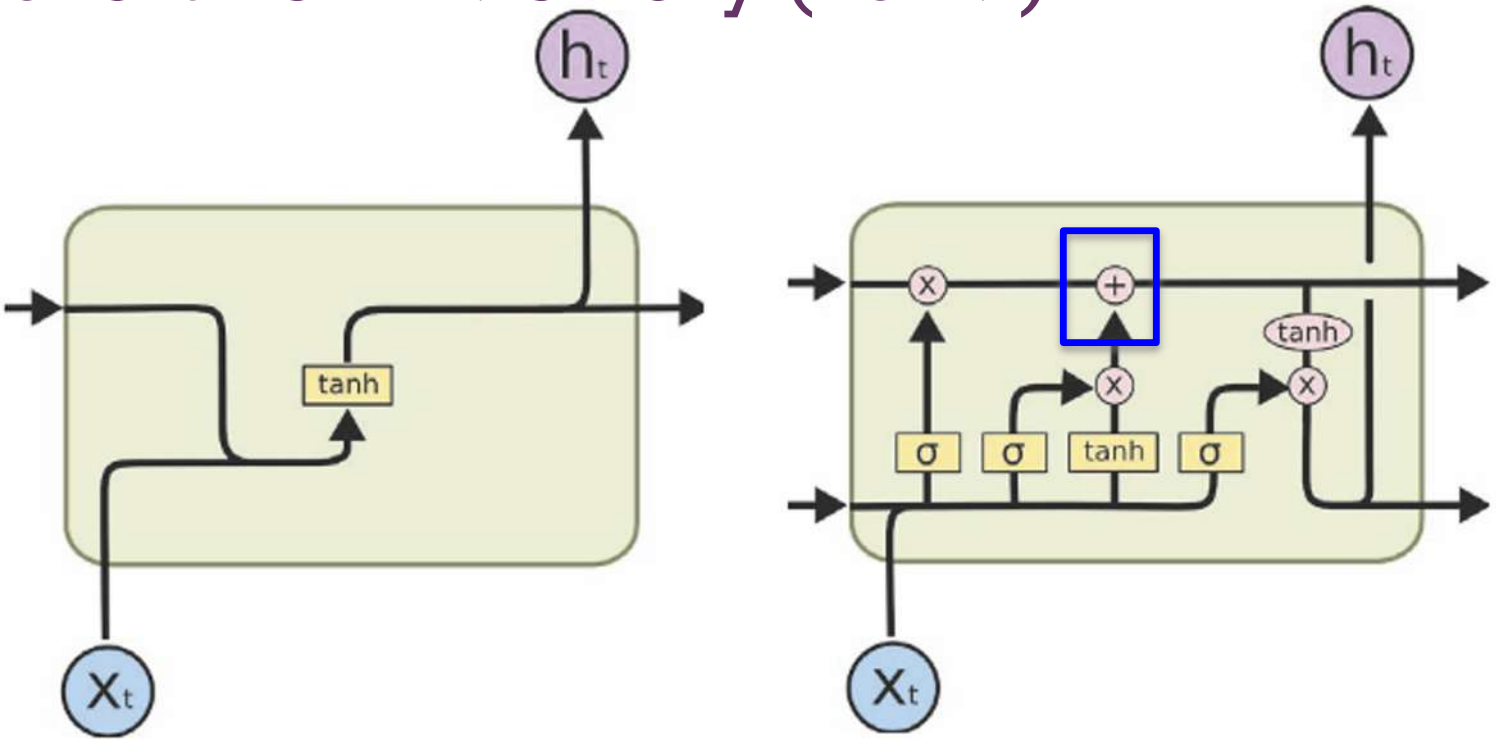
CNN

2



RNN (LSTM)

# 1) Recurrent Neural Networks (RNN) & Long Short-Term Memory (LSTM)

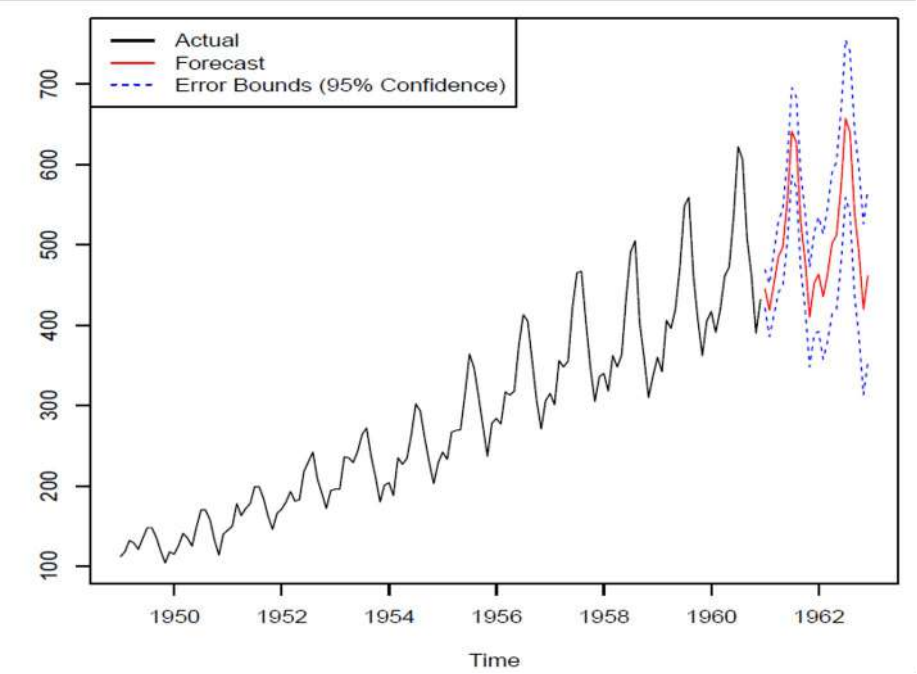


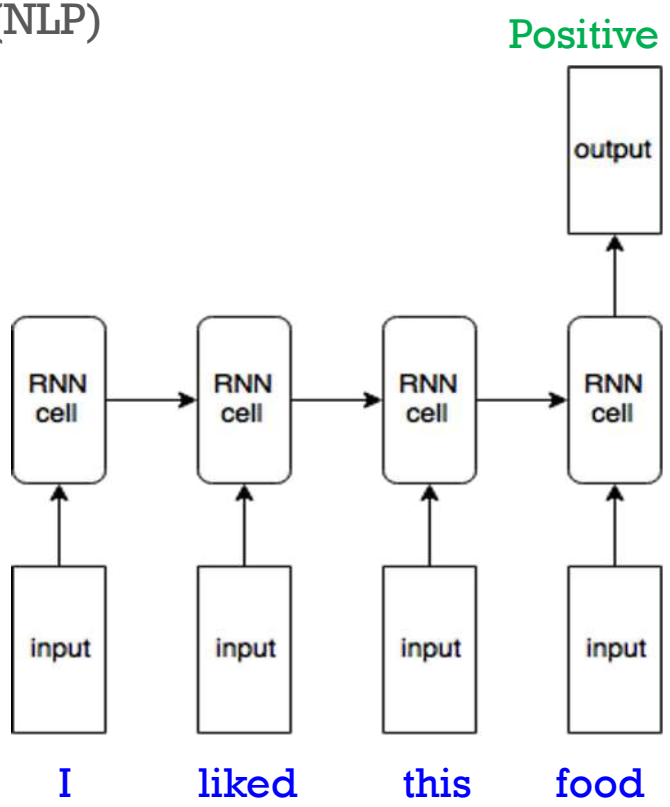

(a) RNN (b) LSTM

Reference: https://cs.uwaterloo.ca/~mli/Deep-Learning-2017-Lecture6RNN.ppt

# 1) RNN & LSTM (cont.)

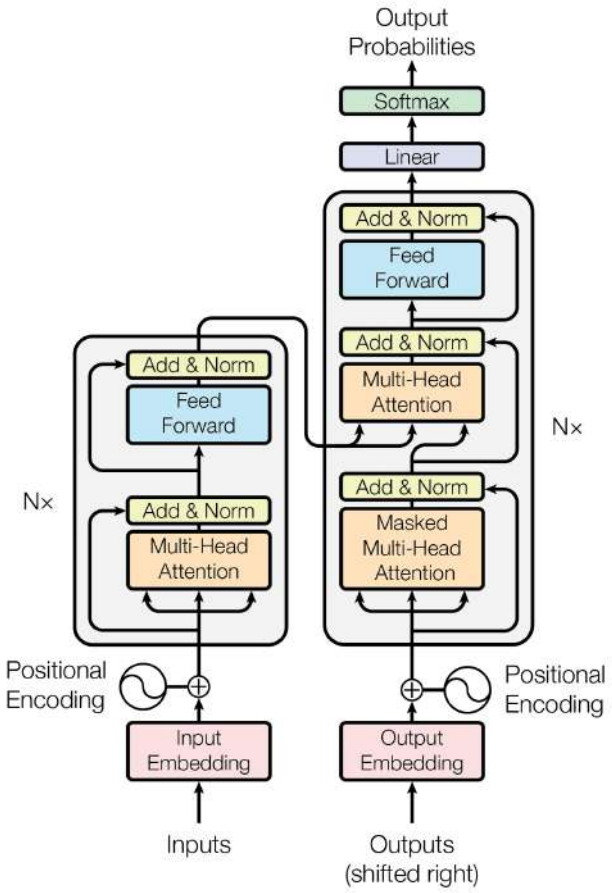
- Time-Series Forecasting & Sequential Data Analysis (NLP)

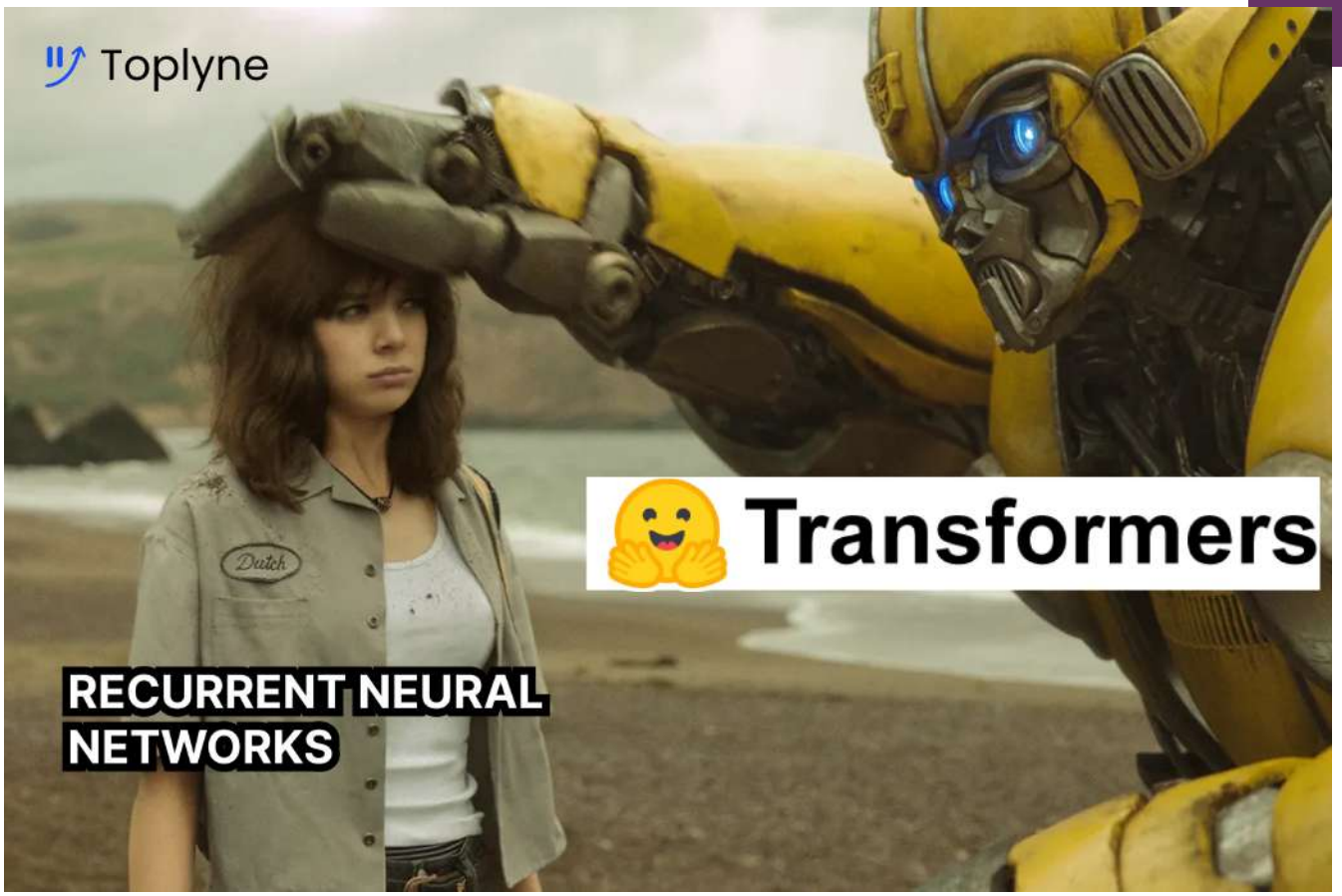


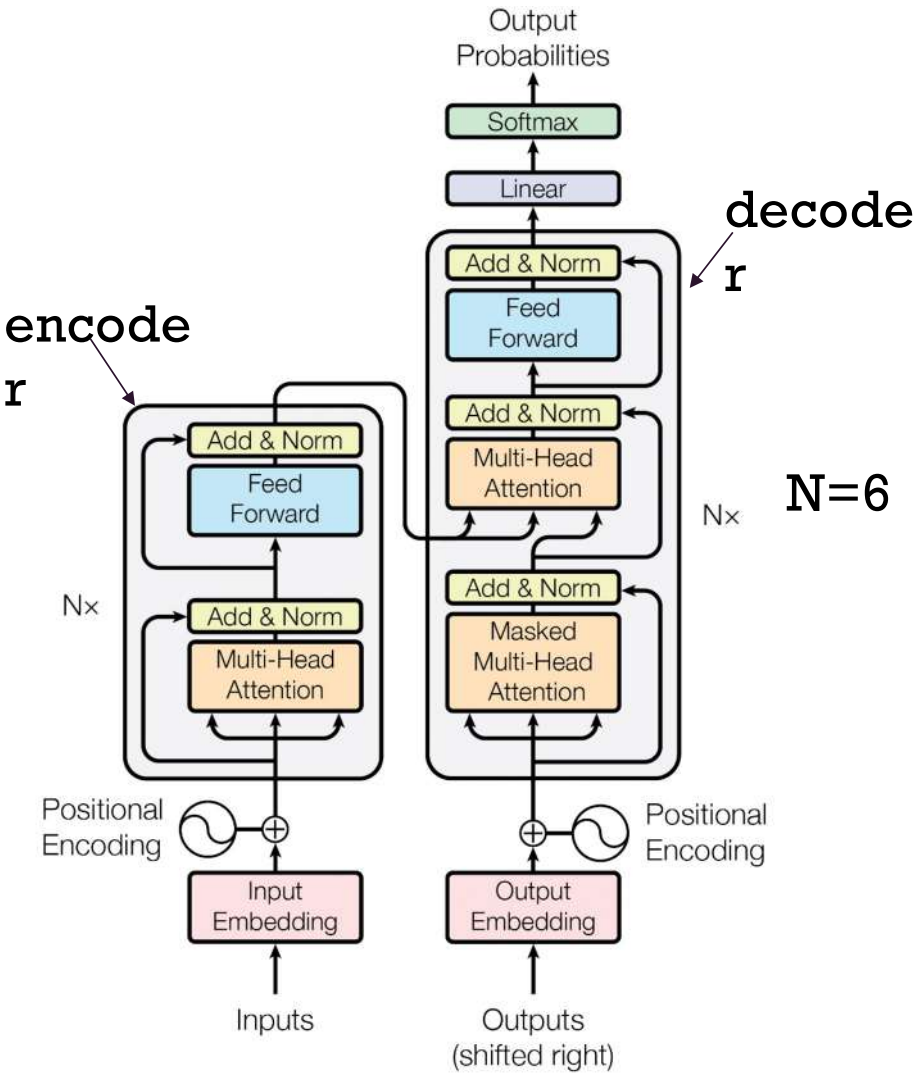



**Credit Prof.Ekapol's slide**

# 2) Rise of Transformer (2017)

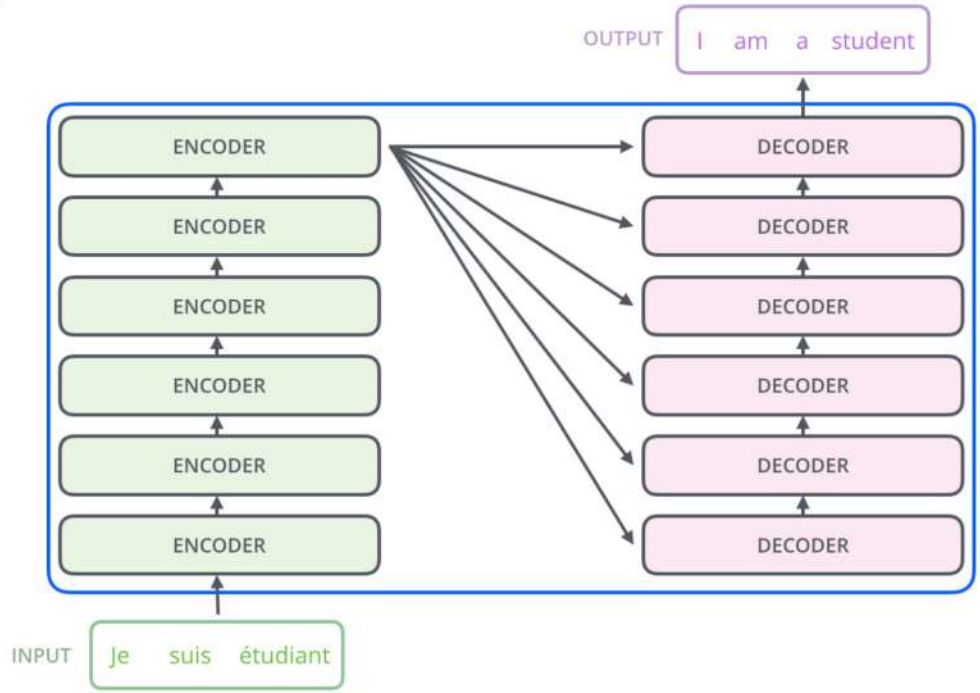


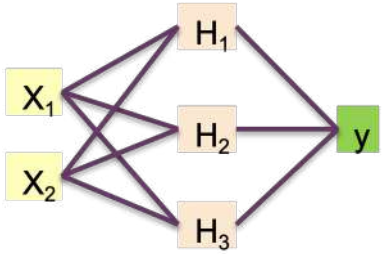






N=6

=



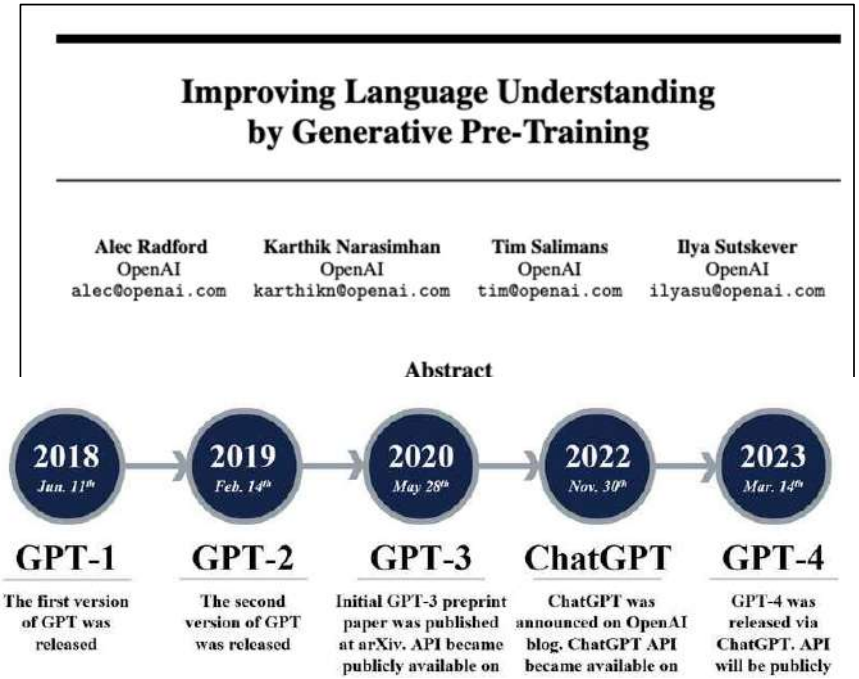




http://jalammar.github.io/illustrated-transformer/

# Transformer-Based Models

## Decoder-based model: GPT

- OpenAI GPT (Generative Pre-Training) [Radford, 2018]



**Improving Language Understanding by Generative Pre-Training**

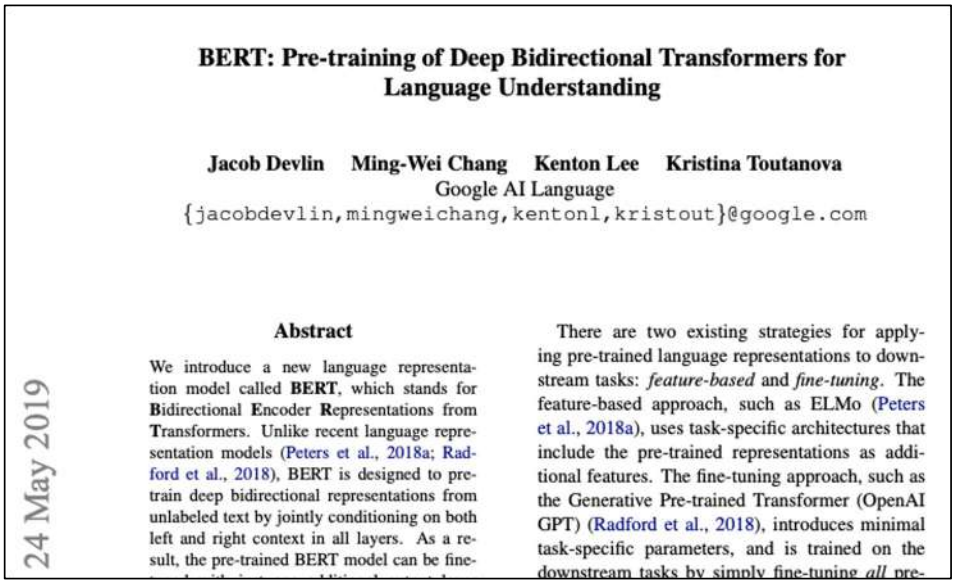

**Alec Radford** OpenAI alec@openai.com
**Karthik Narasimhan** OpenAI karthikn@openai.com
**Tim Salimans** OpenAI tim@openai.com
**Ilya Sutskever** OpenAI ilyasu@openai.com


**Abstract**



## Encoder-based model: BERT

- BERT [Devlin, et al, 2018]: Bidirectional Encoder Representation from Transformers



**BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding**


**Jacob Devlin** **Ming-Wei Chang** **Kenton Lee** **Kristina Toutanova**
Google AI Language
{jacobdevlin,mingweichang,kentonl,kristout}@google.com

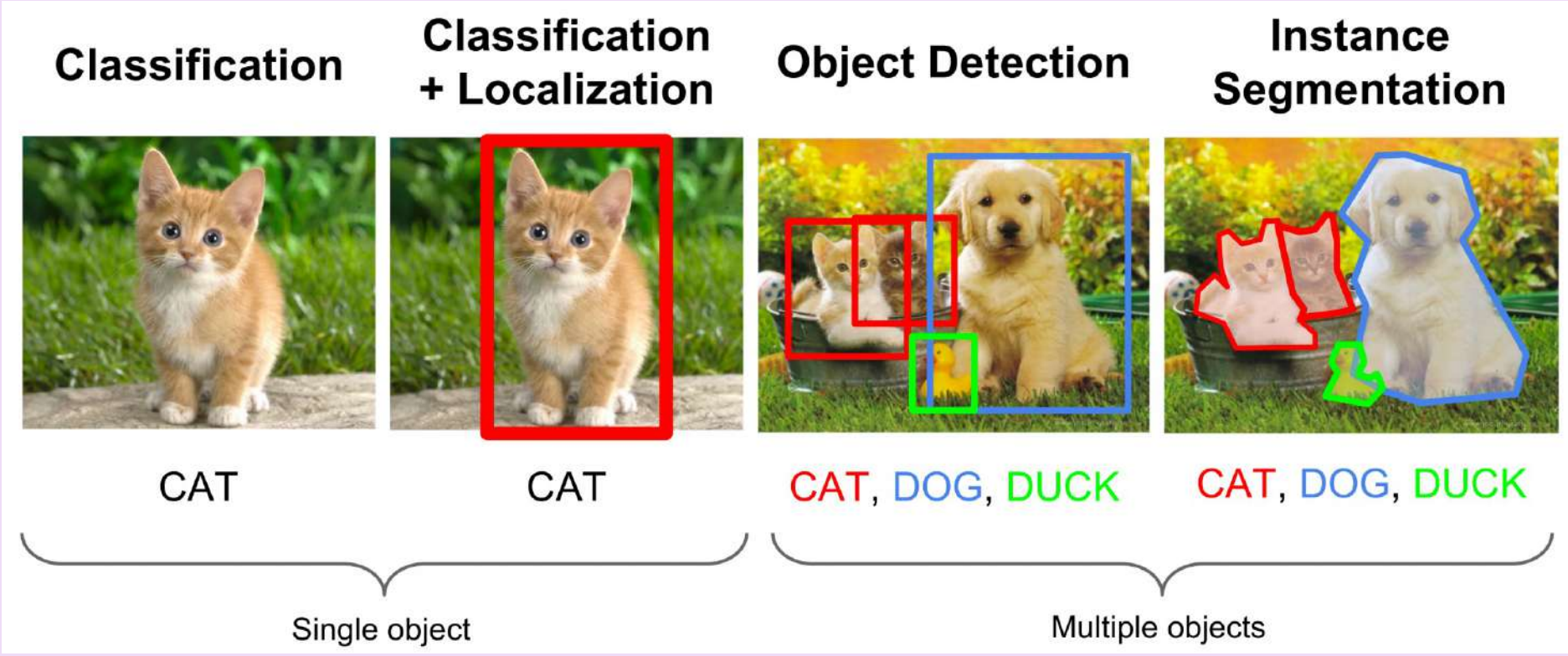

24 May 2019

**Abstract**


We introduce a new language representation model called **BERT**, which stands for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-

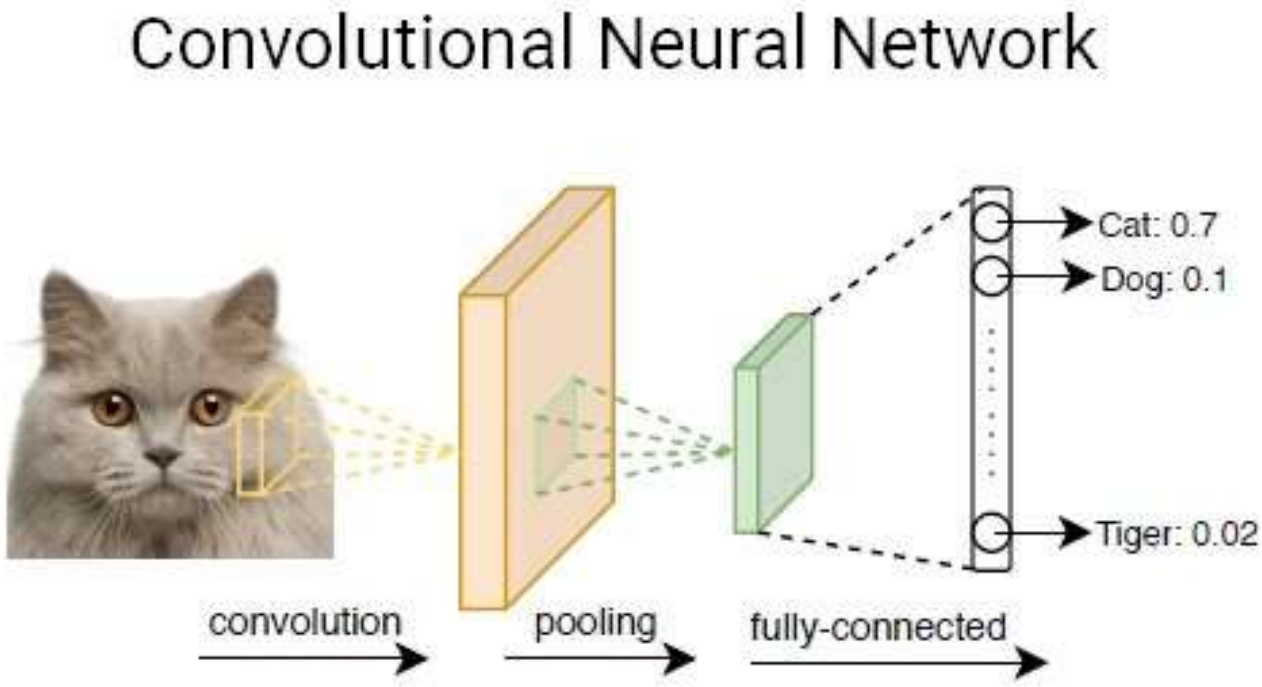
There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-

# Image Classification Tasks

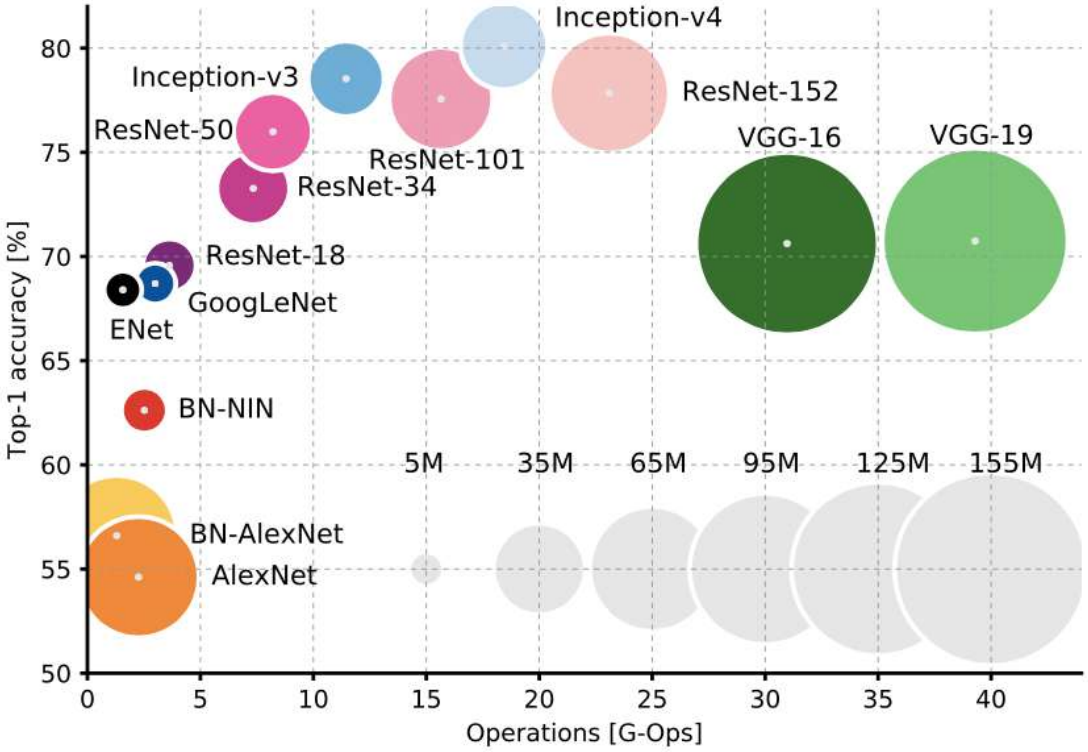



https://analyticsindiamag.com/top-5-image-classification-research-papers-every-data-scientist-should-know/

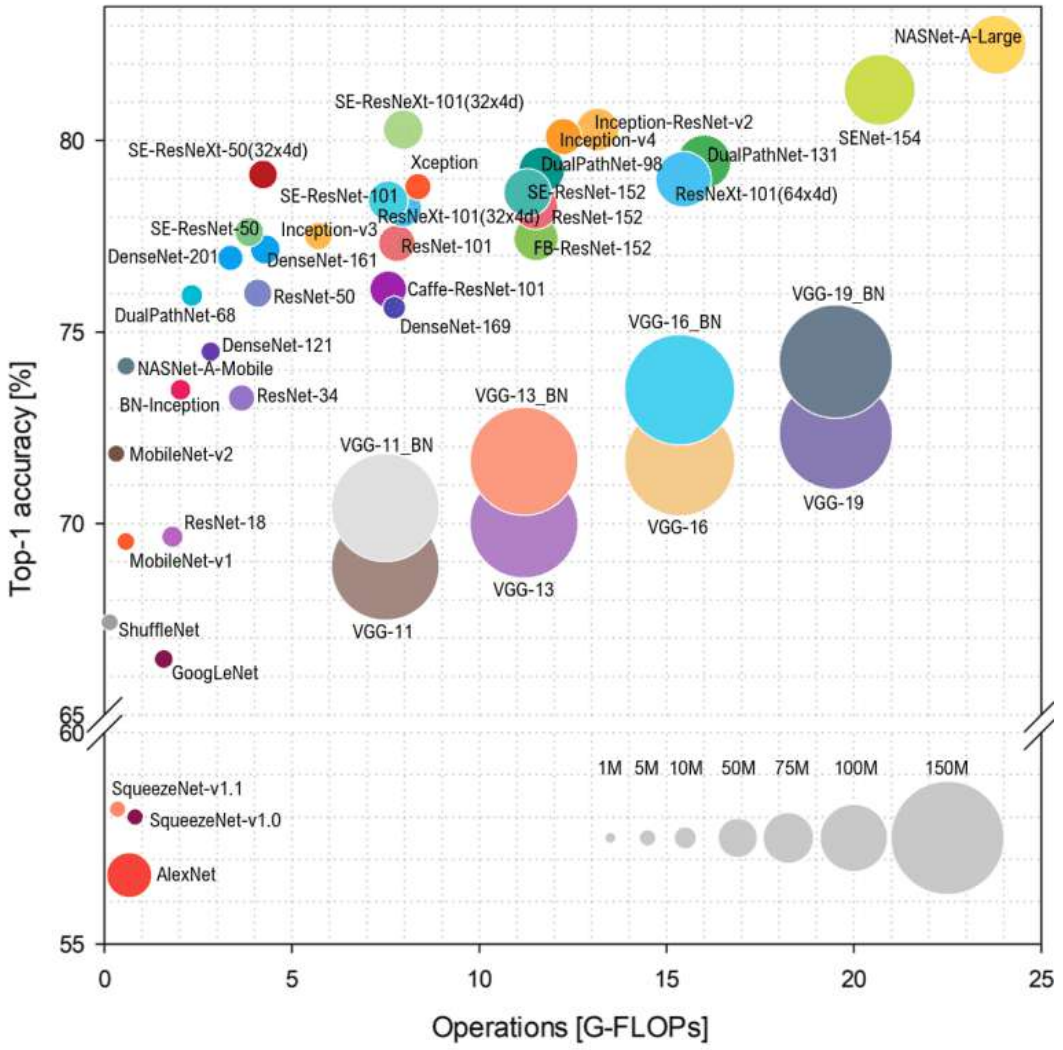
# Image Classification (recap)



# SOTA of Image Classification



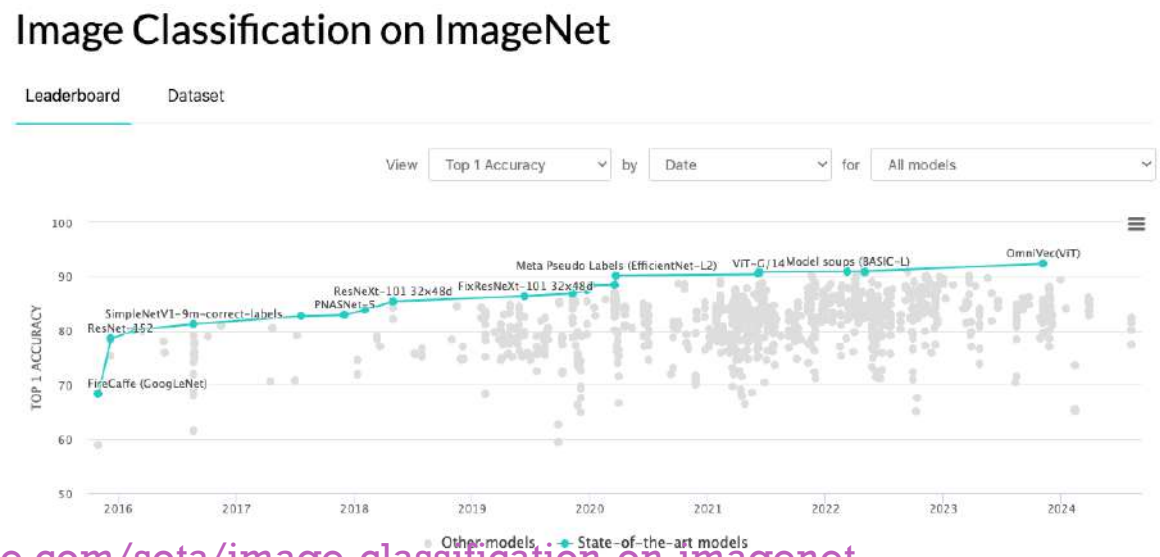

https://blog.csdn.net/qq_34216467/article/details/83061692




https://theaisummer.com/cnn-architectures/

# Common Image Classification Models

| Model | Year | Team / Organization |
|---|---|---|
| ResNet | 2015 | Microsoft Research |
| **MobileNet** | **2017** | **Google** |
| **EfficientNet** | **2019** | **Google** |
| Vision Transformer (ViT) | 2020 | Google Research |
| Swin Transformer | 2021 | Microsoft Research |
| ConvNeXt | 2022 | Meta AI |




https://paperswithcode.com/sota/image-classification-on-imagenet

# Image classfication dashboard (2026)

1. **CNN / ViT → Train/Fine-tune**
2. **DINOv3 → Freeze backbone + Train head**
3. **CLIP / SigLIP → Zero-shot**

- CNN → Vision Transformer (ViT) → Foundation Models / VLMs
  - 2021–2022 → CNN → ViT: CNNs (e.g., EfficientNetV2) remained strong, while Vision Transformers increasingly dominated at scale.
  - 2022–2024 → Large ViTs + Vision–Language pretraining: CLIP & CoCa enabled powerful transferable representations.
  - 2025 → Foundation Vision Models: DINOv3 scaled self-supervised learning to provide strong general-purpose visual representations.
  - 2025–2026 → Foundation Models + VLMs: Vision-only models (DINOv3) and Vision–Language models (SigLIP 2, CLIP-family) coexist as major pretraining paradigms.

- 2026 Key Takeaway:
  - Retrain a foundation encoder → adapt / fine-tune, rather than train an image classifier from scratch.
  - CNN / ConvNeXt → Strong inductive bias; efficient and still relevant.
  - ViT / DINOv3 → Vision-only (image); self-supervised foundation encoder with transferable features.
  - CLIP / SigLIP 2 / CoCa → Vision–Language (image + text); supports zero-shot classification, retrieval, and multimodal tasks.

https://paperswithcode.com/sota/image-classification-on-imagenet

# EfficientNet (May, 2019) → EffNetV2 (2021)

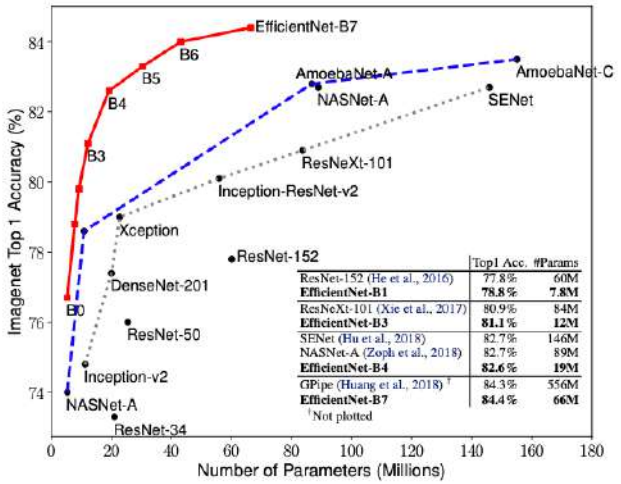
**EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks**

Mingxing Tan [1] Quoc V. Le [1]

**Abstract**

Convolutional Neural Networks (ConvNets) are commonly developed at a fixed resource budget, and then scaled up for better accuracy if more resources are available. In this paper, we systematically study model scaling and identify that carefully balancing network depth, width, and resolution can lead to better performance. Based on this observation, we propose a new scaling method that uniformly scales all dimensions of depth/width/resolution using a simple yet highly effective *compound coefficient*. We demonstrate the effectiveness of this method on scaling up MobileNets and ResNet.

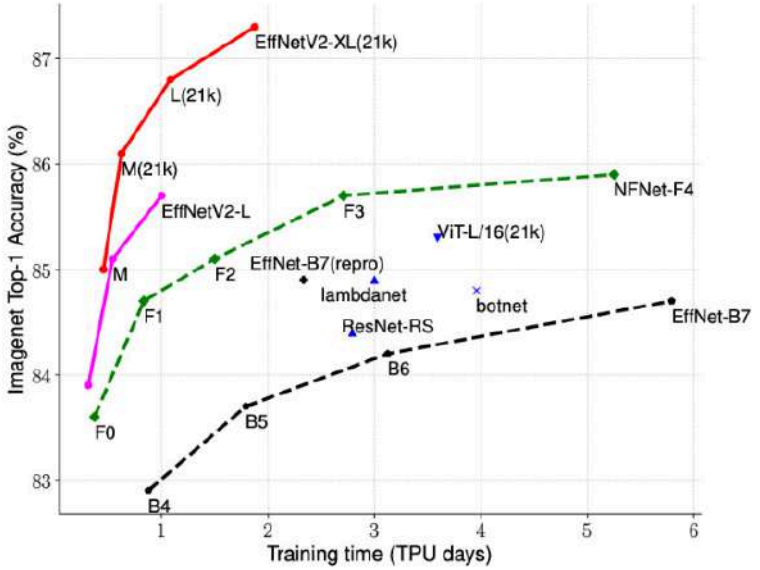
To go even further, we use neural architecture




| | Top1 Acc. | #Params |
|---|---|---|
| ResNet-152 (He et al., 2016) | 77.8% | 60M |
| **EfficientNet-B1** | **78.8%** | **7.8M** |
| ResNeXt-101 (Xie et al., 2017) | 80.9% | 84M |
| **EfficientNet-B3** | **81.1%** | **12M** |
| SENet (Hu et al., 2018) | 82.7% | 146M |
| NASNet-A (Zoph et al., 2018) | 82.7% | 89M |
| **EfficientNet-B4** | **82.6%** | **19M** |
| GPipe (Huang et al., 2018) † | 84.3% | 556M |
| **EfficientNet-B7** | **84.4%** | **66M** |

†Not plotted

*Figure 1. Model Size vs. ImageNet Accuracy. All numbers are*




(a) Training efficiency.

| | EfficientNet (2019) | ResNet-RS (2021) | DeiT/ViT (2021) | EfficientNetV2 (ours) |
|---|---|---|---|---|
| Top-1 Acc. | 84.3% | 84.0% | 83.1% | 83.9% |
| Parameters | 43M | 164M | 86M | 24M |

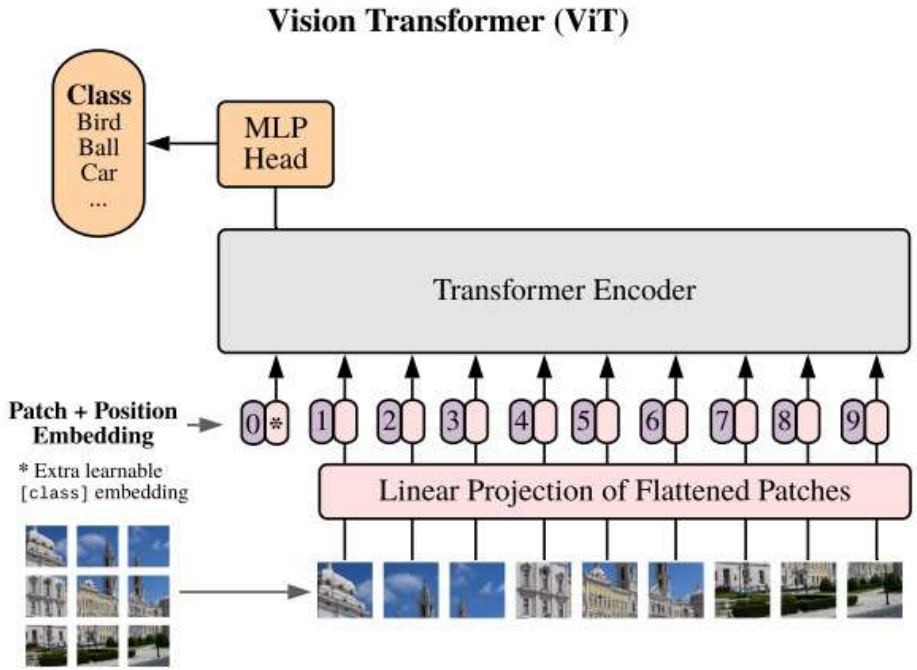
(b) Parameter efficiency.

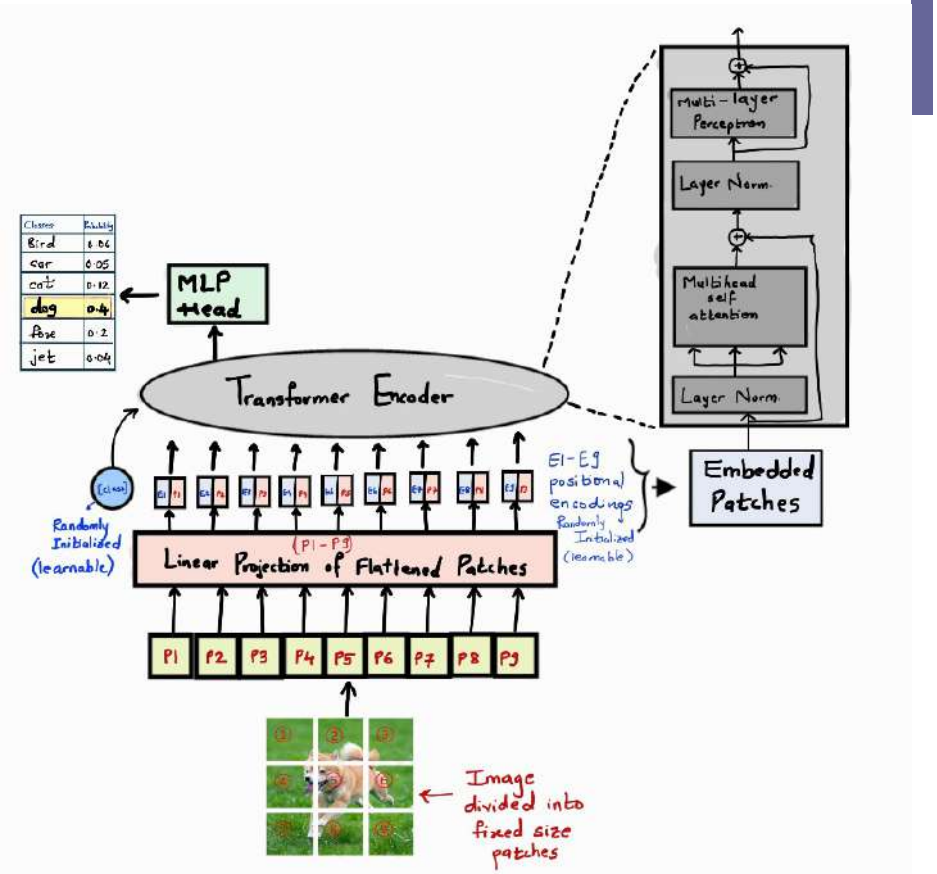
http://proceedings.mlr.press/v97/tan19a/tan19a.pdf

https://twitter.com/TensorFlow/status/1423359562724175873/photo/1

# Vision Transformer (ViT) (ICLR2021)

Hugging Face




https://medium.com/machine-intelligence-and-deep-learning-lab/vit-vision-transformer-cc56c8071a20

Open Source

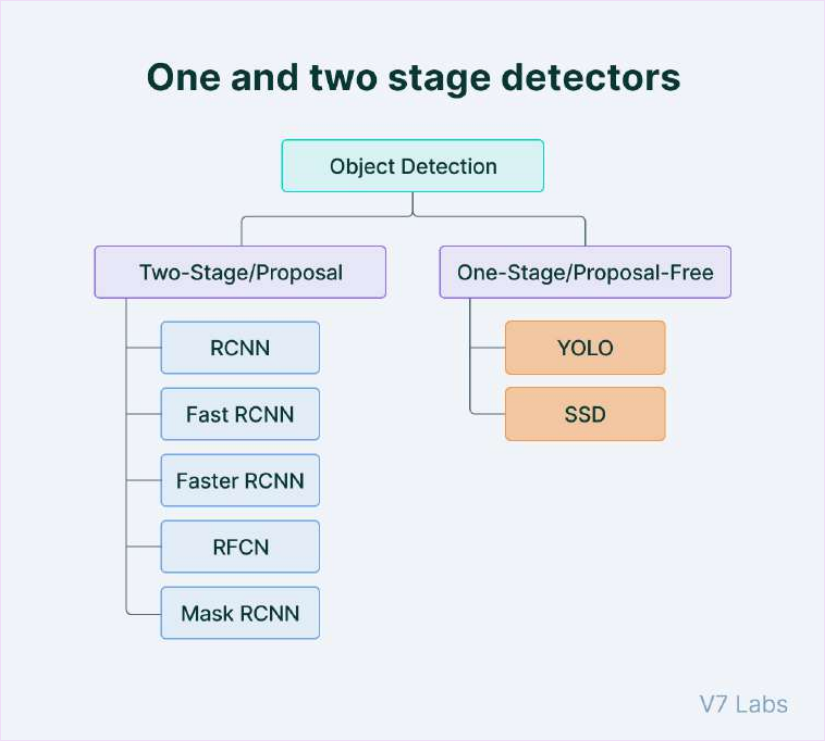
# DINOv3: Self-supervised learning for vision at unprecedented scale

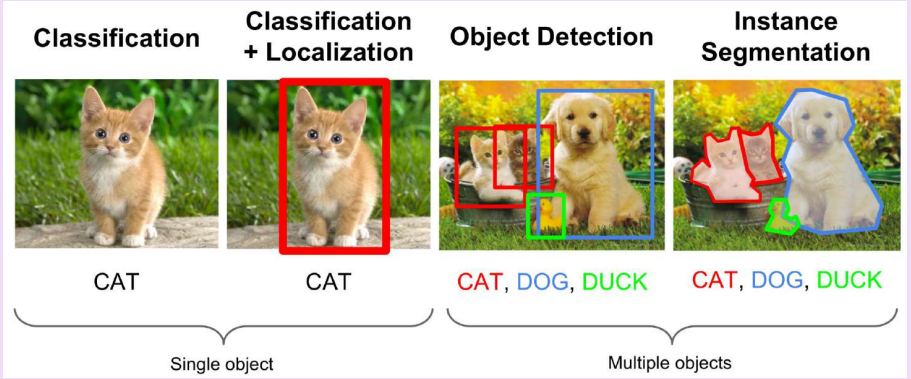
August 14, 2025 · 11 minute read

- DINOv3: Self-Supervised Vision Transformer (Meta AI, 2025)
- Vision-only (encoder-only) backbone (not multimodal, unlike CLIP/CoCa).
- Self-supervised learning → no human labels needed.
- Massive scale: up to 7B parameters, trained on 1.7B images.
- Key innovations:
  - Gram Anchoring → stabilizes feature learning.
  - Axial RoPE → improved positional encoding.
  - Resolution robustness → better handling of varied image sizes.
- Strengths:
  - Provides general-purpose visual representations.
  - Excels in downstream tasks (classification, detection, segmentation).
  - Foundation model role for vision, similar to LLMs in NLP.

Link

# SOTA of Object Detection

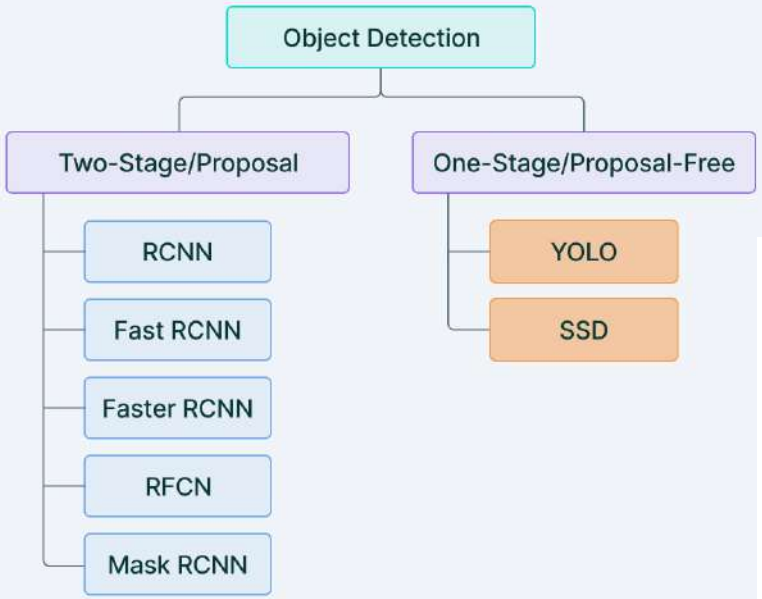



https://medium.com/@jonathan_hui/real-time-object-detection-with-yolo-yolov2-28b1b93e2088

# Object Detection

## One and two stage detectors







| | | units | |
|---|---|---|---|
| | | absolute | normalized |
| corner coordinate | (left, top, right, bottom) | Pascal VOC | Albumentations |
| | (left, top, width, height) | COCO | |
| | (center_x, center_y, width, height) | CreateML | YOLO |

https://dragoneye.ai/blog/a-guide-to-bounding-box-formats/

# YOLO Open Images in New York

YOLO Open Images in New York

Joseph Redmon
6.95K subscribers
Subscribe

13,911 views Nov 13, 2017
*No description has been added to this video.*

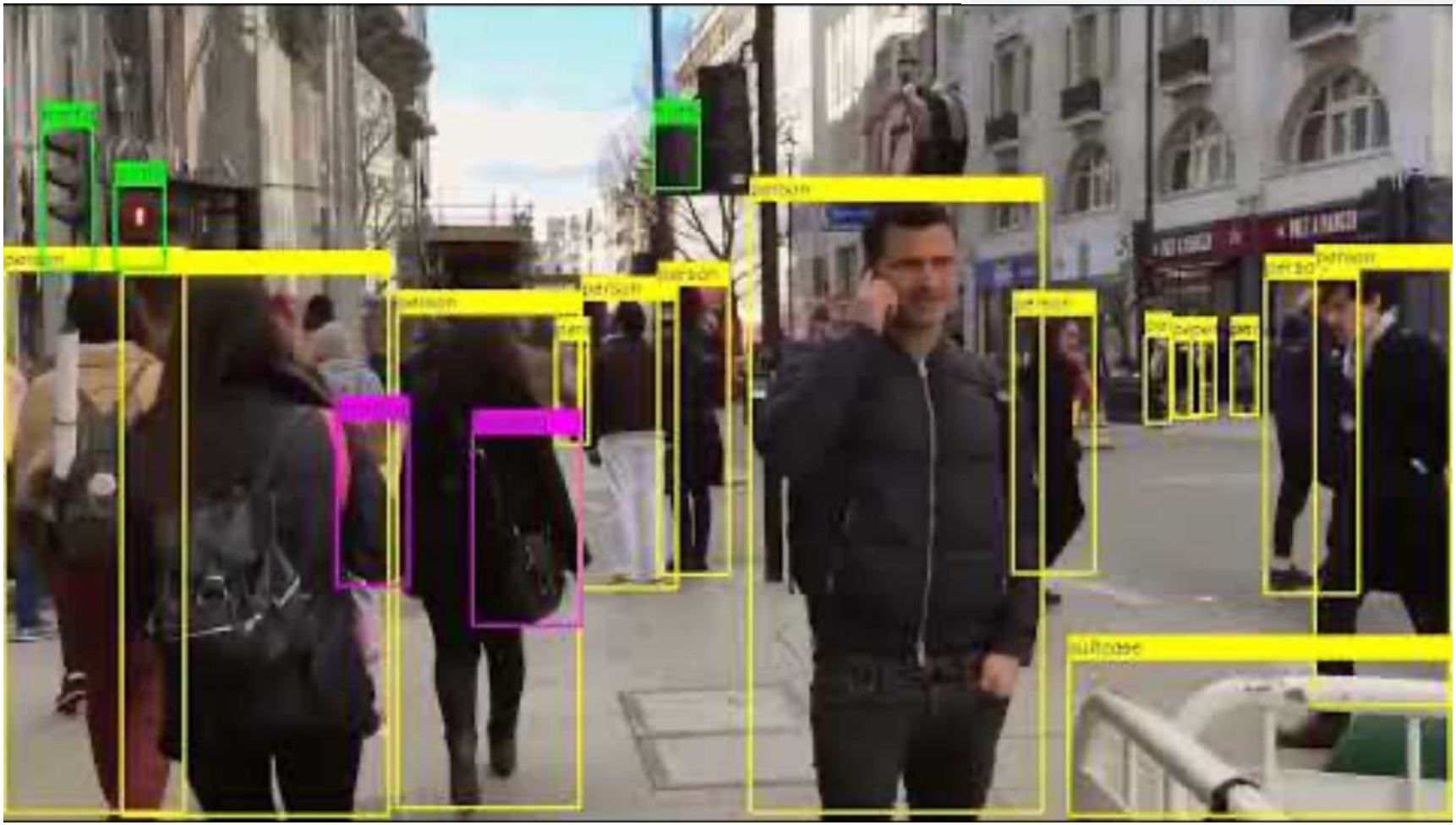


# YOLO v5

**Example of YOLO v5 detection on video file**

Luiz doleron
127 subscribers
Subscribe

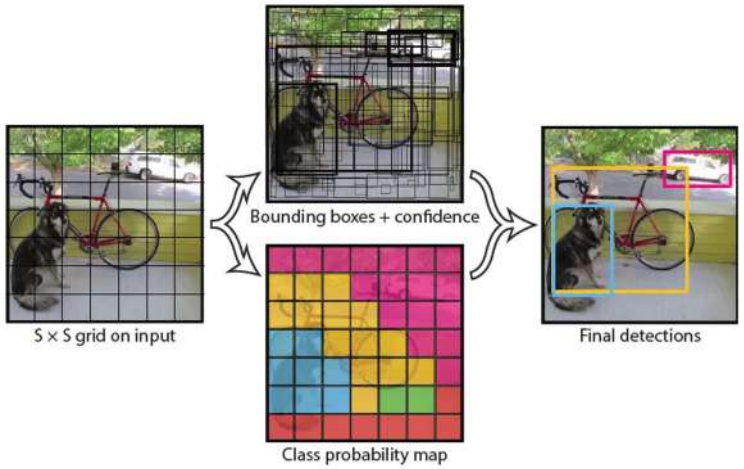
5,781 views Jan 23, 2022
Video example of YOLOv5 performing object detection.
- Check out the explanation at / detecting-objects-with-yolov5-opencv-pytho...
- Github repository is here: https://github.com/doleron/yolov5-ope...



# What is YOLO?

**You Only Look Once (YOLO)** is a one-stage, end-to-end object detection framework that predicts bounding boxes and class probabilities in a single forward pass of a neural network.

Unlike earlier object detection approaches that adapted image classifiers into multi-step detection pipelines, YOLO treats object detection as a single regression problem.




https://medium.com/data-science/yolov1-you-only-look-once-object-detection-e1f3ffec8a89




## News

- We are pleased to announce the LVIS 2021 Challenge and Workshop to be held at ICCV.
- Please note that there will not be a COCO 2021 Challenge, instead, we encourage people to participate in the LVIS 2021 Challenge.
- We have partnered with the team behind the open-source tool **FiftyOne** to make it easier to download, visualize, and evaluate COCO
- **FiftyOne** is an open-source tool facilitating visualization and access to COCO data resources and serves as an evaluation tool for model analysis on COCO.

## What is COCO?



COCO is a large-scale object detection, segmentation, and captioning dataset. COCO has several features:

- **Object segmentation**
- **Recognition in context**
- **Superpixel stuff segmentation**
- **330K images (>200K labeled)**
- **1.5 million object instances**
- **80 object categories**
- **91 stuff categories**
- **5 captions per image**
- **250,000 people with keypoints**

## Collaborators

**Tsung-Yi Lin** Google Brain
**Genevieve Patterson** MSR, Trash TV
**Matteo R. Ronchi** Caltech
**Yin Cui** Google
**Michael Maire** TTI-Chicago
**Serge Belongie** Cornell Tech
**Lubomir Bourdev** WaveOne, Inc.
**Ross Girshick** FAIR
**James Hays** Georgia Tech
**Pietro Perona** Caltech
**Deva Ramanan** CMU
**Larry Zitnick** FAIR
**Piotr Dollár** FAIR

## Sponsors

CVDF
Microsoft
facebook
Mighty AI

https://cocodataset.org/#home

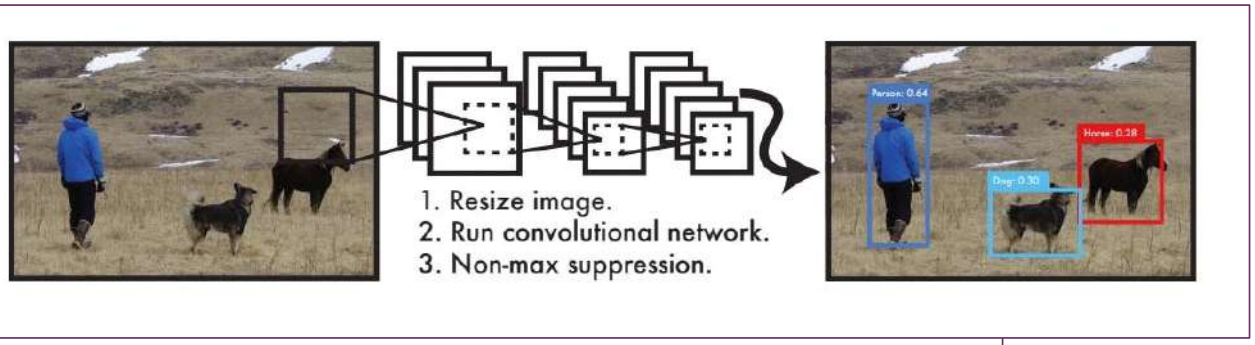
**ultralytics/cfg/datasets/coco8.yaml**

79 classes

```yaml
# Classes
names:
  0: person
  1: bicycle
  2: car
  3: motorcycle
  4: airplane
  5: bus
  6: train
  7: truck
  8: boat
  9: traffic light
  10: fire hydrant
  11: stop sign
  12: parking meter
  13: bench
  14: bird
  15: cat
  16: dog
  17: horse
  18: sheep
  19: cow
  20: elephant
  21: bear
  22: zebra
  23: giraffe
```

https://docs.ultralytics.com/datasets/detect/#ultralytics-yolo-format

# YOLO's output



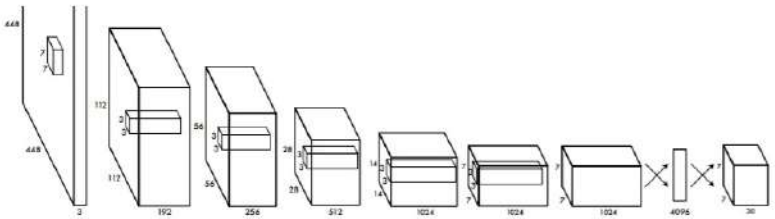
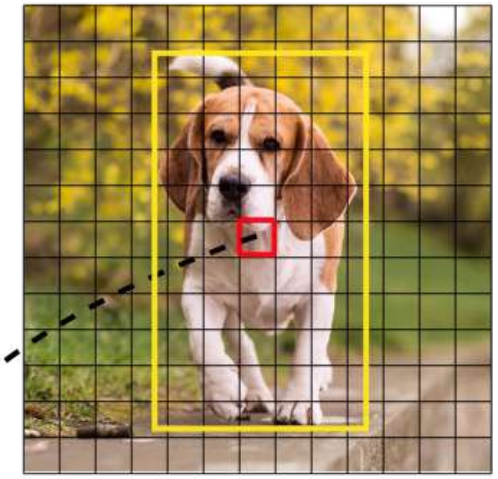
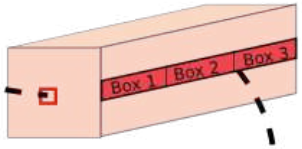




**Figure 3: The Architecture.** Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating $1 \times 1$ convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution ($224 \times 224$ input image) and then double the resolution for detection.

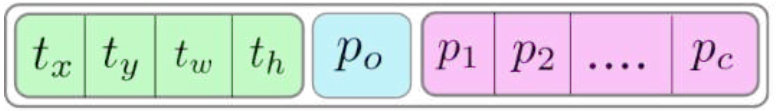
Image Grid. The Red Grid is responsible for detecting the dog



Prediction Feature Map




Attributes of a bounding box

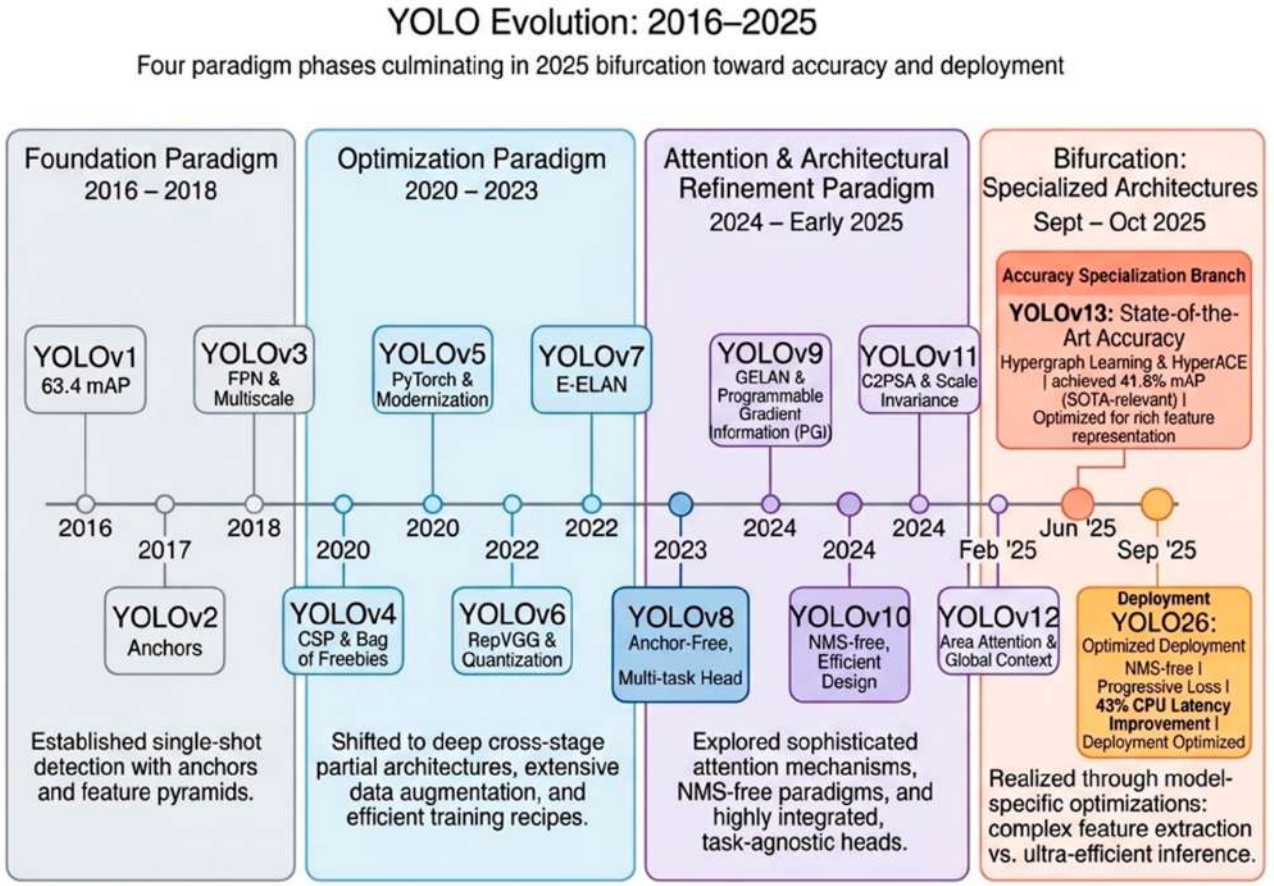



Box Co-ordinates | Objectness Score | Class Scores

https://blog.paperspace.com/how-to-implement-a-yolo-object-detector-in-pytorch/

https://www.preprints.org/manuscript/202607.1076

# The Evolution of YOLO: 2016 - 2025

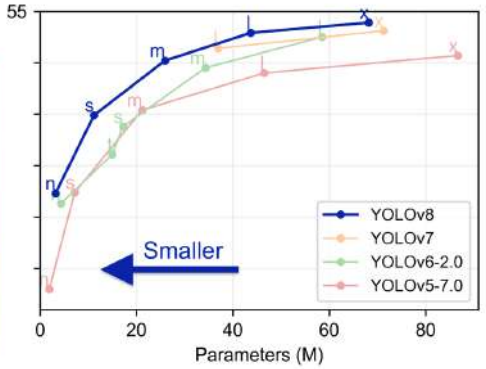


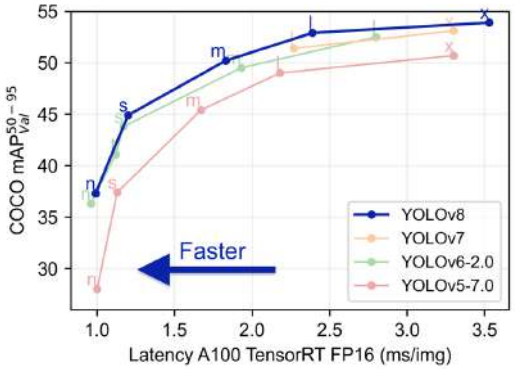

Ultralytics YOLO → production-ready, practical framework (YOLOv5 → YOLOv8 → YOLO11 → YOLO26).
Academic YOLO → new research architectures (YOLOv9, v10, v11, v12).

# Ultralytic's YOLO

## YOLOv5 (2020) → YOLOv8 (2023) → YOLO 11 (2024) → YOLO 26 (2025)




English | 简体中文

Ultralytics CI passing | codecov 87% | DOI 10.5281/zenodo.7347926 | docker pulls 22k

Run on Gradient | Open in Colab | Open in Kaggle

Ultralytics YOLOv8 is a cutting-edge, state-of-the-art (SOTA) model that builds upon the success of previous YOLO versions and introduces new features and improvements to further boost performance and flexibility. YOLOv8 is designed to be fast, accurate, and easy to use, making it an excellent choice for a wide range of object detection and tracking, instance segmentation, image classification and pose estimation tasks.

We hope that the resources here will help you get the most out of YOLOv8. Please browse the YOLOv8 Docs for details, raise an issue on GitHub for support, and join our Discord community for questions and discussions!

To request an Enterprise License please complete the form at Ultralytics Licensing.

### Models

YOLOv8 Detect, Segment and Pose models pretrained on the COCO dataset are available here, as well as YOLOv8 Classify models pretrained on the ImageNet dataset. Track mode is available for all Detect, Segment and Pose models.




All Models download automatically from the latest Ultralytics release on first use.

# YOLO Models (2024–2026) Timeline

| Model | Release | Team / University / Organization | Key Highlight |
|---|---|---|---|
| **YOLO26** | **Jan 2026** | **Ultralytics** | Native end-to-end, **NMS-free**; DFL-free head; MuSGD; optimized for edge deployment |
| **YOLO12** | Feb 2025 | **Yunjie Tian, Qixiang Ye (UCAS), David Doermann (University at Buffalo)** | **Attention-centric YOLO**; Area Attention + R-ELAN |
| **YOLO11** | Sep 2024 | **Ultralytics** | Improved accuracy–efficiency; detection, segmentation, pose, classification & OBB |
| **YOLOv10** | May 2024 | **Tsinghua University** | **End-to-end, NMS-free** detection; consistent dual assignments |
| **YOLOv9** | Feb 2024 | **Institute of Information Science, Academia Sinica, Taiwan** | **PGI + GELAN** for better gradient flow and efficiency |

**Evolution: Better gradients → End-to-end / NMS-free → Multi-task efficiency → Attention → Edge-first deployment**

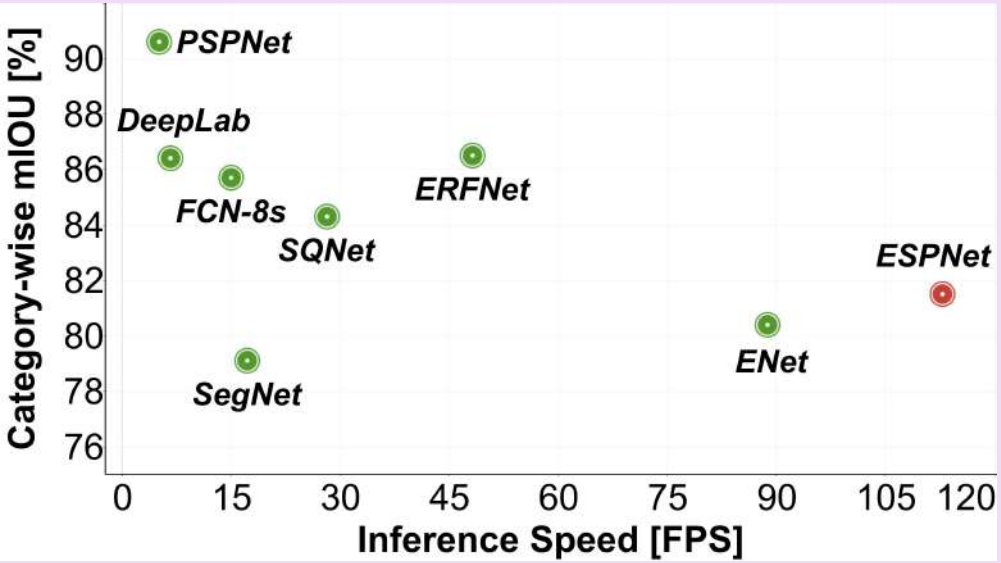
# Object Detection Landscape (2026)

- **NN-based → Transformer-based → End-to-End Detection**
  - 2016–2020 → CNN-based detectors: Faster R-CNN, SSD, YOLO established two-stage vs. one-stage detection.
  - 2020–2022 → Transformer detectors: DETR / Deformable DETR introduced end-to-end set prediction without anchors or traditional NMS.
  - 2023–2024 → Real-time Transformers: RT-DETR brought Transformer-based detection to real-time applications.
  - 2024–2025 → End-to-End YOLO: YOLOv10 / YOLO11 / YOLO12 improved efficiency, attention, and end-to-end detection.
  - 2026 → Efficient End-to-End Detection: YOLO26 emphasizes NMS-free inference and edge deployment.

- **2026 Key Takeaway**
  - Modern object detection is shifting toward end-to-end prediction: Image → Boxes + Classes
  - Faster R-CNN → Two-stage CNN; region proposals + classification.
  - YOLO → One-stage, real-time detector; modern versions increasingly end-to-end.
  - DETR / RT-DETR → Transformer-based, end-to-end object detection.
  - YOLO26 → End-to-end + NMS-free + edge-optimized real-time detection.

- **Evolution:**
  - Two-stage CNN → One-stage YOLO → Transformer (DETR) → End-to-End / NMS-free Detection

# SOTA of Semantic Segmentation

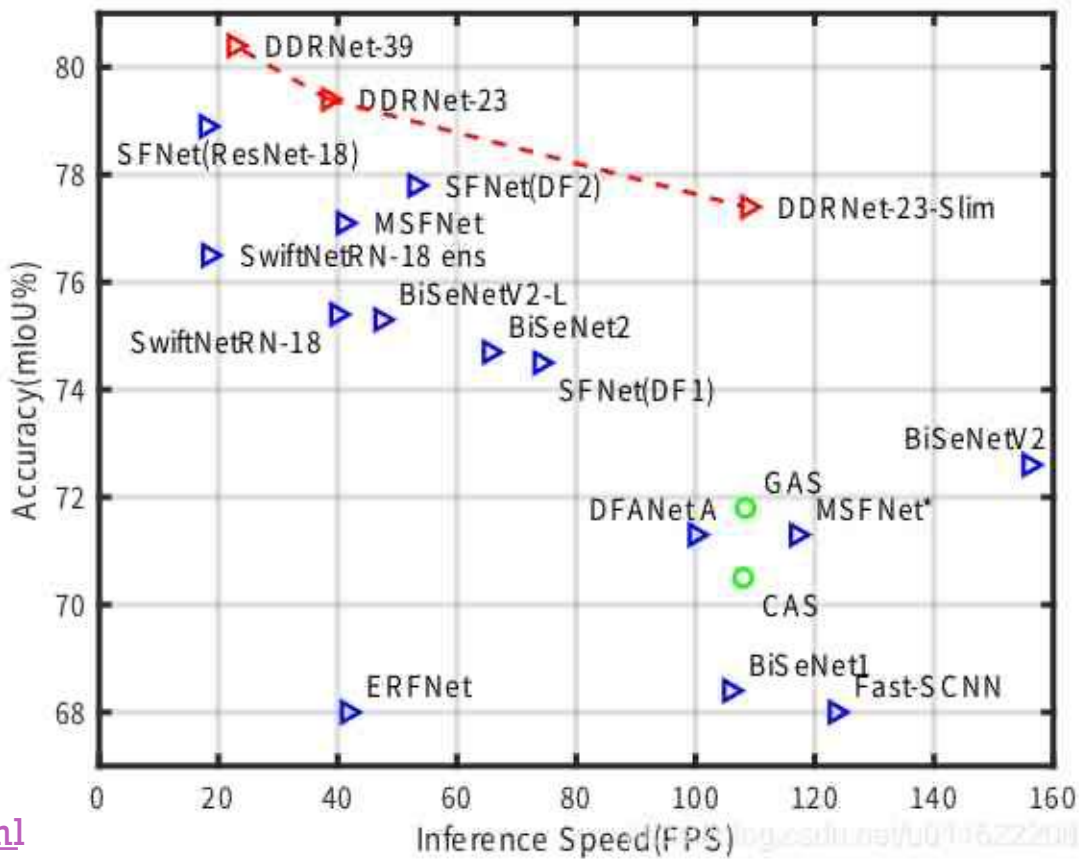



- **Real-time semantic segmentation**
- BiSeNet V1,V2
- DDRNet

https://chowdera.com/2021/11/20211109042359120o.html

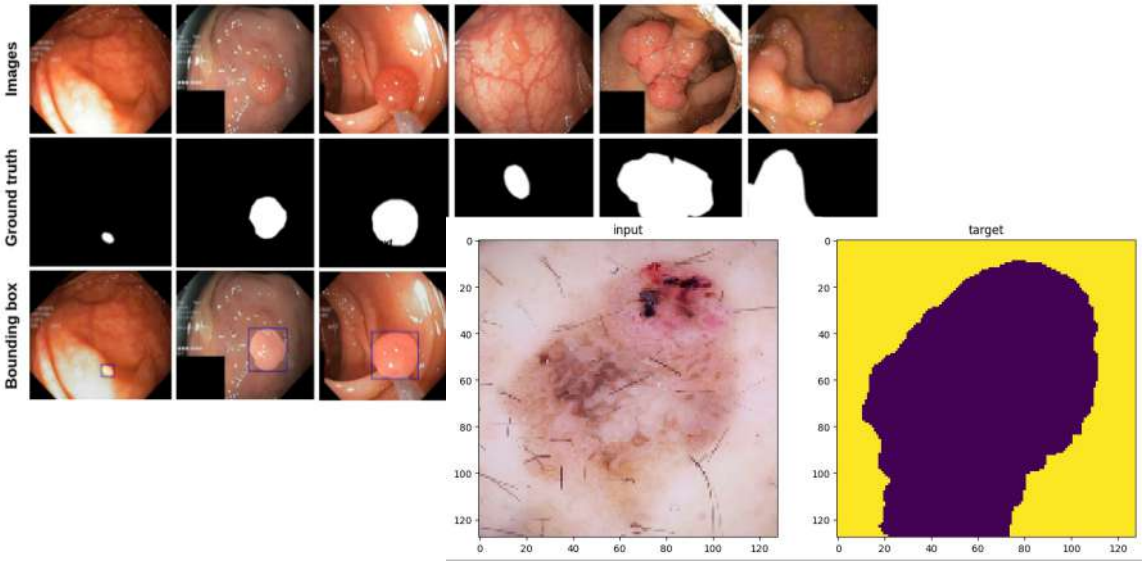
[Personal open source] - -
Real - time Semantic Segmentation ddrnet

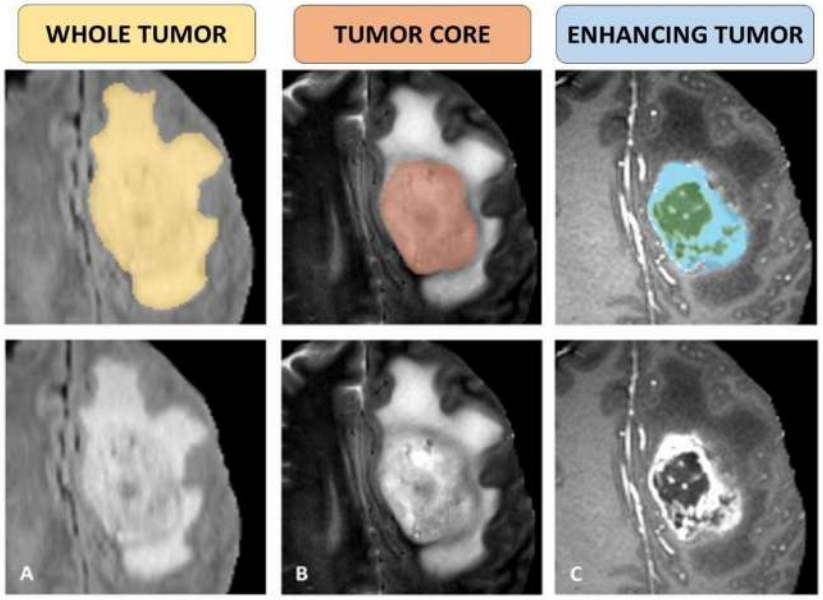
2021-11-09 04:24:01 【Nongfu Mountain Spring 2】



# Semantic Segmentation

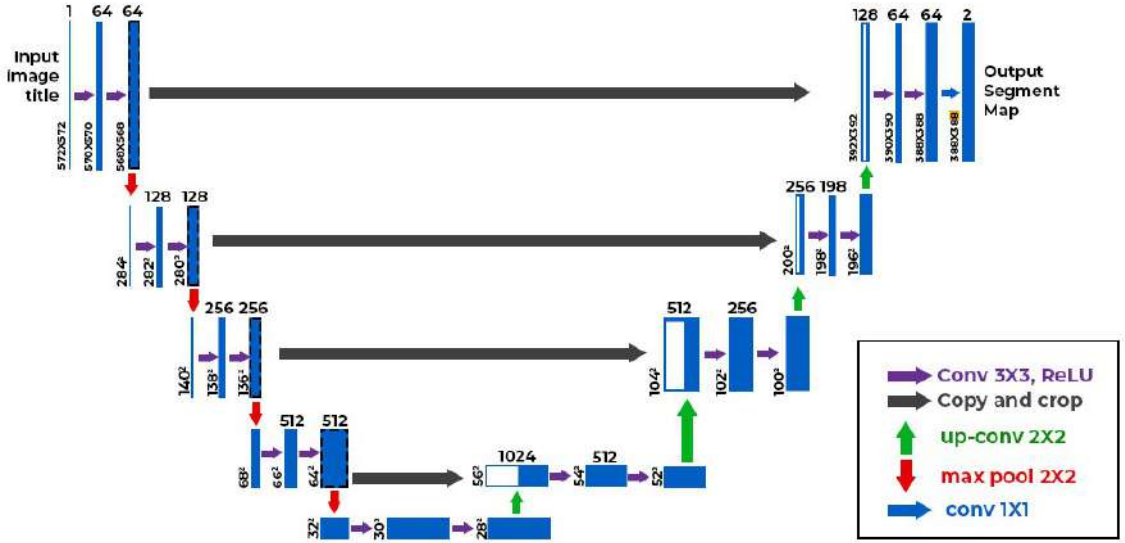
- Semantic segmentation = pixel-wise classification
- Each pixel gets a class label (road, car, polyp, tumor, sky, background).



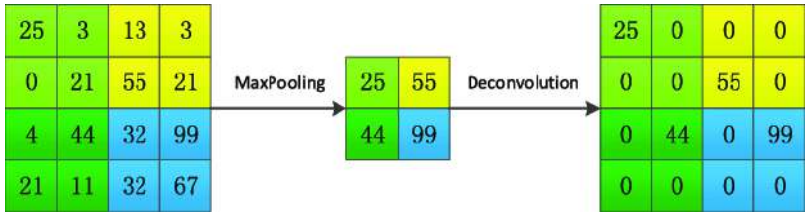



# UNet: Encoder-Decoder Network: Encoder, Decoder, Skip Connections

- **U-Net**




Deconvolutional layer




$\mathcal{L} = \lambda_1 \cdot \mathrm{CE} + \lambda_2 \cdot \mathrm{Dice}$ ; CE (pixel loss) & Dice (region loss; semantic)

Architecture of the U-net for a given input image. The blue boxes correspond to feature maps blocks with their denoted shapes. The white boxes correspond to the copied and cropped feature maps.
Source: O. Ronneberger et al. (2015)

# Image Segmentation Landscape (2026)

- **CNN → Transformer → Foundation Models**
  - 2015–2020 → CNN-based: FCN, U-Net, DeepLab → encoder–decoder architectures for semantic segmentation.
  - 2021–2022 → Transformer-based: SegFormer, Swin, Mask2Former → global attention; semantic, instance & panoptic segmentation.
  - 2023 → Foundation Models: SAM → introduced promptable segmentation using points, boxes, or masks.
  - 2024 → Image + Video: SAM 2 → extended promptable segmentation to images and videos.
  - 2025–2026 → Foundation Vision: DINOv3 + segmentation head → strong pretrained features for dense prediction.

- 2026 Key Takeaway
  - Task-specific training → Pretrained foundation models + lightweight adaptation / prompting
  - U-Net / DeepLab → CNN-based → Train / Fine-tune
  - SegFormer / Mask2Former → Transformer-based → Train / Fine-tune
  - DINOv3 + Head → Freeze backbone + Train segmentation head
  - SAM / SAM 2 → Promptable segmentation / Zero-shot

- Evolution:
  - CNN → Transformer → Foundation Model → Promptable Segmentation

# Introducing SAM 2: The next generation of Meta Segment Anything Model for videos and images

July 29, 2024 · 15 minute read

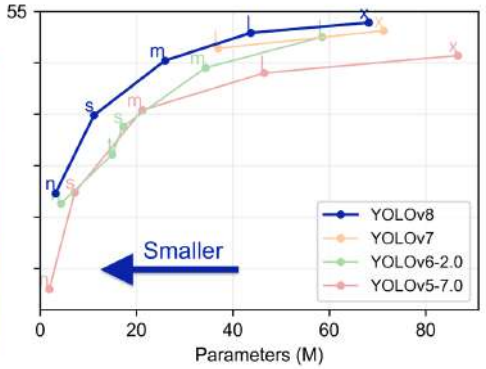


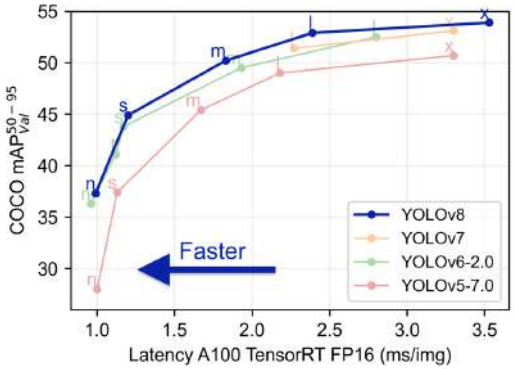

https://ai.meta.com/blog/segment-anything-2/

# Ultralytic's YOLO (extra segmentation model)

## YOLOv5 (2020) → YOLOv8 (2023) → Ultralytics YOLO 11 (2024)




English | 简体中文

Ultralytics CI passing | codecov 87% | DOI 10.5281/zenodo.7347926 | docker pulls 22k

Run on Gradient | Open in Colab | Open in Kaggle

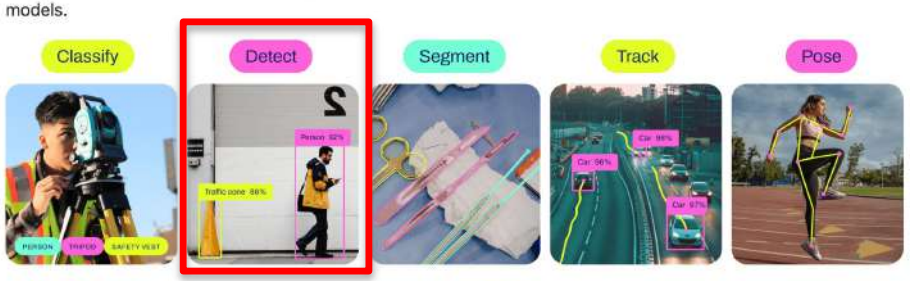
Ultralytics YOLOv8 is a cutting-edge, state-of-the-art (SOTA) model that builds upon the success of previous YOLO versions and introduces new features and improvements to further boost performance and flexibility. YOLOv8 is designed to be fast, accurate, and easy to use, making it an excellent choice for a wide range of object detection and tracking, instance segmentation, image classification and pose estimation tasks.

We hope that the resources here will help you get the most out of YOLOv8. Please browse the YOLOv8 Docs for details, raise an issue on GitHub for support, and join our Discord community for questions and discussions!

To request an Enterprise License please complete the form at Ultralytics Licensing.

### Models

YOLOv8 Detect, Segment and Pose models pretrained on the COCO dataset are available here, as well as YOLOv8 Classify models pretrained on the ImageNet dataset. Track mode is available for all Detect, Segment and Pose models.




All Models download automatically from the latest Ultralytics release on first use.

# YOLOv8-Seg: Object Detection → Segment

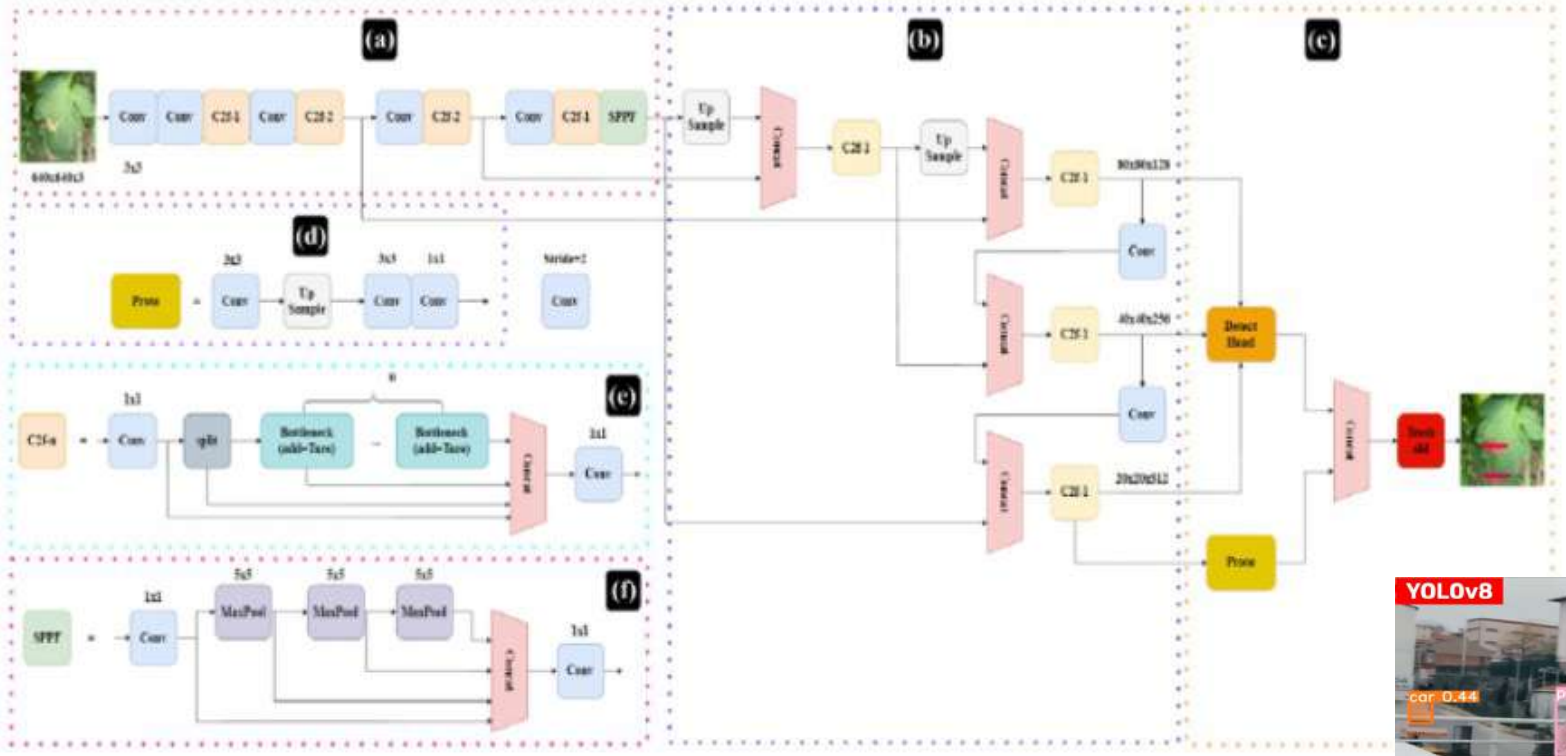
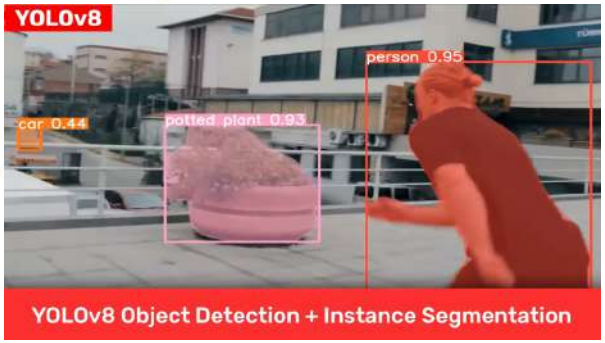


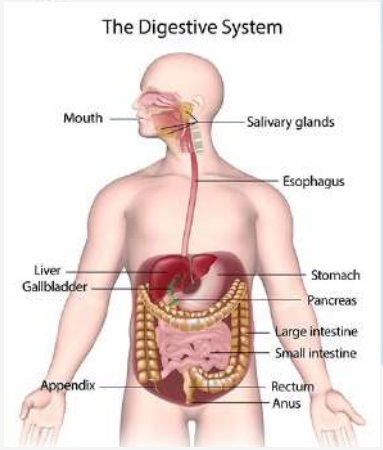

Fig. 3 YOLOv8-Seg architecture; **a** backbone, **b** neck, **c** head, **d** Proto module, **e** C2f module and **f** SPPF module



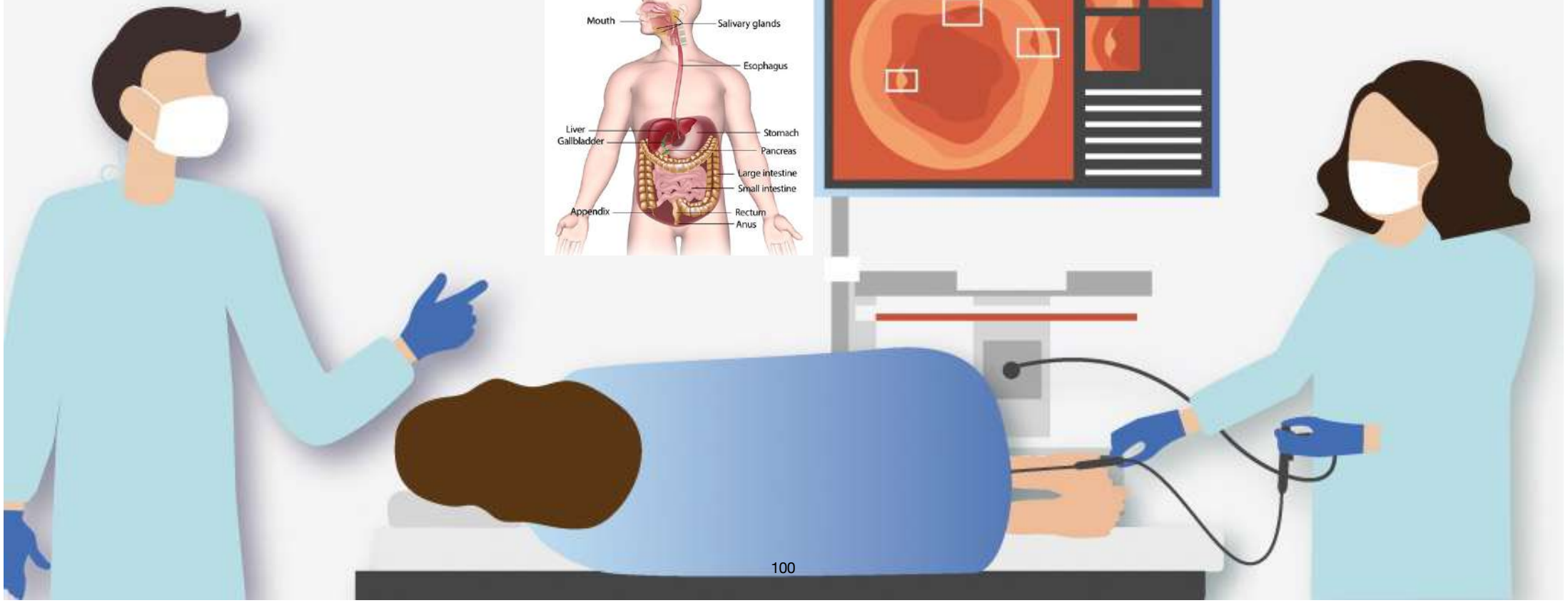

Link

# Real-Time Colonic Polyp Detection

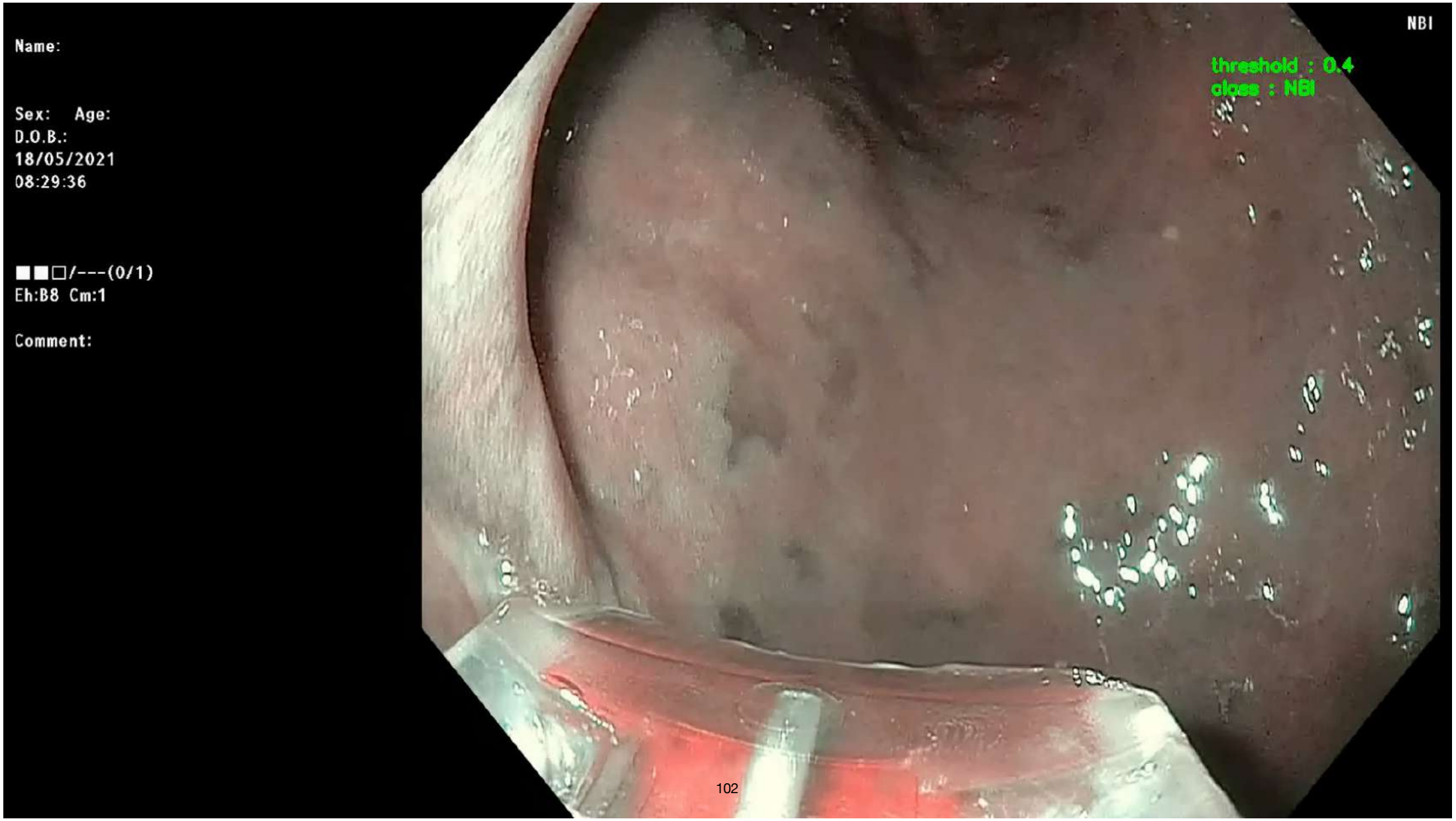
An AI-assisted solution that aims to improve colonic polyp detection in real-time. It is compatible with all standard colonoscope systems.



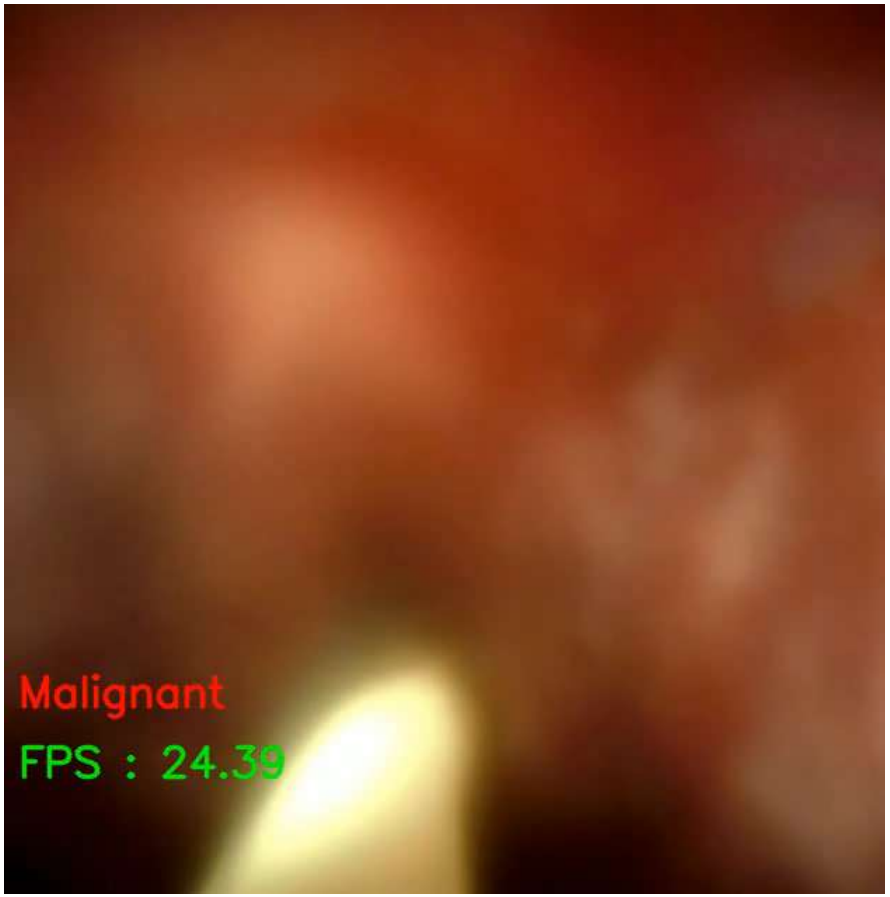


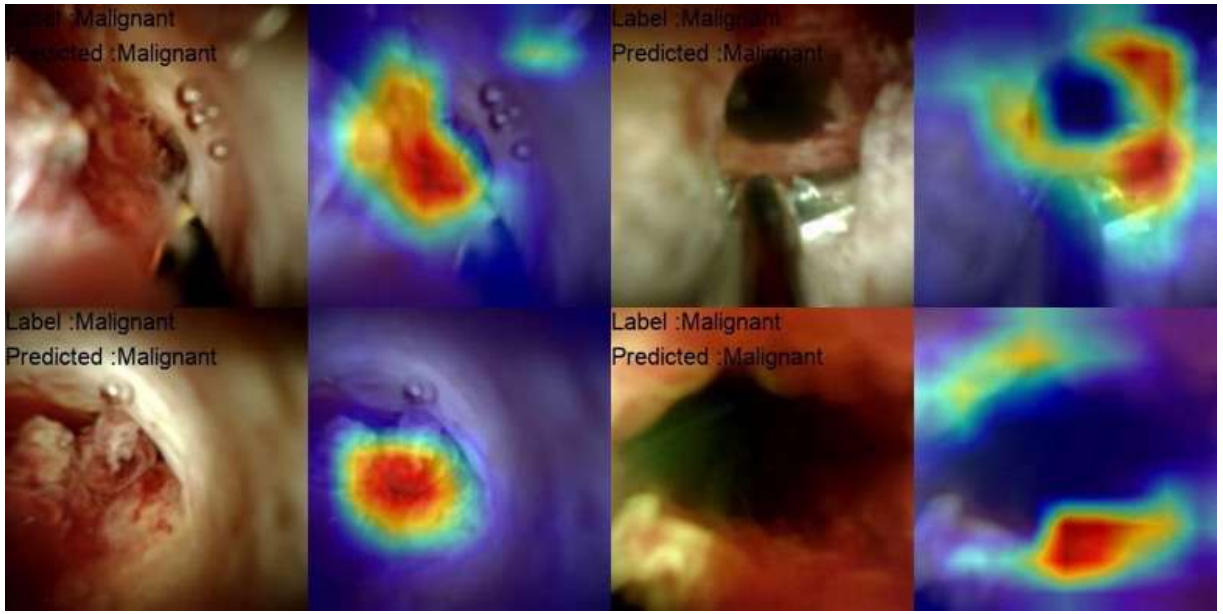




09/11/2021
13:53:24
COLON
*1/200
Lv+2 AUTO
HT NR
SE
3.8
12.0
12.0
S1: F+T
S2: LM
S3: CAD
S4:
EC-760R-V/L
3C729K035
BL-7000
CHULALONGKORN HOS


Name:
Sex: Age:
D.O.B.:
18/05/2021
08:29:36
■■□/---(0/1)
Eh:B8 Cm:1
Comment:
NBI
threshold : 0.4
class : NBI

# Cholangioscopy



# Appendix